The Canonical Society Theorem: Technical Supplement

Jian Gao (高健)

Independent researcher · innersummer@hotmail.com · ORCID 0009-0009-5002-1655

Contents

1 Objects and use of the supplement 3

Appendix A Actor Generation, Root-Law Completeness, and Recourse after First Refusal 4

Appendix B Actual responsibility birth, exit, and same-occurrence regeneration 14

Appendix C Social foundations, institutional reflexivity, and independent interfaces 25

Appendix D Main-text terms and original proof identifiers 39

Appendix E Canonical payment, open operation, and the complete economic process 40

Appendix F Capability portfolios, actual repair, and continuing responsibility 76

Appendix G Fixed formal proofs and source index 104

1 Objects and use of the supplement

This supplement and the main text jointly carry the complete argument of The Canonical Society Theorem. Appendix A develops the actor lineage quotient, the two layers of root-law completeness, the inhabited constitutional point, and recourse from the original Payment after first refusal. Appendix B develops actual responsibility birth, the exit equivalence, exact repair, and same-occurrence regeneration at every depth. Appendix C develops the complete social foundation, clearing and credit, institutional representation, theoretical responsibility, and four independent subsequent interfaces. Appendix D connects main-text terms to the identifiers and type positions in the original proofs.

We retain every operative provenance condition, quantifier, and type transport. A common occurrence is expressed through exact indices; heterogeneous Payments or successors are transported only along supplied equalities. General execution classification and inhabitation of the canonical instance have separate proofs. The assembled theorem in the main text jointly retains these constructions.

Appendix A Actor Generation, Root-Law Completeness, and Recourse after First Refusal

This appendix proves the results on actor generation, the two layers of root-law completeness, and recourse after first refusal used in the main text. Our notation preserves the types of source occurrences, installed programs, and actual successors. When two objects belong to different dependent types, we first establish an equality of indices and then transport along it. Each construction below uses the existing generation rules of the given source.

A.1 Trace seeds, the lineage image, and actors

Let S be the source type and X the state type. For each $s \in S$, let O_s be a rooted occurrence unfolding with trace $\mathrm{tr}(O_s)$. Also fix a seed type Z, a source-local admission predicate $B(s, x, z)$, and a dependent lineage type L_s. First define

$$Z_s = \{z \in Z : \exists x \in \mathrm{tr}(O_s),\ B(s, x, z)\}.$$

An element of Z_s includes evidence of admission; a seed name alone cannot construct it. Fix a lineage map $\ell_s : Z_s \to L_s$, and define actors as its actual image:

$$A_s = \{a \in L_s : \exists z \in Z_s,\ a = \ell_s(z)\}, \qquad \alpha_s(z) = (\ell_s(z), \text{image witness}).$$

Proposition A.1 (Exact generation of actors). For every choice of the inputs above, α_s is surjective, and

$$\alpha_s(z) = \alpha_s(z') \iff \ell_s(z) = \ell_s(z').$$

Every actor has an admitted seed from the same occurrence trace. Put $z \sim_s z'$ if and only if $\ell_s(z) = \ell_s(z')$. Then

$$\bar{\alpha}_s : Z_s/{\sim_s} \longrightarrow A_s, \qquad [z] \longmapsto \alpha_s(z)$$

is a bijection. Injectivity of the seed map α_s itself requires, and is equivalent to, injectivity of ℓ_s.

Proof. Given $a \in A_s$, unpack its image-membership evidence to obtain $z \in Z_s$ and $a = \ell_s(z)$. Equality of subtype elements is determined by equality of their underlying values, so $a = \alpha_s(z)$. This proves surjectivity. The evidence carried by z supplies some $x \in \mathrm{tr}(O_s)$ and $B(s, x, z)$; provenance is therefore part of the generation of the actor, rather than a label appended afterward.

If two actors are equal, projection to L_s gives equality of their lineage values. Conversely, equal lineage values give equal elements of the image subtype. This proves the equivalence. Reflexivity, symmetry, and transitivity of $\sim_s$ follow directly from equality, so $\bar{\alpha}_s$ is well-defined. If $\bar{\alpha}_s([z]) = \bar{\alpha}_s([z'])$, the equivalence gives $z \sim_s z'$, hence equality of the two quotient classes. This proves injectivity. Every actor already has a seed preimage; taking that seed's quotient class proves surjectivity. Finally, if ℓ_s is injective, projecting an actor equality recovers equality of seeds.

If α_s is injective and two lineage values are equal, the equivalence first gives equality of actors, and injectivity then recovers equality of seeds. ■

The result permits actual merging. Take a trace containing both danced(false) and danced(true), and admit the two Boolean seeds by $B(s, x, z) \equiv x = \text{danced}(z)$. The identity lineage generates two separate actors and two distinct quotient classes. The constant lineage $\ell_s(z) = *$ generates the same actor and the same quotient class from those same two distinct seeds. Together, these direct instances show that actor distinctions are determined by the given lineage relation: seed names cannot force separation when the lineage merges them, and arbitrary merging cannot override a lineage that separates them.

A.2 The complete root law of installed executions

Fix a world relation network N, a vocabulary V, a source root R, an installed recognition family ρ, an exact visit v to that root, and a scenario σ. Visible content, output, and claimed effect belong to types D, E, F in independent universes. A complete legislation program P fixes all of the following:

1. A source submission program and the submission g_P that it generates at (R, v, σ).
2. An open responsibility row e_P on the support of the current occurrence, together with its source authority.
3. Inquiry rules, obstruction rules, and an inquiry program I_P.
4. The inquiry face installed at that root, indexed by the actual generated output of I_P.

The execution x_P is a token generated by this complete program, with no further choices. The public compiler consumes only x_P; it takes no additional action, failure, revision kernel, successor, or branch verdict.

There are three disjoint constructors for the use disposition of a submission: presentation, rooted authority, and typed rejection. The compiler first reads this disposition. Presentation and rejection immediately yield their respective results. A rooted disposition proceeds to the predetermined output of I_P. A direct answer in the original language, an answer after obstruction review, and an answer in the old language all count as legitimate presentation. An actual action belongs to the action class; a revision generated by an exact expressive failure belongs to the revision class.

Proposition A.2 (Root law of complete executions). For every root, visit, scenario, three independent types, and complete installed program P as above, its generated execution lies in exactly one of the following four public classes:

Class	Content retained in the same execution
Presentation	A witness of the presentation branch; the fibre of world writes is empty
Rejection	The exact rejection and the disposition equality; the original generated submission itself as the residual bearer object; the root, whole-ledger, and successor factorization of that object; the fibre of world writes is empty
Action	The original rooted authority, source action program, exact source event, generated action, equality identifying it with the installed output, the target's first complete successor, and the actual next

Class	Content retained in the same execution
Revision	The original rooted authority, exact obstruction, review result, exact failure of the old language, revision kernel on the same responsibility row, installed-output equality, first revision write, and successor consumers for the old and new rows

A world write exists if and only if the actual compiled result belongs to action or revision; these two writing classes are disjoint. Every world write has rooted authority. Presentation and rejection cannot write. There is no compiled authority outside these four classes.

Proof. First eliminate the actual submission disposition. The presentation constructor supplies the presentation marker. The rejection constructor supplies the exact rejection and the residual packaging of the original submission; its root–whole-ledger–successor factorization is read directly from that submission. The write-marker types of both constructors are empty.

In the remaining rooted case, eliminate the actual inquiry output. The three answer constructors still have empty write-marker types and all give the presentation class. The action constructor already contains the source-generated action and its output-index equality, so reading the target of that same action gives the first complete successor and the actual next. The revision constructor already contains the obstruction, exact failure, and revision kernel. Reading the kernel's responsibility-row equality, first write, and two successor consumers gives the revision class. No step chooses a new action or failure.

More explicitly, each class marker is a singleton type on the corresponding constructor and an empty type on every other constructor. The world-write marker is a singleton type only on the action and revision constructors. Inspection of the constructors therefore gives the necessary and sufficient condition for writing, disjointness of the two writing leaves, and the absence of writing in the first two classes. The markers establish the classification; the actual data carried by action and revision supply the content of the writes. These are separate parts of the argument.

Finally, check the installation constraint. Every rooted result constructor carries

$$\text{inquiry output of the result} = I_P.\,\text{generate}\,.\,\text{output}\,.$$

Thus, if a candidate result has an output different from the predetermined installed output, eliminating its constructor yields a contradiction between this equality and the assumed inequality. The corresponding result type is empty. This excludes substituting another action or revision after the fact through a public result constructor. ■

The responsibility-birth extension divides the action class further into sequential action and debt admission, both of which retain the action classification above. The additional proofs concerning the new responsibility row, first write, and firstSuccessor appear in Appendix B. The four-class result therefore does not require action to have only one internal constructor.

Corollary A.3 (Conditional revision capability). Let c_P be the actual compiled result above. If its revision branch is inhabited, the same c_P carries the complete package in the table: exact failure, the same responsibility row, first write, and successors for the two rows.

Proof. Apply the classification in Proposition A.2. Presentation or rejection contradicts the inhabited write supplied by the revision branch. Action contradicts the disjointness of action and revision. The remaining case is revision, which carries the complete revision package. ■ The exact quantification is

$$\text{Inhabited}(U(c_P)) \implies \text{Inhabited}(\text{ExactRevision}(c_P)).$$

This establishes that complete revision capability is available whenever the actual compiler selects revision. It does not require every execution to enter revision.

A.3 Raw submissions, the empty local fibre, and the external Payment bridge

The raw-submission layer of the canonical society and its complete installed-execution layer have different inputs. A raw submission g need only have been generated by a submission program; it does not presuppose the existence of a complete local legislation program.

Lemma A.4 (An empty responsibility type excludes local legislation). If the type of open responsibilities on the support of the current source occurrence is empty, then the dependent sum of complete legislation programs at that same occurrence and their generated executions is empty.

Proof. Every complete program must contain e_P, an element of precisely that open responsibility type. Given a candidate program–execution pair, taking its program's e_P contradicts emptiness. The entire fibre of programs at that occurrence is therefore excluded, and so is every local action within it. ■

The current raw-submission root of the canonical M14 construction satisfies this local emptiness condition. It nevertheless has source-generated rooted submissions. For any such g and rooted authority a, the construction

$$B(g, a)$$

first generates the existing transport span T from the original old visit, source index, and boundary. This span comes from the original right replay, and its target is the external Payment occurrence already generated by that replay. The construction takes no target Payment, target legislation, revision kernel, or next as an additional input. It preserves

$$\text{event}(g) = \text{event}(T.\,\text{sourceSubmission}),$$

together with the root–whole-ledger–successor factorization of the original submission. At the target, it reads the Payment root event, responsibility-row authority, execution, proof-carrying legislation, independent consumer, and self-U8 already present in T. The target legislation and the Payment share a responsibility row; the legislation occurrence is that target Payment occurrence; and the runtime consumes the revision kernel already installed there. The failure in the target self-U8 is exactly the original inquiry construction's exact failure δ_9. Arbitrary raw submissions share this source-generated target; a different failure cannot be chosen afterward for each submission.

This source-event equality connects g only to the source end of the transport. The source-submission occurrence and the target Payment occurrence retain their respective types. The formal separation is

$$\mathrm{inl}(\mathrm{event}(g)) \neq \mathrm{inr}(\mathrm{rootOccurrence}(T.\,\mathrm{targetPayment})).$$

It states that these two positions of different types cannot be identified. It cannot be rewritten as equality of the two occurrences, and this separation in a sum type cannot replace the actual same-Payment equalities at the target.

Proposition A.5 (Root law of raw submissions and its inhabited positive instance). For every canonical visit index, scenario, independent types D, E, F, and source-generated raw submission g at that visit, one of the following four types is inhabited: presentation without writing; rejection that retains the exact bearer object and does not write; a local rooted action; or a rooted external whole-ledger–revision bridge. The local-action type is empty; the other three classes are pairwise disjoint. If the raw-submission compiler's actual disposition is rooted authority a, then $B(g, a)$ supplies the complete external positive branch.

Moreover, for every old visit, source index, and visit index, the canonical certified-authority submission g_{act} in the actual scenario satisfies

$$\mathrm{Inhabited}(\mathrm{RootedExternalWholeLedgerU8}(g_{\mathrm{act}})).$$

Proof. Eliminate the three actual dispositions of the raw-submission compiler. Presentation and rejection each retain the original submission's factorization and the local emptiness from Lemma A.4; rejection also retains the residual packaging of the original submission itself. The rooted branch uses $B(g, a)$ to generate the external target package and hence belongs to the fourth class. The third class in the four-class statement requires a complete program at the same occurrence, so Lemma A.4 excludes it. The disposition equalities for presentation, rejection, and the rooted branch point to different constructors. Inhabitation of any two classes would equate distinct constructors, which is impossible.

For the positive instance, the canonical certification program in the actual scenario has already generated g_{act} and proved that its actual disposition is rooted. Eliminate that disposition again. Presentation and rejection contradict rootedness. In the rooted case, take the authority and disposition equality and supply them to the same bridge construction. No complete local legislation program has been assumed to exist. ■

The identity of the target consumer requires a separate check. External acceptance evidence gives an equality of the dependent indices of the target legislation. Transport the existing target consumer along this equality to obtain a consumer at the acceptance index. After eliminating the acceptance constructor, this transport is the identity: its whole-ledger write-back, revision consumption, and generated next all agree with the target consumer. The successor is the actual current after one step of the existing runtime. Inhabitation of the raw layer thus retains an actual consumer as well as a branch classification.

A.4 An inhabited canonical point and the total assembly

Fix a foundation horizon H, an old visit o, a source index i, a boundary b, a health depth h, a requested fuel f, and an observer fuel q. Let S_H be the canonical society construction. Its complete foundation is supplied by the institutional foundation construction in the main text. Here we consume the original recurrence registered by that society, without starting another social root.

The registration supplies an initial cycle c_0 and its cofinal readout. The physical cycle recurrence is

$$c_{n+1} = \mathrm{nextCycle}(\mathrm{step}(c_0, n)).$$

At each depth, the joint continuation pulse reads from this same step the self-U8, root-law transition, and actual birth compatible with the next life index. For the request f, q, the cofinal construction directly takes

$$m = f + 1.$$

Hence $f < m$. The pulse's finite root-law consumer, actual path separation, distinct authority kernels with the same normal form, next Payment, and responsibility retention together supply the material for this point. Platform admission connects to this same point through the original right replay. The source-generated constitutional point then installs, at the same (c_0, m), duties, the distinction between exit and capture, residual responsibility, and the exact self-U8. In particular,

$$\mathrm{generatedFinalNextCycle}(c_0, m) = c_{m+1}.$$

Theorem A.6 (Inhabited canonical root-law completion). For every H, o, i, b, h, f, q, the registered recurrence above is inhabited and supplies some m and a single source-generated constitutional point p such that

$$\mathrm{RootLawComplete}_{M_\infty}(p) \ \wedge \ \mathrm{U8Available}_{M_\infty}(p).$$

The first conjunct includes all of the following: the concrete rooted external positive instance at that point; the complete classification of every source-generated raw submission at every visit index and scenario, with independent D, E, F; and the execution root law of every complete installed program in that range. The second conjunct retains, separately, the external revision bridge for an actual rooted disposition in the raw layer and the compiler's conditional revision capability in the installed layer.

Proof. Take c_0 from the registration of S_H. The health depth h determines the corresponding instance of the existing cross-compiler recurrence. The cofinal generation above supplies $m = f+1$ and the same constitutional point p. For the material at that point, the first-refusal recourse construction supplies source-owned recourse, an actual ordinary whole-ledger write-back, and an ordinary actual next. The point itself supplies a self-U8 tick and the control excluding closure as a runtime that forbids revision. Proposition A.5 supplies the concrete external positive instance

for g_{act} at the point's life index. These objects form the material for root-law completion at that point.

Now construct the two universal conclusions separately. Given any visit index, scenario, and generated raw submission, apply Proposition A.5. Given any complete installed program, apply Proposition A.2. Both read their respective objects from the recognition root currently fixed; neither depends on whether the selected request happens to exhibit a particular kind of program. Apply the rooted bridge construction in the raw layer and Corollary A.3 in the installed layer to obtain the two conditional revision capabilities. Assemble the concrete positive instance, the two universal classifications, and the two conditional capabilities at this same p. This gives the required conjunction. ■

The assembly proof and the execution classification have different tasks. Proposition A.2 proves that any given complete execution has no fifth class. Theorem A.6 actually generates a substantive canonical instance. Raw submissions, complete local programs, target Payments, and constitutional points retain their respective indices; a generalization cannot erase these distinctions.

A.5 Finite acceptance, first refusal, and recourse from the same source

A.5.1 Finite uniqueness of the physical cycle

First fix an initial cycle c_0. For a fuel k, a finite candidate consists of d_j, $0 \le j \le k$, where d_j belongs to the cycle type at boundary $b + j$. If it satisfies the exact base-point condition and every actual successor edge,

$$d_0 \simeq c_0, \qquad d_{j+1} \simeq \mathrm{generatedNextCycle}(d_j),$$

then $d_j \simeq c_j$ for every $j \le k$. Here $\simeq$ denotes heterogeneous equality between dependent types.

Proof. Induct on j. The zero case is the base-point condition. Suppose $d_j \simeq c_j$. Their current indices are the same, so this can first be converted to ordinary equality in that type. Apply congruence for generatedNextCycle, then compose with the candidate's successor edge and the defining equation for c_{j+1}. This gives $d_{j+1} \simeq c_{j+1}$. ■

Consequently, two independent finite candidates agree at every common depth. Executing a prefix and a suffix separately also gives the same physical cycle. Inducting again on the suffix length, the zero case is immediate; the successor case applies congruence to the packaged generated-successor function. Thus

$$c_{r+s} \simeq \mathrm{cycleAt}(c_r, s).$$

Transporting the legislation span, acceptance, and consumer along this packaged equality gives a heterogeneous equality for each of the three. Their erased actual next values satisfy an ordinary equality. This ensures that the refusal argument below preserves the very same finite path that has already been executed.

A.5.2 The exact acceptance condition and first refusal

Let Q_j denote the source Payment proposal recognized by physical node c_j. A finite external program supplies only

$$\pi : \{0, \dots, k\} \to \text{AnySourceBackedPaymentOccurrence};$$

it contains no acceptance verdict, effect, target successor, or root authority. Define the acceptance fibre $\mathcal{A}_j(\pi)$ as the source acceptance of $\pi(j)$ by that node's original legislation span. The acceptance construction gives the exact condition

$$\mathcal{A}_j(\pi) \neq \emptyset \quad \Longleftrightarrow \quad \pi(j) = Q_j. \tag{A.1}$$

The index of the original proof-carrying legislation being accepted is fixed, so the heterogeneous legislation equality in the general acceptance condition specializes here to reflexivity.

Lemma A.7 (Finite acceptance dichotomy). Every π is either accepted at all nodes or has a unique first-refusal node r, satisfying

$$\mathcal{A}_r(\pi) = \emptyset, \qquad \forall j < r,\ \mathcal{A}_j(\pi) \neq \emptyset.$$

The two cases are disjoint. If the first refusal lies within a prefix, restricting to that prefix or extending it preserves the same first-refusal node.

Proof. Consider the finite set $D = \{j \leq k : \pi(j) \neq Q_j\}$. If D is empty, (A.1) gives acceptance everywhere. Otherwise, take its least element r. Since $r \in D$, (A.1) implies that $\mathcal{A}_r(\pi)$ is empty. If some $j < r$ were not accepted, then $j \in D$, contrary to minimality. If two first-refusal nodes satisfied $r < r'$, the acceptance required before r' would contradict refusal at r. The reverse ordering gives the same contradiction, proving uniqueness. Acceptance everywhere directly contradicts emptiness at a first refusal. Restriction to a prefix preserves all entries $j < r$ and the entry at r; extension also leaves those entries unchanged. Uniqueness therefore identifies the first refusal as the same node. ■

Lemma A.8 (Refusal preserves the original actual legislation). At a first refusal r, all source proposals and physical effects at $j < r$ remain unchanged. The fibre of acceptance consumers for the external proposal is empty, but the canonical proposal at c_r remains accepted, its actual legislation still produces effects, and it continues to the literal next physical span.

Proof. Earlier acceptance gives $\pi(j) = Q_j$ by (A.1), and earlier effects are read from the accepted legislation of the same physical cycle. The external acceptance-consumer type is the dependent sum

$$\sum_{a \in \mathcal{A}_r(\pi)} \text{LegislationConsumer}(L_r).$$

Its first projection would lie in an empty type, so this sum is empty as well. On the other hand, Q_r itself is accepted by the source-generated acceptance constructor. The independent consumer obtained along its legislation-index equality is the original target consumer. It retains the

whole-ledger write-back, exact revision consumption, and runtime actual next. By the recurrence definition, the physical successor span is exactly nextSpan(step(c_0, r)); its new acceptance and effects are generated by the same recurrence. Rejection of the external proposal changes neither c_r nor its generation rules. ■

An instance requiring no additional mismatch hypothesis submits Q_1 at node zero. Injectivity of adjacent source Payments gives $Q_0 \neq Q_1$: if the proposals were equal, injectivity of the target Payment with respect to the boundary would yield $b = b + 1$, a contradiction. Thus the program with $\pi(0) = Q_1$ is first refused at zero, while the original legislation at node zero continues.

A.5.3 How the bearer at the refusal node continues

Given a first-refusal node r, let R_r be the existing right replay at the same physical boundary $b+r$. It contains the actual service authority, exit material, and Payment occurrence. Define the repair depth by

$$d_r = \mathrm{depth}(R_r.\,\mathrm{actualServiceAuthority}).$$

Invoke the original ordinary-exit repair construction at depth d_r to obtain $\mathcal{R}_{d_r}$. The first-refusal proof is not used to manufacture a new event: it identifies the node under discussion, while R_r already supplies the repair's source event.

Lemma A.9 (Ordinary recourse on the same Payment). $\mathcal{R}_{d_r}$ and the actual exit in R_r use the same Payment root. The repair targets the original installed operational standing and retains its responsibility consumer, ordinary whole-ledger write-back, selected row commuting with the world, and actual successor. Capacity after repair is one, the next visit is $d_r + 1$, and the old law surface is preserved.

Proof. The actual exit material gives

$$R_r.\,\mathrm{actualExit}\,.\,\mathrm{paymentOccurrence} = \mathrm{periodicSourceBackedPayment}(d_r).$$

Project both sides to their root events and take the symmetric direction. This gives the heterogeneous equality between the ordinary repair root and the Payment root of the original replay. The ordinary repair reads the UBI limitation preinstalled at that depth. Its typed repair index is the prescribed repair for the exact source cause within that limitation. The bounded institutional reform being applied reads the original operational authority, so its target responsibility entry is, by definition, the original operational standing.

The ordinary repair's selected row is the limitation's original temporal selected row; whole-ledger commutation and root factorization come from the same bounded reform. Its generated successor satisfies

$$\mathrm{next}(\mathcal{R}_{d_r}) = \mathrm{rootGeneratedNext}(v_{d_r}), \qquad \mathrm{lawSurface}(\mathrm{next}(\mathcal{R}_{d_r})) = \mathrm{lawSurface}(R).$$

The next visit is v_{d_r+1} by definition, and the next installed program equals the target program of the current disposition. The exact repair maps the actual capture cause to removeCapture;

substitution into that cause's outcome function gives capacity one. The next actual service geometry from the same source also gives one. The responsibility consumer, whole-ledger write-back, and all the equalities above are read from $\mathcal{R}_{d_r}$, so the repair witness remains the same throughout these conclusions. ■

Lemma A.10 (The two successors remain distinct). The successor of this ordinary recourse and the successor of the original canonical revision legislation at node r are unequal in the common packaging of heterogeneous authoritative currents.

Proof. Ordinary recourse is packaged as the generated current on the original M9 network, whose vocabulary-current type is Unit. The acceptance consumer's successor equality rewrites the other side as the revision current of the existing post-submission runtime, whose current type is $\mathbb{N}$. If the common packages were equal, congruence for the natural-number cardinality of the current type would give $\mathrm{Nat.card}(\mathsf{Unit}) = \mathrm{Nat.card}(\mathbb{N})$. The former is one; the latter is zero under this cardinality convention for infinite types. This is a contradiction. ■ The argument separates these two typed authoritative successors; it does not use the different support-type separation proved in the theory-adoption section.

Theorem A.11 (Source-owned recourse after first refusal). For every initial cycle, every finite external proposal program, and every first-refusal node of that program, the refusal's acceptance fibre and dependent consumer fibre remain empty. At the same node, the bearer has ordinary repair on the original Payment, the preserved original standing, an actual responsibility consumer, a whole-ledger write-back, and an actionable successor. The original canonical legislation continues to generate its revision successor, and the two successors remain distinct. For every finite fuel, the instance above that submits an adjacent Payment at the wrong node supplies an actual first refusal and complete recourse.

Proof. Assemble the refusal and continued original legislation of Lemma A.8, the ordinary repair on the same Payment of Lemma A.9, and the successor separation of Lemma A.10, all at the same initial cycle and node. For the existence instance, take the program that submits the adjacent Payment at the wrong node and apply Lemmas A.7–A.10. The constitutional point consumes this complete source-generated recourse; it does not reclassify the refusal as an action. ■

A.6 Three rules for type transport

Three kinds of equality occur in the preceding proofs, each with its own role.

First, equality in the actor image subtype compares only lineage values; its existence evidence creates no additional identity. Second, finite cycles, Payment roots, and dependent consumers use transport along equalities of indices. Heterogeneous equality can be converted to ordinary equality only after the indices agree. Third, the transport span between a source raw submission and a target Payment is retained. Equality of source events and the same-Payment facts at the target have their own equations; they cannot be composed into arbitrary identity across occurrences.

Joint assembly requires each projection to be read from the same generated object. The preinstalled-output equality of a complete execution, uniqueness edge by edge for finite candidates, the dependent-index equality of an acceptance consumer, and the repair's Payment-root equality

each secure this shared origin. None of these proofs uses proof irrelevance for propositions to replace identity of world events.

Appendix B Actual responsibility birth, exit, and same-occurrence regeneration

This appendix gives the complete arguments for actual responsibility birth, the necessary and sufficient condition for exit, and regeneration at every depth. The depth d in the notation belongs to $\mathbb{N}$. The finite foundation horizon h indexes only the canonical society's foundation certificate; it does not restrict the depths below to $d \leq h$. A payment's identity includes its temporal visit and actual event. Values of local payment receipts may recur, but this does not identify the payment occurrences.

B.1 Why responsibility birth requires an actual admission event

Let N be the old responsibility network, U its old support, and $\mathcal{O}_N(U)$ its complete collection of open responsibility rows. Fix a debt-state type S, a budget $b : S \to \mathbb{N}$, and an actual transition family

$$\mathrm{Step}(s,t), \qquad a : \mathrm{Step}(s,t) \Longrightarrow b(t) < b(s).$$

An admission event is (t, a), with the source state s already fixed by the event's type. The target state is not a separately specified prediction.

Responsibilities and claims in the extended network use disjoint sums: the left summands retain the old responsibilities, and the right summands carry the new debt. Support has the form (U, σ), where σ is an optional debt state. Inactive support $(U, \bot)$ has only old open rows. Active support (U, s) has, in addition to every old row, a debt row determined by s, denoted δ_s. Its claim holds natively on active support.

Lemma B.1 (An actual new row and a strict first payment). For every old support U and actual event $a : \mathrm{Step}(s,t)$:

1. δ_s is fresh relative to the complete inactive ledger.
2. The first complete payment on active ledgers carries δ_s to δ_t, supplying a transition receipt for every inherited row at the same time.
3. $b(s) > 0$ and $b(t) < b(s)$.
4. Ordinary ledger evolution requiring an old origin for every target row cannot generate this active ledger from the inactive ledger.

Proof. Suppose an inactive open row had the same debt claim as δ_s. Split on its responsibility tag. If the row lies in the old-responsibility left summand, its claim cannot equal the new debt claim in the right summand. If it lies in the right summand, inactive support supplies no witness that opens this responsibility. Both cases are contradictory, so δ_s has no old origin.

The actual event a determines the complete transition between the active ledgers at s and t. For the new debt row, the target definition directly supplies δ_t. For each old row, a transition receipt from the same a preserves its responsibility. The strict budget inequality follows from the

actual transition law. Since $b(t) \in \mathbb{N}$, we have $0 \leq b(t) < b(s)$, hence $b(s) > 0$, without a separate positive-budget assumption.

Finally, if ordinary ledger evolution could reach the active ledger from the inactive ledger, its target-origin condition would supply an inactive old row for δ_s, contradicting the first step. Preservation of old responsibilities and admission of new responsibilities thus have distinct, explicit roles in the types. □

An admission fragment is not yet an installed social action. It must be joined to an actual root event, the initial ledger rows, and the successor generated by the root. The following construction performs that joining.

B.2 How two actions share the complete root law

Fix an old root R, a temporal visit v, its exact event e, an old open row o, and its root authority. A complete action program fixes a function

$$e \longmapsto T(e)$$

before the event is emitted. The execution credential generated by the run contains no target root, ledger, or successor that a caller may replace; all these objects are read from the same $T(e)$.

Sequential action

The target of a sequential action has a conservative semantic transport and an initial open-row map

$$i : \mathcal{O}_N(U) \to \mathcal{O}_{N'}(U'), \qquad r \circ i = \mathrm{id}.$$

Every initial target row is the image of an old row, responsibility and claim transport commute, and the budget does not increase. Thus i is both injective and surjective: applying r to $i(o_1) = i(o_2)$ proves injectivity, while the old-origin field of every initial target row proves surjectivity. The initial ledger can neither merge distinct old rows nor conceal additional new ones.

The target program also supplies a row witness for $i(o)$ in the target's initial actual patch, the target's first complete successor F, and

$$\mathrm{AnswerNext}_{T(e)}(i(o)).\,\mathrm{next} = \big(T(e).\,\mathrm{root},\ v_0'.\,\mathrm{next}(F)\big). \tag{B.1}$$

The execution's successor is therefore the visit actually generated by this complete write, rather than a separately selected target state.

Responsibility-admission action

A responsibility-admission action uses the extended network of B.1. Its target program retains all of the following data:

- The old root event e's own complete successor E.
- A row witness for every old row in the target's initial patch.
- A row witness for the new row δ_s in that same initial patch.

- The target's first complete successor F.
- The target support of the first payment,

$$\operatorname{supp}(F.\,\text{target}) = (\operatorname{supp}(E.\,\text{target}), t).$$

- Heterogeneous equality between F's rowwise destinations and those of the joint evolution consisting of the old ledger evolution E and the actual debt transition a.
- Injective embeddings of all old projections and preservation of their complete outputs.

Heterogeneous equality here handles only the type transport dependent on the support equation. After transport along that equation, the two sides are the same rowwise destination function; no separate ledger is introduced. The initial authorities for the old and new rows are generated from their respective memberships in the initial patch. Eliminating the disjoint sum of extended responsibilities shows that any initial target open row is either an inherited old row or δ_s. This exhaustion takes no additional coverage classifier as input.

The answer successor of the admission action also satisfies (B.1), with F the complete first-payment successor just retained. Birth of a new responsibility therefore comprises the row's freshness, its native validity, its first complete payment, preservation of the entire old ledger, and the actual next visit.

Theorem B.2 (Birth remains within the action class). For every installed complete legislation program and its generated execution, the action class is the sum of two dependent leaves:

$$\text{Action} = \text{Sequential} \sqcup \text{DebtAdmission}\,.$$

Both leaves retain root authority, the event from the same source, the identity of the installed output, the first complete write, and the exact successor. The admission leaf additionally retains the new row and first payment of B.1. They do not add a fifth root authority.

Proof. The public compiler first eliminates the submission disposition generated by that same complete program. Presentation and refusal enter their respective non-writing leaves directly. A rooted disposition leads to elimination of the inquiry output already installed by the program: direct answers, answers after U7, and answers in the old language enter the presentation class; actual sequential action and actual responsibility admission enter the two leaves of the action class; exact failure and its revision core enter the U8 class.

In the admission branch, the compiler retains the equality identifying the admission output with the installed output. The complete root-law consumer then reads the admission target's admission, firstSuccessor, and answer-successor equation directly. The sequential branch similarly reads its own target's firstSuccessor and successor equation. In the remaining branches the action type is empty. Elimination of the compilation result therefore yields

$$\text{Writes} \iff \text{Action} \vee \text{U8}, \qquad \text{Action} \cap \text{U8} = \emptyset.$$

Presentation and refusal do not write, and writing requires root authority. If a result is constructed for another inquiry output, its output-identity equation forces that output to equal the installed output. The result type for a different output is empty. □

The institutional reform/revision readout also comes from the actual runtime. Given a runtime ρ and depth d, the finite prefix is $\rho.\mathrm{run}(d)$, and the current occurrence is $\rho.\mathrm{tickAt}(d)$. The reform readout covers ordinary answers, U7 answers, old-language answers, sequential actions, and responsibility admissions. Only the revised constructor of that actual tick supplies revision. Its answer, independent branch receipt, and heterogeneous actual successor are all read from the same tick. In the revised leaf, the answer successors of the old row and the new obstruction row come from the same revision core and share the successor exactly. The labels themselves provide no constructor for returning an authority object.

B.3 Temporal visits, payments, and source installation

The canonical old root has a native continuing branch. Starting from the initial visit v_0, define recursively

$$v_{d+1} = \mathrm{next}(v_d).$$

The history's causal-depth function satisfies $\mathrm{depth}(v_d) = d$: the initial history has depth zero, and each actual predecessor extension increments depth by one. Induction gives the equation immediately.

Lemma B.3 (Adjacent payments retain distinct identities). The map $d \mapsto v_d$ is injective, as is $d \mapsto p_d$, where

$$p_d = (v_d, \text{the source-backed payment generated by this visit}).$$

Proof. If $v_d = v_k$, taking causal depth on both sides gives $d = k$. If $p_d = p_k$, first project the visit of the payment occurrence and then use the preceding conclusion. In particular, $p_d \neq p_{d+1}$. □

Each v_d reads the same UBI source law installed in advance. This law supplies a source seed, a local responsibility consumer, a disposition, a selected row in the original complete patch, and the complete successor of that same occurrence. These outputs form one dependent package. The root's renewal projection and the local law's projection have heterogeneously equal outputs; the generated ledger, the renewal projection, and the next of the same root answer successor factor together.

Source installation preserves the dependent type and value of the original observation. The extended projection inventory consists of a renewal coordinate plus every inherited coordinate. On inherited coordinates, activation, payload, and output are preserved by definition. The renewal coordinate is generated from the same classification result of the source law, without consulting a table of completed future receipts.

Periodic service also indexes p_d. The two existing maintenance events supply receipts for the two served slots. The actual-service structure transports each slot's ordinal to the same ordinal among actual runtime events, ensuring that each service slot was served by that charged run. Local maintenance shapes may recur, but service authority includes the index of the current payment occurrence. Generating service from d is therefore a different construction from reading a stored success table at an externally supplied integer.

B.4 The complete necessary and sufficient condition for actual exit

Fix one such payment. Let C be the finite set of future codes generated by the source law, r the independent exit relation, and $j \in J = \mathrm{Fin}(|\mathrm{served}|)$ an actual service slot. The original employment relation's future has been revoked while the worker persists; r differs from that employment relation. These are fixed indices of the exit program.

Write $\mathrm{Cont}(r, c)$ for target-side continuation in r, $\mathrm{Idle}(c)$ for continuation without action, and $\mathrm{Cap}(c)$ for the capture classification generated by the source law. Define the surviving code set by

$$C_{\mathrm{live}} = \{c \in C : \mathrm{Cont}(r, c) \text{ has a witness}, \ \neg\,\mathrm{Idle}(c), \ \neg\,\mathrm{Cap}(c)\}.$$

For $c \in C_{\mathrm{live}}$ and $j \in J$, a feasible exit edge must additionally contain a receipt that this purchasing-power standing pays for c, and a route receipt from c to the j-th served crossing. Its complete structure retains the actual event of the same ordinal and its service receipt.

Theorem B.4 (The necessary and sufficient condition for exit). At every canonical foundation horizon h and every $d \in \mathbb{N}$, an actual exit renewal exists if and only if

$$\begin{aligned} \exists c \in C, \ \exists j \in J : \ & \mathrm{Cont}(r, c) \text{ has a witness } \wedge \neg\,\mathrm{Idle}(c) \ \wedge \neg\,\mathrm{Cap}(c) \\ & \wedge\, \mathrm{Pay}(p_d, c) \text{ has a receipt} \\ & \wedge\, \mathrm{Route}(c, \mathrm{crossing}(j)) \text{ has a receipt.} \end{aligned} \tag{B.2}$$

Here J consists of the actually served slots of this occurrence. Payment, route, and service are joined through the same feasible edge.

Proof. From an actual renewal, take its code c, slot j, and feasible edge. Unfolding membership in the surviving-code set gives membership in the source set, target continuation, non-idleness, and non-capture. Reading the payment and route receipts from the edge gives the right-hand side.

Conversely, the first four conditions on the right-hand side give $c \in C_{\mathrm{live}}$. Supply the payment and route receipts to the feasible-edge construction. The indexed actual-service settlement gives a transport

$$\mathrm{Fin}(|\mathrm{served}|) \longrightarrow \mathrm{Fin}(|\mathrm{run.events}|),$$

preserving the ordinal and supplying the service receipt for the corresponding actual event. This constructs a complete feasible edge; (c, j, edge) then constructs an actual renewal. Both directions retain the same code, slot, and occurrence index. □

Actionable exit capacity K_d is the maximum matching size of this finite bipartite graph. It is computed from feasible edges; it is neither a nominal payment amount nor an arbitrary capability score specified in advance. For finite matchings, positive capacity is equivalent to an existing edge: a nonempty matching contains an edge, while a single edge forms a singleton matching. Consequently,

$$K_d > 0 \iff \mathrm{ExitRenewal}(p_d) \neq \emptyset. \tag{B.3}$$

Neither endpoint can be reused in a matching, so

$$K_d \leq |C_{\mathrm{live}}|, \qquad K_d \leq |\mathrm{served}| \leq \mathrm{capacity}(p_d). \tag{B.4}$$

In the canonical instance, the source queue has three arrivals, root capacity is two, and two slots are actually served. The sole surviving non-idle exit code is leave. Existing payment and route receipts give a feasible edge between leave and the first slot, so $K_d \geq 1$. The uniqueness of the surviving code gives $K_d \leq 1$. Thus $K_d = 1$ at every depth. The unserved queue remains nonempty, showing that service congestion and actual exit can coexist. Full capture removes every surviving exit code, so the captured geometry has zero capacity and an empty exit type.

B.5 Three origins of the same zero reading

Fix d and construct the following three source geometries. They use the same actual service run and retain common program coordinates for payment, purchasing power, original employment loss, and the exit interest.

Origin	Capture	Exit payment incidence	Route incidence to actual service	Actual service
z_c: full price capture	Holds	Present	Present	Present
z_p: missing payment incidence	Does not hold	Empty type	Present	Present
z_r: missing route incidence	Does not hold	Present	Empty type	Present

Missing payment incidence does not mean that the underlying periodic payment event is absent. It means that this payment cannot supply the required incidence receipt for the exit code under consideration. Missing route incidence likewise retains actual service and payment; what is absent is the incidence connecting the exit code to that served crossing.

The three origins' criteria are generated from source fields; capacity is not an input. The capture origin carries capture settlement and the exact temporal payment, while the other two use an empty incidence family and retain the remaining coordinates. Place these origins, each with its source signature, in a disjoint-sum type Z_d, and write $q_d : Z_d \to \mathbb{N}$ for the capacity readout.

Theorem B.5 (The zero-capacity fibre and failure of reconstruction). The three origins are pairwise distinct, and

$$q_d(z_c) = q_d(z_p) = q_d(z_r) = 0.$$

Thus q_d is not injective, and there is no $g : \mathbb{N} \to Z_d$ with $g \circ q_d = \mathrm{id}_{Z_d}$.

Proof. Under full capture no code survives, so (B.4) gives zero capacity. When payment incidence is missing, any feasible edge would supply a payment receipt in an empty type, a contradiction. The same argument applies to missing route incidence. By (B.3), these latter two geometries also have zero capacity. The three origins lie in disjoint-sum branches with different source signatures, so they are pairwise distinct.

If g were a left inverse, then

$$z_c = g(q_d(z_c)) = g(0) = g(q_d(z_p)) = z_p,$$

contradicting the distinction between the branches. □

Capacity still answers exactly how many exits can be realized simultaneously. What it cannot reconstruct on its own is the full source cause of that capacity. The next result shows that discarding the cause also discards the exact repair.

B.6 Exact repair and the actual ordinary successor

Let the actual failure origins f_c and f_r denote tenure capture and an uncaptured route bottleneck, respectively. The former reads capture, the tenure interest, relation phase, bearer, and role from the UBI disposition installed at that depth. The latter reads the exact route interruption, payment, service, and same root event from an independently installed route-restoration coordinate. Their repair language has only

$$a_c = \text{remove capture}, \qquad a_r = \text{restore route}.$$

Repair effects change only their target coordinate: removing capture changes neither payment nor the route family; restoring the route does not change payment. Applying the opposite operation retains the coordinate responsible for the original failure.

Proposition B.6 (The unique exact repair). At every depth, capacity after repair is

	a_c	a_r
f_c	1	0
f_r	0	1

. (B.5)

If effective means that capacity after repair is positive, then a_c is the unique effective repair for f_c, and a_r the unique effective repair for f_r. No repair selector reading only the original capacity is effective for both.

Proof. Applying a_c to f_c gives the uncaptured geometry of B.4, with capacity one. Applying a_r retains full capture, with capacity zero. Applying a_c to f_r leaves the empty route family unrestored, so capacity remains zero. Applying a_r yields the complete uncaptured geometry, with capacity one. Exhausting the two repairs proves uniqueness.

Both original failures have zero capacity. A selector reading only capacity must return the same $a(0)$ on both origins. Uniqueness then forces $a(0) = a_c$ and $a(0) = a_r$, a contradiction. □

Retaining the exact cause permits a positive construction: the installed repair compiler reads the cause's source-generated kind and returns its corresponding canonical repair $\text{repair}(f)$. By (B.5), every cause receives outcome capacity one, and

$$\text{Effective}(f, a) \iff a = \text{repair}(f). \tag{B.6}$$

An actual ordinary renewal then consumes this geometric conclusion. For tenure capture, the canonical repair is precisely removeCapture in the occurrence's installed disposition. The selected

row of that source payload generates a bounded ordinary reform satisfying

$$\begin{aligned} \mathrm{row}(\mathrm{reform}_d) &= \mathrm{selectedRow}(p_d), \\ \mathrm{ledger}(\mathrm{reform}_d) &= \mathrm{wholeLedger}(p_d), \\ \mathrm{next}(\mathrm{reform}_d) &= \mathrm{next}_R(v_d), \\ \mathrm{law}(\mathrm{next}(\mathrm{reform}_d)) &= \mathrm{law}(R). \end{aligned} \tag{B.7}$$

These equations follow from the same source installation: the reform's selected row is the actual payload row, its row evolution is the restriction of the complete ledger to that row, and its successor reads the same answer successor. Responsibility lineage and claim are preserved along the row evolution, and the progress budget does not increase. This construction does not borrow the first write of the revised language.

Unfolding the original root's recursive visit and target disposition further gives

$$\begin{aligned} \mathrm{next}(\mathrm{reform}_d).\,\mathrm{visit} &= v_{d+1}, \\ \mathrm{program}(p_{d+1}) &= \mathrm{targetProgram}(\mathrm{disposition}(p_d)), \\ K_{d+1} &= 1. \end{aligned} \tag{B.8}$$

The first two equations follow from the original visit recursion and the definition of the installed target program. The third uses the next occurrence's own actual-service geometry. Restored capability has therefore entered the actual successor, rather than remaining only in a counterfactual table of effects.

B.7 The first policy rupture and two distinct successors

The actual payment occurrence at depth zero carries all of the following: original uncaptured capacity one, fully captured capacity zero, the same purchasing-power standing on both sides, and two still actually served slots. The source law's mother-law classifier generates a policy-rupture object from this actual event. Its type does not yet contain a revision target or first write.

Joining this rupture object with the failure face installed at the same old-root visit gives a mother-law audit. The audit's scoped entry is exactly the root responsibility row of that failure face. The causal authority and existing successor of the same row then generate a U8 gate. The root failure and minimal coface it produces are precisely those of the already constructed self-revision core. Its unique faithful factor map enters that specific revised root, carrying the exact first write. The actual inquiry runtime consumes this core at depth zero.

Proposition B.7 (Actual revision at the rupture from the same source). The original policy rupture, mother-law audit, self-U8, first revised write, and claim migration share the source identity just described. Both the ordinary removeCapture successor and the self-U8 successor exist, and they are unequal in the heterogeneous root-successor type.

Proof. Expanding the policy classifier on the sole inhabited old event gives the same periodic payment and the capacity conclusions for both sides. The audit is constructed from the failure face installed at that occurrence, so its root responsibility row is the failure row. The U8 gate's root failure, minimal coface, and attached first write agree by definition with the existing self-U8. The

actual runtime's branch receipt proves that it consumes this very core. The first write migrating the old claim is consequently the same as the first revised write here; a separate future settlement cannot stand in for the present revision.

B.6 generates the ordinary branch, which continues in the original root network; the actual core generates the revised branch. If the heterogeneous next states were equal, projecting first the network and then its support type would identify the old and revised supports. In this construction, the old support is Bool, whereas the revised support is

$$(\mathsf{Bool} \sqcup \mathsf{GeneratedDirection}) \times \mathsf{Bool}.$$

The new-direction type contains the existing completion, so the revised support contains three points,

$$(\mathrm{inl}(\mathsf{false}), \mathsf{false}), \quad (\mathrm{inl}(\mathsf{true}), \mathsf{false}), \quad (\mathrm{inr}(\mathsf{completion}), \mathsf{false}),$$

that are pairwise distinct: the Boolean values distinguish the first two, and the disjoint-sum branch distinguishes the third. The old support has only two points. The assumed type equality would transport these three points injectively into Bool, which is impossible. Hence the two next states are unequal. □

They share an occurrence and responsibility row while using two existing, distinct action coordinates. They do not assign two values to one deterministic runtime output: the ordinary repair coordinate generates the old-law successor, and the actual inquiry's revision coordinate generates the new-law successor. Branch and coordinate identities in the types determine which successor each must use.

B.8 The joint construction of same-occurrence regeneration at every depth

Let R_d be the actual ordinary repair renewal of B.6, D_d the complete capture-responsibility object on the same payment, and V_d the repaired successor object with actual continuation. D_d retains the two actual services, capture settlement, residual, beneficiary counterparty, and ordinary repair. V_d retains the next depth's actual exit, root responsibility consumer, reauthorization, residual consumer, observation correction, and existing U8 route.

Define the joint regeneration object at depth d by

$$S_d = (R_d, D_d, V_d;\ D_d.\,\mathrm{repair} = R_d,\ V_d.\,\mathrm{repair} = R_d). \tag{B.9}$$

The two equations are heterogeneous equalities in the original dependent types; they express that the three components retain the same renewal. The sole generating function takes only d and invokes the already constructed R_d, D_d, V_d. Both renewal equations hold by definition.

Theorem B.8 (Six joint regeneration conclusions at every depth). For every $d \in \mathbb{N}$, S_d exists. For any joint regeneration object S of the same type, the following six groups of conclusions hold together at its own single payment occurrence:

1. **Continuation of the original law and lineage.** The selected row of $S.R$ evolves within the original complete ledger, preserving responsibility lineage and claim. Its generated successor is $\mathrm{next}_R(v_d)$, its law surface is unchanged, and its successor visit is v_{d+1}.
2. **No contraction of ordinary exit.** Both the current and next occurrences have actual exit, and $K_d \le K_{d+1}$.
3. **Repair of the captured fibre.** The exit type of the current fully captured geometry is empty. The next occurrence actually installs the target program carried by the current repair disposition and has restored actual exit.
4. **Retention of the residual bearer and consumer.** Capture returns the exact original open settlement, and the residual bearer is unchanged. The activated residual consumer exists, and its next equals the original Payment's next. The repaired successor has its own root responsibility consumer.
5. **Revision capability at the successor.** The repaired successor retains a faithful factor map from the existing policy U8's minimal coface to that same concrete revision core, together with the receipt that the original actual runtime has consumed the core. This economic occurrence has no rooted terminal landing.
6. **Responsibility across actors.** The protected interest of the capture settlement belongs to another counterparty generated from the source, distinct from the worker. Settlement identity and responsibility incidence owned by the source are retained.

Proof. Fix any S. Its renewal, capture-death object, and repaired successor carry the conclusions of their original constructions. The two dependent equalities in (B.9) connect them to the same renewal. The proof does not select another future, repair, or successor.

The first group follows from the selected row and complete-ledger factorization from the same source in B.6. It gives the exact root successor, preservation of the law surface, and (B.8), while the row evolution preserves responsibility lineage.

For the second group, $S.D$ gives current uncaptured capacity $K_d = 1$; the repair-successor conclusion of $S.R$ gives $K_{d+1} = 1$, hence $K_d \le K_{d+1}$. The two directions of (B.3) provide actual exit instances at the current occurrence and successor.

For the third group, $S.D$ gives captured capacity zero. If the captured exit type were inhabited, (B.3) would make its capacity positive, contradicting zero. The successor's program identity comes directly from the repaired-face conclusion of $S.R$; positive capacity and actual exit were proved in the second group.

The fourth group uses the capture disposition's exact settlement equation,

$$\mathrm{captureSettlement}(c) = \mathrm{exactOpenSettlement}(p). \tag{B.10}$$

Applying the residual-bearer and residual projections to (B.10) proves their preservation. The original purchasing-power standing already includes the activated consumer and source

responsibility incidence. Its successor commutation is

$$\mathrm{next}(\mathrm{activatedConsumer}) = \mathrm{next}(\mathrm{purchasingPowerStanding}) = \mathrm{next}(\mathrm{Payment}). \tag{B.11}$$

The first equation comes from the same standing's exact readout of its activated consumer; the second comes from the original Payment's successor identity. The next-depth payment installation of $S.V$ supplies its own root responsibility consumer. No residual is transferred to an invisible bearer in this argument.

The fifth group reads the existing faithful coface map and the original runtime's consumption receipt from $S.V$. What is retained here is the constructed U8 route; no additional revision execution is generated for each d. The original root's terminal classifier proves the absence of a terminal landing: every actual evolution of this root generates a continuing branch, and the terminal-event type is empty. Any economic disposition claiming a rooted terminal landing would supply an event of that empty type.

The sixth group uses (B.10) once more. The counterparty-acceptance receipt in the original purchasing-power standing identifies the protected interest as the interest of that source-generated counterparty. The same acceptance event supplies the counterparty's reciprocal obligation. The counterparty actor and the worker have different underlying source identities, so they are unequal. Residual responsibility incidence comes from the original activated consumer's boundary source; another actor's name cannot replace it. All six groups therefore hold together. □

The jointness in this theorem has substantive content: capture death, ordinary repair, and the repaired successor must all refer to the same R_d. The root occurrences of capture and repair must each equal the exact event of the visit to which p_d belongs. Six conclusions located at different events do not satisfy (B.9).

Theorem B.9 (The actual next continues to the next occurrence). For every d, every joint regeneration object S at depth d, and every joint regeneration object T at depth $d+1$,

$$\begin{aligned} S.R.\,\mathrm{next}\,.\,\mathrm{visit} &= v_{d+1}, \\ T.R.\,\mathrm{rootOccurrence} &= \mathrm{event}_R(v_{d+1}), \\ p_d &\neq p_{d+1}. \end{aligned} \tag{B.12}$$

Proof. The first equation is the repair-successor identity of $S.R$, the second the root-occurrence identity of $T.R$, and the third Lemma B.3. What is joined here is the preceding actual next and the visit that generates the following occurrence; the two occurrences are not identified. □

At depth zero, further assemble the rupture object and self-U8 of B.7 with S_0. The ordinary repair retained by this policy revision is exactly the repair in $S_0.D$, and the rupture occurrence is exactly the occurrence of $S_0.R$. We therefore have together the certificate of six periodic regeneration conclusions, self-U8 from the same source, actual runtime consumption, and two distinct successors.

Finally, the canonical society construction first retains the complete M_∞ foundation, then retains, field by field, the consumers for exit, capture responsibility, ordinary repair, policy revision, and joint regeneration described above. Its joint-regeneration field actually takes the same

certificate of Theorems B.8 and B.9. For every foundation horizon h and every depth d, the field's generation at each depth therefore supplies S, and its conclusion for every joint object supplies the six conclusions for that S. This result neither depends on an accidental nonzero capacity at some finite depth nor stores every future occurrence in advance in the source program.

Appendix C Social foundations, institutional reflexivity, and independent interfaces

This appendix provides written proofs for the results on social foundations, clearing, cultural positions, and theory adoption. The constructions retain their original dependent types. Parallel components of the foundation certificate do not become one occurrence merely because one total object contains them. Identifications require the corresponding installation from the same source, explicit transport, or equality of root writebacks. The independent interfaces at the end use their own source laws and actual histories.

C.1 How the complete foundation enters the total construction

Fix a foundation horizon h. Let $\mathcal{F}_h$ be the dependent product consisting of the actor-lineage quotient, four inverse-seed provenance constructions, universal lifecycles, relations and roles, native settlement, the responsibility chain from finite representation to installed institutional updates, source-generated space, source-generated probability, and actual theory reflexivity. Each component carries its own types, seeds, events, and direct consumers. The dependent product retains these constructed objects together with their proofs.

Proposition C1 (Inhabited assembly of the foundation). For every $h \in \mathbb{N}$, a complete $\mathcal{F}_h$ exists. The foundation field of the canonical society S_h is the output of this construction, rather than an object containing labels alone.

Proof. The actor component is the lineage image of source-trace seeds together with the quotient by their equality kernel. The provenance component retains the four existing inverse-seed constructions for economic, biological, multi-agent, and Go sources. The lifecycle component is the complete compilation of one actual lifecycle: target t_5 is born; s_0 survives as t_0; s_1 splits; t_3 is a merge; s_4 exits; and t_4 succeeds s_5. Classification concerns sources, targets, and their occurrence incidences. Split and merge can both hold in a many-to-many relation, so a relation cannot be forced into a single exclusive label. The complete compilation retains original responsibilities; an empty fragment with no method cannot supply a missing source seed.

The relational component uses the actual three-actor relation and its role transport. Let the local phases of the three directed edges be $1, 1, 0$, with role transport

$$\rho_{\mathrm{t}(e)} = \rho_{\mathrm{s}(e)} + \delta(e).$$

If these local phases arose from a global potential u, summation around the cycle would give

$$\delta(e_{01}) + \delta(e_{12}) + \delta(e_{20}) = (u_1 - u_0) + (u_2 - u_1) + (u_0 - u_2) = 0.$$

The actual sum is 2, a contradiction. Hence no global potential exists. Replacing δ by $\delta + \mathrm{d}u$ leaves the cycle sum unchanged, so changing the potential does not remove the obstruction. This is the decisive argument for the actual H^1 obstruction. The two-actor linear control has no such cycle.

The settlement component uses the four-leg exchange, its original responsibility slots, and its complete fulfillment run. Each obligation's bearer comes from its leg's actor, and its beneficiary from the original counterparty leg; both are supplied by the same exchange seed. Live and terminal are distinct constructors of responsibility slots. A change of presentation or an observation without service does not change the slot's state. The price readout is defined on open settlements, giving

$$\mathrm{PriceObs}(e) \simeq \mathrm{OpenSettlement}(e).$$

Thus an open responsibility can have a zero quote, while a completely fulfilled settlement can have an empty price domain. These are different situations.

The subsequent responsibility chain adds native source-code recovery, completeness of finite strategic representation, control examples with the same classical image but different source occurrences, the factorization obstruction to reconstructing source authority from observation, and the actual joint installation of election, purchasing-power, relationship, and AI faces. Completeness of finite representation asserts that every target finite game has a valid source code representing it. Source recovery asserts that every valid source code recovers its own material. Both quantifiers enter the construction. The existing Arrow, Gibbard–Satterthwaite, and Sen-type readouts retain their explicitly stated axiom bundles and applicability conditions.

The capacity-phase component uses capacity $C = 2$ and three exact prefixes of the original queue. Lengths $1, 2, 3$ generate the stable, critical, and overloaded phases, with spare capacities $1, 0, 0$; the third item remains unprocessed. The first position and the recurrent position remain distinct occurrence positions even when they display the same state. A reordered list is not an exact prefix of the original queue. The spatial component retains actual crossings and maintenance responsibilities. Its two settled items and three maintenance debts are counted by their respective original indices and cannot cancel each other. The probability component uses the actual finite measure on a source fibre. For event E and conditioning fibre F, define

$$\Pr(E \mid F) = \begin{cases} \text{none}, & \mu(F) = 0, \\ \text{some}(\mu(E \cap F)/\mu(F)), & \mu(F) > 0. \end{cases}$$

A nonempty fibre of zero mass still returns none; an empty event under a positive-mass condition returns some 0. The actual joint example returns some $1/3$, and its probability and boundary faces read the writeback and next of the same installation.

Finally, take the actual theory-reflexivity construction proved in Section C.4. Placing these objects field by field in the dependent product gives $\mathcal{F}_h$, which is then assigned to the canonical society's foundation field. Parallel component proofs have not been turned into a fictitious single execution. At joint installations, equal writebacks and successors continue to be justified by the original installation equalities. ■

C.2 Common clearing and source-generated credit

C.2.1 One complete clearing run and unilateral fulfillment

Let I be the finite set of exchange legs and $\sigma : I \to I$ the original exchange's counterparty involution. For every $i \in I$, the exchange supplies a bearer a_i, a responsibility q_i, and an original ledger slot j_i. The exchange used here satisfies

$$\sigma^2(i) = i, \qquad a_i \neq a_{\sigma(i)}, \qquad j_i \neq j_{\sigma(i)}.$$

Exact fulfillment authority supplies an executable native fulfillment event u_i for each i. Its source responsibility incidence points to q_i, not merely to an external obligation with the same content. The original leg-to-slot map $i \mapsto j_i$ is injective, and the leg list is exhaustive and has no repetitions. The complete run follows this finite enumeration of native events, recording the obligation's fulfilled terminal archive only at its authorized slot.

Proposition C2 (Common clearing). The actual complete run fulfills every exchange leg and leaves no open settlement. The unilateral edge executing only u_i does not fulfill counterparty leg $\sigma(i)$.

Proof. Induct on the length of the run's finite enumeration. The empty prefix supplies no fictitious fulfillment. Adding u_i, its exact source incidence and fulfillment receipt write terminal fulfilled at j_i. Updates at distinct slots preserve previously terminal slots, while untouched slots retain their original live values. Thus all processed legs are fulfilled after every prefix. The enumeration covers I, so every original leg is fulfilled at the end. Any remaining open settlement would arise from an original leg's live slot, but that slot is already terminal, contradicting constructor separation.

For the unilateral edge, $j_i \neq j_{\sigma(i)}$ ensures that updating the list at j_i does not change its value at $j_{\sigma(i)}$. The latter remains the original live $q_{\sigma(i)}$, so it cannot also be terminal fulfilled. The proof uses the distinct actual slot indices; agreement by both parties cannot replace the counterparty leg's own fulfillment. ■

The common-clearing seed of the actual four-leg source is generated from the existing complete run itself. Its constructor takes neither a second run submitted by the caller nor a submitted coverage proof. After this source-generated settlement face is installed in the periodic root, for every depth d the row selected by proof-carrying legislation, legislative mandate, operational authority, root consumer, and whole-ledger writeback share the same visit coordinate, with actual next at $d + 1$. The local settlement occurrence retains its own type. Installation coordinates connect it to the periodic root occurrence without an equality across their ambient types.

Different depths have distinct occurrences. A periodic occurrence is a dependent pair

$$\omega_d = (v_d, e_d).$$

If $\omega_d = \omega_{d'}$, projecting the visit coordinate gives $v_d = v_{d'}$, and visit injectivity gives $d = d'$. In particular, adjacent occurrences differ, and no single fixed occurrence carries every depth. The universally quantified producer constructs the certificate at a given depth; it does not require a stored table of already-completed infinite clearing.

C.2.2 Where credit responsibility is borne

In the actual partially settled state, leg a has executed and leg b remains open. The borrower is $a_b = t_1$, and the creditor is $a_{\sigma(b)} = a_a = t_0$. The original source generates a backstop activation seed for the open leg b. Its tender carries an actual boundary source event, while the counterparty leg's accepted assumption supplies an acceptance receipt. This tender and counterparty acceptance generate purchasing-power standing. The actual next appends fulfillB to the original generated state.

Proposition C3 (Credit responsibility from the same periodic source). For every d, this standing and its backstop generate a credit object. Its downside bearer is exactly the original bearer of q_b, and borrower and creditor are distinct. Its actual next writes b's original slot as fulfilled. The root claim and debt lineage are preserved through the original root update; in this fixed instance, the progress budget is $2 \to 2$.

Proof. The original counterparty relation of a and b supplies distinct actors. The credit relation is generated by b's source seed. If the creditor also had the borrower-interest incidence for this relation, expanding the seed equalities would make the seed's target token both t_1 and t_0, a contradiction. Receipt of payment by the creditor therefore cannot exchange these relational positions. The downside bearer is read directly from the open b obligation and equals the bearer map of b's actor.

The standing's source event occurs in the original event trace, and its ownership points to the same q_b. The generated proof of the native next appends fulfillB to the partially settled state, and the history appends exactly that event. The native fulfilled rule therefore makes b's slot terminal. A live obligation at the same slot would make one list entry both live and terminal, a contradiction. The unilateral control in C2 shows why the earlier fulfillA cannot establish this conclusion.

The periodic credit object depends on the original backstop at the same depth; it introduces no second credit occurrence. On the root side, sourceClaim, futureClaim, and debtLineage are read from the original applied reform. The lineage's claim equality identifies the two claims. This actual row evolves by carried, with both source and target budgets defined to be 2, excluding a strict budget debit. Local obligation content and the living-root claim remain separate typed readouts. Their respective provenance is retained in the certificate; equal underlying types in a small instance do not identify them. ■

The credit construction supplies both source accountability and actual change after payment. Its row update carries responsibility identically. A strict haircut or shared loss requires its own update material and does not follow from the equality $2 \to 2$.

C.3 Exact extensions of institutional, class-price, and spatial representation

C.3.1 Equality kernels of consumers

Let X be a source-generated restriction domain, and let $(c_i : X \to Y_i)_{i \in I}$ be its registered consumers. Define

$$x \sim_C y \quad \Longleftrightarrow \quad \forall i \in I,\ c_i(x) = c_i(y).$$

Kernel exactness for a complete face $F : X \to A$ means

$$F(x) = F(y) \quad \Longleftrightarrow \quad x \sim_C y. \tag{C.1}$$

Given a new independent readout $r : X \to B$, extend the face and consumer family by

$$F^+(x) = (F(x), r(x)), \qquad C^+ = C \cup \{r\}.$$

Lemma C4 (Exact extension of representation). If (C.1) holds, the equality kernel of F^+ is exactly $\sim_{C^+}$, and its image is naturally bijective with the quotient $X/\sim_{C^+}$. Every registered consumer factors uniquely on $\mathrm{im}(F^+)$. If some x, y satisfy $F(x) = F(y)$ but $r(x) \neq r(y)$, then r cannot factor through the old F.

Proof. Projecting an equality of pairs gives $F(x) = F(y)$ and $r(x) = r(y)$. By (C.1), all old consumers and the new consumer agree. Conversely, equality of all consumer readouts first recovers $F(x) = F(y)$; pair extensionality then gives $F^+(x) = F^+(y)$.

Define the map from the quotient to the image by $[x] \mapsto F^+(x)$. The forward kernel implication makes it well defined; the reverse implication makes it injective; and a source witness for each image value makes it surjective. For a registered consumer c, define $\bar{c}(F^+(x)) = c(x)$. Kernel equality makes this independent of the representative. Every other factorization is forced to have this value at each image point by a corresponding x, proving uniqueness. Finally, if $r = g \circ F$, an old-face collision immediately yields $r(x) = r(y)$, contradicting the different new readouts. ■

On each of the three actual restriction domains, the new face also retains coordinates sufficient to recover the source restriction, making the face function itself injective. Consequently, every readout on that domain factors uniquely through the face's image. Uniqueness is asserted on the actual image; it does not require unique values at arbitrary points of a larger codomain that have no source witness.

C.3.2 Three actual extensions

Institutional authority. The institutional restriction domain consists of three paid essential-access channels: existing tenure and health, and the new platform gate. The complete old cultural face sends the corresponding restrictions to the same old cultural occurrence. The independent institutional readout retains the channel code, exact relation seed, actor, target role, and selection, admission, and material-service coordinates. If two such readouts agree, projection of the exact relation seed recovers the same source channel. The new readout is therefore injective. Tenure and platform have the same old face but different new readouts, providing an actual escape from the old kernel in C4.

The higher-level material connects the three positions to their respective relations, Payment, later material spans, and complete admissions from the same source. The new platform direction consumes an already-paid U8 and its actual admission; representation extension is not treated as another mother-law failure. Payment and the later material occurrence retain their own types. The institutional face outputs both the Payment-root next and the material-action target, connected by the original transport. Neither output can be discarded to manufacture a single occurrence.

Class and price positions. At the actual target depth after the first refusal, the restriction domain contains the controller and affected relational positions. They share the same Payment, relation event, and complete refusal face. The class readout returns the generated actor. The price readout contains that actor, tenure/provider, price-open, capture, the source-owned residual, root factorization, and whether the position is the actual payment counterparty. The last field is false for the controller and true for the affected position; their actors also differ. Thus even the complete old refusal face and Payment do not determine the new class and price positions.

The new face is the product of the old refusal face, class, and price, so it retains all old content. Source coordinates supply the exact counterparty incidence: the affected position is the counterparty actor of this actual activation. The controller's position does not make it the bearer of this payment. The existing no-forgery controls retain their actual source scope: inputs lacking forward control or original debt, and already-fulfilled inputs, cannot reconstruct authority through protective terminology. C4 gives the extension's exact kernel, quotient-image equivalence, and unique factorization of every registered consumer.

Spatial repair. On the same post-refusal Payment root and crossing, the original route source generates blocked and restored restrictions. The blocked exit-route fibre is empty. The restored restriction carries an actual route receipt pointing to a feasible departure edge at the first served slot of the next depth, with capacity 1. Its low-level coordinates are therefore

$$(\text{crossing}, \text{none}), \qquad (\text{crossing}, \text{some}(1)).$$

The independent accessibility readouts carry, respectively, an empty-fibre proof and an actual receipt; the capacity readouts are none and some 1. These values are generated by route feasibility and are not retrospective labels. The complete old art-production face agrees at the two points, while either new readout separates them. Append the new coordinates to the old face and apply C4 again.

C.3.3 How representation acquires root authority

Connections across roots use exact singleton restrictions. The source side is $\{x \mid x = \text{emitted}(2)\}$ or $\{x \mid x = \text{emitted}(5)\}$; the target side is the exact singleton restriction of the actual post-refusal Payment or actual route root. The two constant maps are inverse because every element at either end equals its retained original object. This is a constructive presentation between the restriction domains, not an identification of their ambient occurrences. A separate equality from the same source identifies the urban route root with the refusal-target Payment root retained by the class-price construction. The transport certificate keeps both original receipts.

The new faces are installed through projection cofaces of the original root. Each payload retains the original canonical ledger compilation and appends its new low-level face. Inherited injections transmit old installations through the layers, while adding a readout leaves the emitter and law intact. The institutional, class-price, and urban extensions are active at original currents $0, 2, 5$, with next currents $1, 3, 6$. The intervening refusal, teaching, and art-production layers retain their original installation positions, giving six inherited stages from 0 through 5.

There are two obligations here. Representation must have the consumer completeness of C4,

and installation must supply actual consumption and a successor in the original root. A quotient-space theorem alone does not establish that its quotient enters institutional execution. Conversely, stating that a field is installed does not establish the exact kernel theorem. The three actual whole consumers jointly consume source material, the kernel theorem, unique factorization, inherited installation, and the exact next.

C.4 Theory adoption, mother-law failure, and later retirement of an old claim

C.4.1 Theory effects precede the adoption certificate

Fix an old theory T, a source game, and a seed s from its trace. The theory-feedback compiler supplies a classification $\chi(s)$, a policy decision $\pi(s)$, a dependent effect type $E(s)$, and an actual effect $e_s : E(s)$. A separate interface supplies an effect consumer

$$K(s, e_s).$$

It consumes this actual effect; the value of $\chi(s)$ or $\pi(s)$ cannot take its place.

In the actual M13 instance, classification and policy are both true, and the effect is the existing localLifecycle. The adoption source adds causal theory recognition at the exact splice of the original joint runtime. This composite local consumer retains the existing relation consumer. The original root installation generates the adopted whole ledger, visit, selected row, and next. The ordinary next still has old theory T as its law surface.

Lemma C5 (An effect without a consumer is not the actual adopted effect). If $K(s, e)$ is empty for a candidate effect e, while the actual $K(s, e_s)$ is inhabited, then $e \neq e_s$.

Proof. If $e = e_s$, transporting the actual consumer along this equality produces an inhabitant of $K(s, e)$, contradicting its emptiness. ■

The lemma locates independent consumption of an effect in a typed occurrence relation. It also explains why a theory text, classification value, or policy label cannot replace the actual adoption object.

C.4.2 Mother-law audit and exact revision

The mother-law contract classifies each original occurrence into an active or inactive scope. An active scope supplies a row r and an obstruction whose claim is exactly r.claim. The deletion audit generates a near-miss from this actual source material. A mother-law breaking audit additionally requires an installed failure face at the same occurrence and an equality identifying that face's scope entry with r. A rooted gate further requires causal authority and a successor for the same row.

Proposition C6 (Mother-law failure generates same-occurrence self-U8). The actual M13 audit and its authority on the same row generate rooted failure and a minimal coface. The original exact revolution and its consumption supply self-U8, the revised first write, and the two rows' common successor.

Proof. Substituting the actual active evidence into the mother-law contract yields r and the obstruction targeting r.claim. The scope-entry equality identifies the installed failure face's entry

with r. The row's causal authority and successor then construct the rooted gate. The obstruction claimed by the failure object is therefore aligned with the object consumed by the original root.

The self-U8 construction is indexed by this exact revolution and exact consumption. Its current, root, first write, and old/new row next are projected from those indices and cannot be independently chosen. The consumption's first-write and two-row successor theorems show that the revised first write is the actual next and that the revised readouts of the old and new rows lead to a common successor. With these indices fixed, the self-U8 wrapper retains only its sealed witness, so any two such wrappers are equal. ■

Ordinary adoption preserves T, while the mother-law audit generates revision along its own failure path. These are distinct actual branches in the same system of constructions. Ordinary adoption does not require revision failure, and revision is not mechanically inferred from the classification value of ordinary adoption.

C.4.3 The live responsibility at first write and its subsequent fulfillment

Let q be the claim borne under the old authority, and q' the inherited obligation after revision. From the old mother-law disposition event, the source disposition compiler generates a native superseded rollover: the old authority receives its superseded disposition, while the inherited slot receives live q'. The semantic disposition may preserve, weaken, or replace; the actual M13 instance uses replacement. Equality of meaning and equality of responsibility lineage are separate coordinates. The actual content and bearer of q' are read from the original revised row, and the rollover preserves lineage and source anchor.

Proposition C7 (The revised first write retains debt, and later retirement has its own fulfillment). The actual first write targets revised visit 1, where the inherited slot is live q'; this target is therefore not closed. The original source subsequently generates an actual retirement event at revised current 1. Its consumer writes a fulfilled archive to the same inherited slot, so the original live claim disappears at this edge.

Proof. The revised first write corresponds to the original migration event at revised current 0. The native rollover's target-state rule directly makes the inherited slot live q', preserving lineage and source anchor through the same rule. If the slot were already closed, it would be a terminal archive, contradicting separation from the actual live constructor. Omitting the row from a sparse display does not change the actual slots, so omission is not retirement.

The later settlement consumer is indexed by revised current 1 and consumes the unique actual retirement event there. Its fulfillment rule changes the inherited slot to terminal final fulfilled and appends that event to the original history. The live witness in the source state and the impossibility of a live value in the target state jointly establish DisappearsAt. Indeed, comparing any candidate live value in the target with the terminal equality at the same slot gives a constructor contradiction. The first-write target visit is 1, so the later consumer continues the first write's actual successor rather than an external copy of the ledger. The original retirement run retains the equality between the later generated next and the two rows' common revised next. ■

C.4.4 Strict separation of ordinary and revised next

The original network in the actual M13 construction has support type

$$O = \mathrm{Bool},$$

whereas the revised network has support type

$$R = (\mathrm{Bool} \sqcup D) \times \mathrm{Bool},$$

where D is the direction type generated by rooted failure and contains completion. Consider the three points

$$r_0 = (\mathrm{inl}(\mathrm{false}), \mathrm{false}), \quad r_1 = (\mathrm{inl}(\mathrm{true}), \mathrm{false}), \quad r_2 = (\mathrm{inr}(\mathrm{completion}), \mathrm{false}).$$

They are pairwise distinct: the first two differ in their Bool value, and the third is separated from both by the disjoint-sum constructors.

Proposition C8 (The actual ordinary next differs from the revised next). The original network's ordinary authoritative successor differs from self-U8's revised authoritative successor.

Proof. If the successors were equal, first projecting their network and then Support would give $O = R$. Transport along this type equality yields an injection $R \to \mathrm{Bool}$. The injection sends r_0, r_1, r_2 into a type with only two values. If the image of r_0 is false, the other two images cannot both avoid false and remain distinct; the case where it is true is identical. At least two points therefore have the same image, contradicting injectivity. ■

This proof applies to M13 and to the policy-breaking point that consumes its separation result. The separate ordinary/revolutionary separation after the first refusal uses its own Current type and indices. The two actual inequalities retain their original objects; similarity of their names does not make their proofs interchangeable.

C.5 Four independent interfaces for further results

Capital relations, public functions, public production, and protocol fulfillment each have their own sources, runs, and completion objects. They share this paper's interfaces for native responsibility and original-ledger successors. The following applications read actual objects in their respective sources.

C.5.1 Minimal permission support for capital

Let L be the original raw law. Write $a_L(\tau, p, x) \in \{\mathrm{false}, \mathrm{true}\}$ for whether actor x is admitted to port p at technology state τ, and $c_L(\tau, p)$ for that port's controller. Define

$$\mathrm{NoAsymmetry}(L) \iff \forall u, p, x,\ a_L(u.\tau, p, c_L(u.\tau, p)) = \mathrm{true} \Rightarrow a_L(u.\tau, p, x) = \mathrm{true}.$$

A private phase is exactly a permission witness from the same source for which the premise is true and the conclusion false. The complete $\mathrm{NoCapital}(L)$ quantifies over all original states, words, and

relevant histories, simultaneously excluding private phases, capital kernels, proper titles, actual surplus, and fresh control in original histories.

Proposition C9 (The capital interface's law-wide characterization).

$$\mathrm{NoCapital}(L) \quad \Longleftrightarrow \quad \mathrm{NoAsymmetry}(L).$$

Proof. Suppose the complete NoCapital contract holds. Its private-phase component excludes permission asymmetry for every u, p, x. Given admission of the controller, split on the actor's admission value. True gives the conclusion; false produces an excluded private phase.

Conversely, suppose NoAsymmetry holds. Every private phase directly contradicts its implication. The checked part of a capital kernel retains its actual private phase. A proper title or surplus retains its original kernel dependency, so neither can exist. Fresh historical control retains its original history input, current original state, port, actor, and fresh phase. Substituting that input's law equality gives the same contradiction. Thus every universally quantified component of NoCapital holds. ■

The result gives a substantive control example. Replace a law's allowed function uniformly by false. The premise of NoAsymmetry is always false, so NoCapital still holds. Yet whenever technology, ports, and actors are inhabited, this law is not fully open. Absence of capital therefore cannot replace the conditions for open services or payment of needs. Full openness is a sufficient special case; uniform closure shows why this sufficient condition cannot be read as necessary.

The capital paper's complete root contract additionally takes an original production seed and gross claim, retaining interfaces for actual clearing costs, contribution, appropriation, and escape along finite histories. For every actual epoch and finite length, the source's history contract preserves migration of the entire original claim, prevents budget refilling, and accounts for materials and financial addresses. Evidence of capital regeneration supplies an exact permission discrepancy between the active handler and the claimed law. Its legal legislation programmes continue to fall under this paper's four-class root law and conditional U8. C9 determines permission support; these actual material and historical conditions remain necessary parts of their own contracts.

C.5.2 Why completing a public task cannot terminate old responsibilities

In an actual packet of the public-function source, the original responsibility type is the seven-way disjoint sum

$$\begin{aligned}\mathrm{Row} = {} & \mathrm{CurrentGrant} \sqcup \mathrm{PastGrant} \sqcup \mathrm{CurrentChallenge} \sqcup \mathrm{PastChallenge} \\ & \sqcup \mathrm{CurrentService} \sqcup \mathrm{PastService} \sqcup \mathrm{Finance}\,.\end{aligned}$$

Each row carries its own original vocabulary and complete native state. The census traverses the actual lists in order, retaining multiplicity. Current service rows are traversed by their original finite slot indices, and the finance row is appended at the end. For a row r, let $T(r)$ be the Boolean test that every slot of the row is terminal, and set

$$\mathrm{settled}(P) = \bigwedge_{r \in \mathrm{census}(P)} T(r).$$

Proposition C10 (Complete termination testing and residual response). For every actual packet,

$$\text{settled}(P) = \text{true} \iff \forall r, j,\ \text{TerminalAt}(r, j) \iff \text{WholeTerminationReceipt}(P).$$

Every original live slot forces this source's dispatch to take the continued branch. A past Grant whose authority has been withdrawn remains part of the test.

Proof. Case analysis on the seven constructors of Row shows that each row occurs in the census. The six families are obtained by original list membership or original Fin indices, and finance by the final singleton list. For any finite slot list, the all-terminal test is true exactly when the value at every index has a terminal archive. This follows by classification of each slot: live returns false, terminal returns true. The two finite all tests are therefore equivalent to terminality at every original row and slot.

WholeTerminationReceipt includes terminality of all responsibilities and terminality of all current and past services. The former universal statement implies the latter by restriction to the service subfamily. Conversely, projection of the all-responsibilities component gives the second equivalence. If a slot is actually live, universal terminality would also make the same slot terminal, a contradiction.

Dispatch first computes settled. Its true branch must carry the complete receipt above. Its false branch generates an actual request and the source-generated response following that request. From the current holder it reads either the holder's own exit intention or an intention for public revision. A successful original menu check generates a paid word; failure retains the original refused item and its specific shortage. The actual source executes that literal word. All old rows are transported through the actual input evolution and the subsequent response evolution. For any finite number of iterations, concatenate these actual evolutions. Induction on that number preserves each original origin, makes the complete original history a prefix of the new history, and transports each original financial slot through the actual ledger. ■

In the actual past-Grant control, the state saved after withdrawal still contains the original live slot. After any history of external input words, this original grant is transported to the actual response and still makes settled false. Completing the service task or clearing the original holder field therefore does not terminate all responsibilities. An actual paid exit does clear holder and offered in the same root next. The original Grant's native bearer release remains governed by its own native disposition rule.

The broader public-function interface generates four paid openings in one word, or an exact blocker, for every inventory, primitive input, and finite depth of this Bool/Unit source. It also retains all old cohorts, finance, the whole ledger, and literal next. The seven-family termination equivalence proved here is that interface's complete receipt gate. A response over a finite menu does not assert success for arbitrary sources or an infinite future.

C.5.3 Material conditions generate new cells and retain savings as executable capacity

The public-production source uses actor addresses in $\mathbb{N}$ and seven goods: seed, feed, life, spare, waste, energy, and labor. Technology has two Boolean coordinates; here we take public, improved technology $(\text{true}, \text{true})$. Parent address p and child address c are distinct, with $c \neq 3$, since address 3 has the original recycling role. Write s, f, e, ℓ, l, m for seed, feed, energy, labor, life, and spare, respectively; in particular, ℓ denotes labor and l denotes subsistence goods. Suppose the parent's inventory satisfies

$$(s, f, e, \ell, l, m) \geq (2, 8, 1, 2, 2, 2),$$

and every child inventory is nonnegative.

The source-generated replication word consists, in order, of the parent's labor audit, transfer of a replication kit from the parent, the child's production, the child's subsistence use, and the child's maintenance use. The replication kit contains

$$(1 \text{ seed}, 8 \text{ feed}, 1 \text{ energy}, 1 \text{ labor}, 1 \text{ life}, 1 \text{ spare}).$$

Proposition C11 (Original resources generate legal replication). These resource conditions generate itemwise legality and actual execution of the five-item original word. The complete financial claim matrix is preserved, and the child's subsistence and maintenance have actual payments within the word. Net inventory changes are:

Address	seed	feed	energy	labor	life	spare
Parent p	-1	-8	-1	-2	-1	-1
Child c	$+3$	0	0	0	$+5$	$+1$

Proof. The initial audit consumes one unit of the parent's labor, leaving one unit available for transfer. The parent's lower bounds make every debit of the replication kit payable, and both signer and sender are p. Adding the kit to the child's nonnegative inventory supplies the seed 1, feed 8, energy 1, and labor 1 required for production. Public, improved technology permits this production, whose actual credit is seed 3, life 5, and spare 1, with no waste. The child then pays for subsistence and maintenance from its own life and spare. Substitution at each step into $\text{stock}' = \text{stock} - \text{debit} + \text{credit}$ gives the net changes in the table.

None of the five items issues debt, assigns a claim, or settles a financial claim, so every item has zero claim delta. Summing pointwise at the original financial addresses preserves the whole claim matrix. The last two items have membership and Valid evidence in the actual execution, satisfying the actual use conditions for subsistence and maintenance respectively. Payment of needs thus comes from actual services in the word, not merely from positive final child inventory.

At original visits $9, 10$, the actual source applies this resource construction to two different parent/child pairs. The admitted word equals the actual submitted word, and the actual next equals the target state of that word under the original law. The source projection, whole ledger, and literal next jointly consume this result. ■

The net labor consumed by replication is 2, not 1. A one-unit labor dividend arises from a separate actual shared-audit construction. The original complete word at round 4 produces a paid

PaidMethod, retaining the original before state, audited item, membership, Valid evidence, and actual labor debit of 1. At round 7, reuse parameterizes and retargets the tail of the paid original word, checking every resource again at the new current and avoiding a repeated audit. By frame 11, total credits agree, and total debits differ only by one less labor unit at parent address 0; claim, technology, clock, and round also agree. Let $W_a(u)$ be the complete state obtained by adding nonnegative a only to labor at address 0. Then

$$u_{\text{shared}} = W_1(u_{\text{original}}).$$

Lemma C12 (Surplus preserves every old legal continuation). For every original state u, $a \geq 0$, and finite word w whose items are legal at their execution states from u, the word remains itemwise legal from $W_a(u)$, and

$$\text{execute}(W_a(u), w) = W_a(\text{execute}(u, w)).$$

Proof. For a single request, the signature, permissions, technology, and claim guard are unchanged under W_a. The right-hand side of the stock condition only gains a nonnegative amount, so the original debit remains payable. The stock update of the paid step commutes with adding a, and all other fields of the complete state agree. Induct on word length: apply the single-request result to the head and the induction hypothesis to the tail at the transported target state. ■

Substituting the actual u_{original} and $a = 1$ transports the cost saving into preservation of every old executable continuation. The saved cost, increased final inventory, and surplus retained through successors are three readouts of the same unit of labor, not three dividends to be added together. Public production's Participation construction and its two complete acceptance procedures separately account for production after exit, newborn responsibilities, and archives of original payments in their own 25-frame source. These results retain their own actual runs rather than being attached to this 11-frame comparison.

C.5.4 Completion of every branch in the receipt window

The protocol interface uses its own two-site source. A site's action reads only that site's queue and current signal. A notification is consumed only when the site is actually activated, its submitted reply is literally the required reply, and every reply item is valid. Original sends and deliveries pay their original costs, and reception records retain their own birth prefixes. Set the rotation period and source bounds to

$$c = 2\,\text{sites} = 4, \qquad a = 1, \qquad s = c(a+1) = 8, \qquad H = 3s = 24.$$

The public source programme votes at index 0, sends at $4, 12$, and uses the public quiet baseline elsewhere. Actual receptions add new notifications to the queue. The reply rule is compiled from that queue rather than selected according to whether the Goal has been completed.

Each of the two permitted deliveries has two original timing choices, giving

$$\Theta = \{(0,0), (0,1), (1,0), (1,1)\}.$$

The first delivery occurs at 5 or 7, and the second at 13 or 15. The reply position after each delivery is the first slot owned by site 1 in the following complete rotation. The send separation leaves room for a complete reply rotation after the latest first delivery.

Proposition C13 (The actual local fulfillment window). For this original source and every adaptive callback, if all actual inputs of the independent active executor satisfy the original communication admission condition, the executor completes the original Goal itself. Its actual history has length 24, and the queue and send-readback are jointly closed. The complete history is consumed by the same original root, preserving the whole ledger and literal next.

Proof. The central distinction is between finite path coverage and actual active execution. At each cut, the coverage procedure traverses every original admitted move at that cut. For each move it retains candidate paths with the same head move, checks that their entire local rule equals the rule generated from the current queue, checks equality of all sites' queues after consumption, and recursively checks the suffix. This procedure does not inspect the final Goal.

Induct on programme length. If coverage is true and the actual callback is compliant at the current cut, its actual move belongs to the original admission list and therefore has a checked branch. Equality of the complete rule and queue makes the active executor's actual next state equal to that branch's next state. Apply the induction hypothesis to the suffix. For the empty programme, coverage guarantees an empty suffix. Thus every compliant actual execution agrees with a complete path for some $\theta \in \Theta$ in state, status, and pending queue. The candidate path has not been supplied to the executor as an input.

For each of the four source-generated timings, all 25 historical snapshots retained across the cuts enter the service registry. A nonempty original intention in any snapshot can belong only to site 1's corresponding service window, where its original complete local reading is preserved. The original reply item in that window carries a local activation, the literal original word, itemwise Valid evidence, and membership in the actual record. It therefore supplies a complete service witness for each original intention. Actual reception and consumption separately give complete readback of both sends and the empty final queue; the original native-responsibility gate also closes along the same path. Hence all four source paths satisfy the entire Goal.

Transport this result along the complete path equality to any compliant callback, obtaining jointComplete. Each path has length H, so the actual execution also has length 24. The actual eager callback generates fresh deliveries from its own sent/past data and satisfies the original admission condition, so the premise is inhabited. Finally, the originally installed Census.answer is exactly the active executor's output. After success, the full history is literally the executed actual history, with no second recovery run appended. Readout factorization of the original facade supplies the whole ledger and literal next in the same execution. ■

This protocol completion, the task responses of public functions, the material replication of public production, and the law-wide support theorem for capital each consume their own source conditions. Their common interface consists of actual responsibility, the original ledger, and successors. A success certificate from one source does not supply another source's inputs for free.

Appendix D Main-text terms and original proof identifiers

The main text uses terms for institutional and social relations. This table connects them to common identifiers in the original proofs. A shared ordinary-language term does not remove an object's dependent indices. Written arguments appear at the listed locations; Appendix G fixes the source and machine-checked types.

Main-text term	Original proof identifier	Location in this supplement
Complete payment	`Payment`	A.3, B.3
Payment at a visit	`SourceBackedPaymentAtVisit`	B.3
Payment occurrence identity	`AnySourceBackedPaymentOccurrence`	Lemma B.3, Theorem B.9
Independent processing operation	`Consumer`	A.2, C.3
Actual successor	`actual next`	A.5, Theorems B.8–B.9
First-write successor	`firstSuccessor`	B.1–B.2
Same-source law revision	`self-U8`, `SameOccurrenceU8`	A.2, B.7, C.4
Sequential action	`sequential`	B.2
New responsibility admission	`debtAdmission`	B.1–B.2

Complete payment and occurrence identity. Complete payment includes actual performance and the complete resulting state of the responsibilities concerned. A payment at a visit has a type dependent on that visit. To compare payment occurrences at different visits, we first package the visit together with the payment, then compare identities within this common carrier. The injectivity conclusion in B.3 concerns this complete occurrence identity. The external bridge in A.3 retains the separate positions of the original source event and the target complete payment and connects them by the supplied transport.

Independent processing operation. A consumer in the original proof processes materials, effects, or responsibility according to an installed source. It can read performance, check an effect, or generate a corresponding disposition; each operation has its own input type and authority. In the main text it does not denote a person buying a service. Completeness of a representation of prices, responsibility contents, space, or theory is stated relative to a specified family of these operations.

Two kinds of successor. An actual successor is produced by the source execution and adopted by the subsequent run. A complete payment's responsibility poststate, the program after repair, and the next constitutional visit can inhabit different types. Calling them successors does not equate them. Theorems B.8 and B.9 in Section B.8 provide the relevant installation factorization and continuation of visits. A first-write successor is specifically the target of a first complete responsibility write; it is generated together with the destinations of all old rows and the newly born row.

Same-source law revision. This term denotes revision jointly supplied by exact failure in the original occurrence, authority on the same responsibility row, the revision core, and its actual consumption. It includes the revised first write and the successor relations of the old and new rows. Observing failure does not by itself supply these data. In A.2, conditional revision capability reads an actually inhabited revision branch. B.7 and C.4 provide their respective actual revisions without merging them into a single run.

Responsibility content and standing. The original term `claim` denotes content that must be explained or performed. In migration, its inherited status, lineage, source anchor, and later settlement are checked separately. `standing` denotes an action position or purchasing-power position already obtained from a source. A `backstop` is credit support; a `tender` is an actual submission for payment. The credit construction in C.2 generates purchasing-power standing from the submission and counterparty acceptance while reading the risk bearer from the still-open responsibility.

Complete foundation and total object. The original materials organize the complete foundation in C.1 through M_∞, the `MInfinity` family, and the actual theory-responsibility chain in C.4 through the `M13` family. The `M14` construction `canonicalSociety` retains the complete foundation, constitutional point, and regeneration consumers. The main text's $\mathcal{F}_h$, S_h, and joint regeneration carrier correspond to these generated foundations, total objects, and joint certificates. Their assembly preserves each component's own source; a claim of common occurrence still requires the corresponding installation equality.

Appendix E Canonical payment, open operation, and the complete economic process

E.1 Complete canonical preservation for every original payment

E.1.1 Inputs, occurrence indices, and the target projection

Fix an original old visit v, source index s, and boundary b, and take any cycle in the original registered type:

$$C : \mathrm{CycleAt}(v, s, b).$$

The cycle carries its original source and paid revision recursion. All preservation results below are generated from it. Write

$$i_d = b + d + 2$$

for the starting index of its lifecycle at depth d. The actual current of the post-settlement source at index k is $c_k = r_0(s) + k$. Denote its exact root event by ω_k, complete canonical current by T_k, native payment by $\bar{p}_k$, and complete future birth by γ_k. Thus $c_{k+1} = c_k + 1$.

The payment type P_k is indexed by v, s, k, the exact root event ω_k, and the actual exit there. An element contains a native write, a complete responsibility successor, exercise, response, leaving, and continuation outside the role for the same continuation action, together with external responsibility. The payment relation and actual settlement retain their original sources. This is a complete type carrying data. We quantify over every element of P_k, retaining the input rather than replacing it with a preselected canonical element.

A payment's target is projected from its original responsibility successor:

$$\mathrm{NextPaidBy}(p) = p.\mathrm{successor.targetOccurrence}.$$

For objects written at different dependent indices, $\simeq$ denotes equality after transport along the proved index equality. The formal source expresses this as heterogeneous equality. Such transport does not identify source and target occurrences.

Lemma E.1.1 (Target coherence of payments at one occurrence). For fixed v, s, k, ω_k and actual exit, and every $p, p' \in \mathsf{P}_k$,

$$\mathrm{NextPaidBy}(p) = \mathrm{NextPaidBy}(p').$$

Moreover,

$$\mathrm{NextPaidBy}(p) \simeq \omega_{k+1}.\mathrm{occurrence}.$$

Proof. In the fibre indexed by this exact root event and actual exit, the responsibility-successor carrier is a singleton type. Hence $p.\mathrm{successor} = p'.\mathrm{successor}$. Applying the target-occurrence projection gives the first equality. This argument uses only the singleton successor fibre; it neither declares the complete payment type a singleton nor establishes $p = p'$.

Take $p' = \bar{p}_k$. The native payment's successor current is $c_k + 1$. Since $c_k = r_0(s) + k$, we have $c_k + 1 = c_{k+1}$. After transport along this equality, the native successor's target occurrence is exactly the occurrence of ω_{k+1}, with no remaining choice in its definition. The first equality then gives the second claim for arbitrary input p. ■

E.1.2 Constructing each component of preservation

Given any $d \in \mathbb{N}$ and $p \in \mathsf{P}_{i_d}$, put $i = i_d$. Construct the paid step $Q(C, d, p)$ by defining its source payment to be the input p, its source whole ledger to be the original whole-ledger write-back of ω_i, and its target current to be T_{i+1}, generated by the original rule. Its target root event, payment, and future birth are respectively

$$\omega_{i+1}, \qquad \bar{p}_{i+1}, \qquad \gamma_{i+1}.$$

Here $\bar{p}_{i+1}$ is the target's own native payment. The preceding p produces the target occurrence; the target's own payment performs the new service and exit.

Lemma E.1.1 supplies the decisive connection:

$$\mathrm{NextPaidBy}(p) \simeq \omega_{i+1}.\mathrm{occurrence}.$$

The target compiler's actual compilation of ω_{i+1} is exactly the native write carried by $\bar{p}_{i+1}$. The actual exit extracted from that write is the actual exit of γ_{i+1}. The target future birth's root event and payment therefore recover those of T_{i+1}.

The original cycle recursion satisfies

$$i_d + 1 = i_{d+1}, \qquad \mathrm{cycleAt}(C, d+1) = \mathrm{stepAt}(C, d).\mathrm{nextCycle}.$$

Unfolding the actual lifecycle construction identifies, by transport, T_{i+1}, $\bar{p}_{i+1}$, and ω_{i+1} with the source current, source payment, and source root event of the next lifecycle. There is consequently one common target, and the preservation proof for every input payment uses this original connection.

This supplies all identity fields of the complete preservation carrier: the original input is unchanged; the source index is $b + d + 2$; the target index is $b + d + 3$; the actual target is the next lifecycle's source occurrence; the target future birth and target current have the same root event and native payment; and the original step generates the actual next cycle. The following constructions supply the remaining institutional content.

E.1.3 All institutional content at the same target

The complete exit constitution. Write $\mathsf{E}_C(d)$ for the complete exit constitution at depth d. It contains a common renewal witness, continuation code a, and actual service slot j. These are linked witnesses with the following properties.

- The renewal witness's exit code is exactly a. The code belongs to the original future-code carrier and admits continuation in the relation outside the role; it is neither idle nor captured.
- The original employment relation differs from this independent exit relation. The same a has a source-backed payment receipt and a route receipt reaching slot j of the actual settlement.
- The same a enters the actual lifecycle's occurrence sequence, response execution, continuation outside the role, external-residual disposition, and actual services and future. Both the outer authority and actual service–exit authority retain injectivity along the occurrence sequence, while the visible action may remain equal across stages.
- The activation seed for purchasing-power standing comes from the original actual trace, and the purchasing-power payment reaches its exactly generated successor. The lifecycle's source root event and actual payment recover, respectively, the original cycle's legislation span's source event and target payment.
- The original cycle state carries an actual same-source law-revision certificate. Its target core is the generated core for boundary $b + d$, its response and next state admit the original root factorization, and its actual next cycle satisfies the recursion above.

To construct this material, take the complete lifecycle already generated by the original cycle at depth d. Its actual exit renewal supplies concrete a, j. The surviving-code criterion supplies carrier membership, continuation outside the role, non-idleness, and noncapture. The same feasible edge supplies the payment and route receipts. The lifecycle's existing response, continuation, external disposition, actual services, and future supply the temporal material. The original cycle state's revision certificate supplies the exact target core and root factorization. Together these construct $\mathsf{E}_C(d)$.

The reverse direction is also checked component by component. Extract payment, route, and all temporal material for the same a, j from the explicit constitution. They reassemble the complete lifecycle contract, and the original equivalence for the maximum contract returns the corresponding lifecycle witness. "Complete" therefore refers to an explicit, checkable contract. Repeating the construction at $d + 1$ supplies the preservation object's target constitution $\mathsf{E}_C(d + 1)$.

Target future birth, channels, and external responsibility. The bearer's exit at γ_{i+1} is read from the installed root face there. Its payment occurrence and actual settlement equal the corresponding projections of the target's actual exit. The original birth bearer agrees with the counterparty identity of the purchasing-power relation. Leaving is an available continuation outside the role and differs from working. The original geometry has exit capacity 1, whereas the fully captured version has capacity 0. The target current, actual next occurrence, and the conclusion that the original birth is not replayed all retain their original occurrence indices.

For every channel χ in the original type EssentialAccessChannel, read its consumer from the target's installed root face. This type is the common carrier of the original tenure and health relation authorities. The consumer returns a source relation seed z_χ satisfying

$$\mathrm{actor}(z_\chi) = \mathrm{actor}(\chi), \qquad \mathrm{relationOf}(z_\chi) = \mathrm{relation}(\chi).$$

Its service receipt comes from the channel's slot in the actual settlement. Its feasible exit edge is generated from that same actual settlement, the original payment receipt, and the route receipt. The payment occurrence and actual settlement retained by the consumer are precisely the two projections of the target's actual exit. The independent processing operation has the original root factorization, and the consumer's actual next occurrence equals

$$\mathrm{emit}(c_{i+1} + 1).$$

Channel consumption therefore returns to the next occurrence generated by the target's own payment.

The target's local open-responsibility type is empty in this construction. At the same time, its actual exit retains external payment-source ownership and an independent root consumer. Each is obtained from the original external-responsibility construction. Local emptiness supplies no deletion of that external responsibility.

All this target material is obtained from the original construction of T_{i+1}. Quantifying over the input p requires no additional premise supplying a target future birth, future channel, or external consumer.

E.1.4 Why exact deletion and divergence reports fail

The complete preservation consumer also checks nine exact counterexamples. The first four claim, respectively, that the input payment's actual next occurrence differs from the proved target, or that the target current, target payment, or target root event differs from the corresponding source object of the next lifecycle. Each contradicts its matching equality in E.1.2.

The remaining five require deletion of the target exit constitution, target future birth, the actual consumer of a specified original channel, external source ownership, or the external root consumer. Each asserts emptiness of the corresponding type. Section E.1.3 supplies an actual element of every such type. Applying the asserted emptiness to that element yields a contradiction. The channel argument applies to arbitrary χ, retaining the original universal quantifier.

The target lifecycle also carries its original capture and Job Guarantee counterexamples, the impossibility of reporting a complete lifecycle under full capture, and the distinctions between external source ownership, its consumer, and local emptiness. Price and action observations at every observation depth o, and administrative observations for every Boolean labelling function, retain their original impossibility of faithful reconstruction. The price and action results concern the original absolute-prefix observers; the administrative result retains quantification over the labelling function. Each follows from its own original witnesses with equal readouts and different authorities, and is then installed in the same target's preservation consumer. The complete counterexample proofs are given in the lifecycle and observation sections of this appendix. This assembly does not turn a prefix observer into a truth criterion for an individual target.

E.1.5 Every finite request and the actual whole construction

Proposition E.1.2 (Preservation with full quantifiers). For every v, s, b, C, o, d and every $p \in \mathsf{P}_{i_d}$, the preservation object of E.1.2–E.1.4 exists with all its coherence properties and exact counterexample conclusions.

Proof. From C, d, p, first generate the paid step indexed by the original input as in E.1.2. Supply the exit constitutions at both ends, target future birth, channels, external consumer, and counterexample conclusions from E.1.4 to their corresponding fields. Every field has already been generated from the fixed input. There is no additional premise that preservation holds or that a target exists. The observation parameter o specifies only the finite scope of the original observer conclusions; the input payment remains arbitrary in its exact original type. ■

Given arbitrary $n \in \mathbb{N}$, apply Proposition E.1.2 to every $d \in \{0, \dots, n\}$ and every $p \in \mathsf{P}_{i_d}$. This gives the per-payment conclusions throughout the requested prefix. Apply it once more at $d = n+1$ to obtain the actually generated next edge. The original finite consumer uses observation parameter $n + 1$ for the prefix and $n + 2$ for the next edge, retaining the exact lifecycle counterexamples at each corresponding target.

The original registration in S_h supplies, for every v, s, b, n, an actual initial cycle and this finite certificate. The finite result is therefore instantiated in an inhabited cycle type. The strict-progress consumer below uses the whole construction's registered root-law cycle and the same initial cycle, inheriting this section's conclusions from its existing payment-preservation field. The two main results can thus be assembled at this common instance.

E.1.6 The roles of canonicity and inhabitedness

In the fixed source program used here, actor seeds, genealogy rules, responsibility mechanisms, and their consumers have already been specified. The named construction $S_h = \mathrm{canonicalSociety}(h)$ has the explicit foundation-horizon parameter h. Its registered fields in turn provide universal functions over old visits, source indices, boundaries, and finite requests. Canonical currents, generated future-birth tokens, and paid steps at fixed indices have their respective canonical equalities. This section additionally establishes coherence of the target projections of original payments.

The inhabited result for root-law completeness and conditional revision is proved in Theorem 4.3 and Appendix A. For each prescribed parameter tuple, the original registered recursion jointly supplies a complete external-disposition witness, root-law completeness, and conditional revision

capability at the same generated constitutional point. "The same" refers to the common point supporting these conclusions for that parameter tuple.

These results establish neither equality of all whole objects satisfying the society contract nor uniqueness up to isomorphism. Canonical has its mathematical sense of a standard construction from given data, equipped with the proved local coherence relations. The actual construction and its common constitutional point establish society-level inhabitedness and joint properties.

E.2 Strict canonical progress of the paid frontier

E.2.1 The actual objects recorded by the frontier

Continue to fix v, s. Let γ_k be the post-settlement future birth already completely generated by the original rules at index k. For arbitrary starting index a and length m, define

$$H(a,m) = [\gamma_a, \gamma_{a+1}, \dots, \gamma_{a+m-1}], \qquad A(a,m) = \{K_{a+r} \mid r < m\},$$

where $H(a,0)$ is the empty list and $K_k = \kappa(\gamma_k)$ is the complete actual service–exit authority at depth $c_k = r_0(s) + k$. This object retains the original actual visit, payment occurrence, actual settlement, feasible exit edge, and uncaptured renewal.

Its connection with the original write can be checked directly. At a fixed actual visit, the payment, settlement, feasible edge, and renewal in a complete service–exit object each equal the canonical construction at that visit. Substitute these equalities in turn. The remaining proof fields agree, so the fibre has at most one element. The actual exit extracted from γ_k's native payment write therefore equals the actual exit contained in K_k. Thus K_k reads the exit performed by this native write; its identity is supported by that execution.

Lemma E.2.1 (Index injectivity of actual authority). $K_j = K_k$ implies $j = k$.

Proof. Apply the payment-occurrence projection to the equality of complete authorities. The two original actual payment occurrences are equal. Index injectivity of the original periodic payment occurrences gives equality of their actual visit depths, $c_j = c_k$. Substituting $c_j = r_0(s) + j$ and $c_k = r_0(s) + k$, and cancelling the common initial value, yields $j = k$. ■

E.2.2 The full proof of proper inclusion

The finite-list definition gives

$$K \in A(a,m) \quad \Longleftrightarrow \quad \exists r < m,\ K = K_{a+r}.$$

It also gives the exact list identity

$$H(a, m+1) = H(a,m) +\!\!+ [\gamma_{a+m}].$$

The first statement follows from the pointwise image of the original list, and the second from the final-element decomposition of a natural-number prefix.

Lemma E.2.2 (A new member outside the entire old image). For every a, m,

$$K_{a+m} \in A(a, m+1), \qquad K_{a+m} \notin A(a, m).$$

Proof. For the first statement, take offset $r = m < m+1$. If the second failed, the membership criterion would give some $r < m$ with $K_{a+m} = K_{a+r}$. Lemma E.2.1 would imply $a + m = a + r$, hence $m = r$, contradicting $r < m$. This argument excludes every old offset, so the new member lies outside the entire old image. ■

Lemma E.2.3 (Exact characterization of the frontier order). For every a, m, n,

$$A(a, m) \subseteq A(a, n) \iff m \leq n, \qquad A(a, m) \subsetneq A(a, n) \iff m < n.$$

Proof. Suppose $m \leq n$. Any old image member has an offset $r < m$, by the membership criterion. The same offset satisfies $r < n$, so the member belongs to the new image. If $m < n$, this inclusion is proper: K_{a+m} belongs to the image of length n and, by Lemma E.2.2, does not belong to the image of length m.

Conversely, suppose $A(a, m) \subseteq A(a, n)$ but $n < m$. Applying the preceding paragraph to $n < m$ gives $A(a, n) \subsetneq A(a, m)$, contradicting the reverse inclusion. Hence $m \leq n$. If inclusion is proper, equal lengths would make the sets equal. Thus $m \neq n$, and $m < n$. ■

The main text uses notation that includes the last indexed occurrence:

$$H_i = H(0, i+1), \qquad A_i = A(0, i+1).$$

Consequently,

$$H_{i+1} = H_i +\!\!+ [\gamma_{i+1}], \qquad A_i \subsetneq A_{i+1}, \qquad K_{i+1} \notin A_i.$$

E.2.3 A strict order with complete constitutions at both ends

For any original cycle C, define

$$C_r \prec_{\text{can}} C_t \quad \iff \quad \mathsf{E}_C(r) \wedge \mathsf{E}_C(t) \wedge A(0, i_r + 1) \subsetneq A(0, i_t + 1).$$

Proposition E.2.4. For every $r, t \in \mathbb{N}$,

$$C_r \prec_{\text{can}} C_t \iff r < t.$$

This relation is irreflexive and transitive.

Proof. Section E.1.3 constructs $\mathsf{E}_C(r)$ and $\mathsf{E}_C(t)$ at arbitrary depths. Lemma E.2.3 makes the third conjunct equivalent to $i_r + 1 < i_t + 1$. Substitute $i_d = b + d + 2$ and cancel the common terms to obtain $r < t$.

Setting $t = r$, irreflexivity follows from irreflexivity of strict inequality on the natural numbers. If $C_r \prec_{\text{can}} C_t$ and $C_t \prec_{\text{can}} C_u$, retain the source constitution of the first relation and the target constitution of the second, and apply transitivity of proper inclusion. This gives $C_r \prec_{\text{can}} C_u$. ■

E.2.4 The new member and the original input payment share one edge

An increasing index does not by itself complete Theorem 1.3. The new actual authority must be placed on the original paid edge. Fix arbitrary C, o, d and $p \in \mathsf{P}_{i_d}$, and put $i = i_d$. Choose the original responsibility-difference construction's boundary

$$\beta = i - 1 = b + d + 1.$$

Since $i = b + d + 2 \geq 2$, natural-number truncated subtraction causes no ambiguity. The construction's left and right actual replays occur at

$$\beta + 1 = i, \qquad \beta + 2 = i + 1.$$

By definition, their complete authorities are K_i, K_{i+1}, and their root events are transport-equal to the source and target root events of $Q(C, d, p)$. The source retains the original input p, and Lemma E.1.1 gives

$$\mathrm{NextPaidBy}(p) \simeq \omega_{i+1}.\mathrm{occurrence}.$$

Thus the member proved by Lemma E.2.2 to be outside the old image lies at this very target produced by p.

The same-edge connection also retains the following responsibility structure. The left replay's root-event whole-ledger write-back equals the original paid step's source whole ledger. The right actual exit equals the step's target actual exit, with source ownership of its external responsibility and an actual independent consumer. The root occurrence of its service payment is the exact event of the original independent root; the installed payload retains the original inquiry–revision source entry. This independent service root and the outer post-settlement root have their own types. The proof retains the source relation between them rather than equating their occurrences.

For any original channel χ, the seeds returned by the left and right consumers each recover the same original actor and relation. The left consumer's payment is the left actual exit's payment; the right consumer's payment is the right actual exit's payment. The right side's own native payment performs the new service. The original input p producing the target, the target native payment performing the new service, the retained channel identities on both sides, and the retained old whole ledger therefore form one checkable payment–responsibility connection.

The future births at both ends also carry the full certificates listed in E.1.3: actual exit from native write, installed bearer exit, payment and service agreement, the same original birth bearer, continuation outside work, exact capacity, coexistence of local emptiness and external responsibility, actual next occurrence, and non-replay of the original birth. The new member is actual by virtue of its own payment, service, and responsibility source.

The joint object's root-law transition retains the original lifecycle offset. Its root-law parameter is still i, but that law's current visit has actual occurrence $i + 2$. Two original payments connect the lifecycle start to this occurrence:

$$i \longrightarrow i + 1 \longrightarrow i + 2.$$

The root law's own actual successor payment then connects

$$i+2 \longrightarrow i+3,$$

while the target root law has parameter $i+1$. The target of this third edge is that target law's actual occurrence. The complete root-law consumer separately reads source and target grammars, actual and search coverage, whole ledgers, arbitrary exact payments, and the generated next root law. It is assembled with the first paid edge in one object through the original lifecycle, retaining all three actual edges and their respective identities.

E.2.5 The joint main result and finite consumer

Proposition E.2.5 (Preservation and novelty on the same paid edge). For every original cycle C, every observation depth o, every $d \in \mathbb{N}$, and every original payment $p \in \mathsf{P}_{i_d}$, the following certificates hold jointly:

$$\begin{gathered}\text{complete canonical preservation of the original input } p,\\ \text{with the complete target certificate,}\\ C_d \prec_{\mathrm{can}} C_{d+1},\\ H_{i_d+1} = H_{i_d} +\!\!+ [\gamma_{i_d+1}],\\ K_{i_d+1} \in A_{i_d+1} \setminus A_{i_d},\\ \mathrm{NextPaidBy}(p) \simeq \omega_{i_d+1}.\mathrm{occurrence}.\end{gathered}$$

These certificates jointly retain the original whole ledger, external-residual consumer, actor and relation of every original channel, complete future births at both ends, and the original root-law transition of E.2.4.

Proof. Take complete preservation from Proposition E.1.2 and the strict order from Proposition E.2.4 applied to $d < d+1$. Take the list equality from the original prefix's final-element decomposition. Lemma E.2.2 gives a new member outside the old image; the left and right replay identities in E.2.4 identify it with the actual right authority. The same-edge connection supplies the input payment's actual target, original whole ledger, external consumer, and all channel identities. Construct each future birth at its original index, and take the root-law transition from the corresponding lifecycle of the same original cycle. Every component uses the fixed C, d, p and its actual target, so they assemble jointly. ■

For a finite request n, the original consumer retains the complete root-law certificate of the same initial cycle. For every original payment at each $d \in \{0, \dots, n\}$, apply Proposition E.2.5 with $o = n+1$, and apply it again with $o = n+2$ to every original payment at $d = n+1$. Every strict comparison $r < t \le n+1$ follows directly from Proposition E.2.4.

The strict-progress consumer of S_h extracts an actual initial cycle and its existing proof from the original root-law registration, then applies this finite construction to that same initial cycle. Thus no alternative initial society is chosen to supply progress. Even at request parameter $n = 0$, the original independent root's initial visit with $s = b = 0$ yields two actual strict edges, $0 \prec_{\mathrm{can}} 1$ and $1 \prec_{\mathrm{can}} 2$.

E.2.6 What the strictness distinguishes

The first control lies inside the same original cycle. At any pair of adjacent steps, the original exit capacities both equal 1, so instantaneous capacity is unchanged, while Proposition E.2.4 still gives strict progress. The second control lies on that same edge: the left and right actual paid replays have equal real pulse normal forms but different actual authorities K_i, K_{i+1}, and their paid frontiers stand in proper inclusion. Repeated visible readouts do not erase the responsibility identity of a new service occurrence.

Two further finite counterexamples test the two requirements of the strict order. If the old image A is entirely preserved but no member is added, $A \subsetneq A$ is false. If the old image is $\{0\}$ and the new image is $\{1\}$, then 1 is indeed outside the old image, but 0 has been deleted, so $\{0\} \not\subsetneq \{1\}$.

Finally, take $f(n) = \lfloor n/2 \rfloor$. For any n, choose $m = 2(n+1)$. Then $m > n$ and $f(m) = n+1 > f(n)$, so every finite cutoff has a later strictly greater value. Yet $f(0) = f(1) = 0$, so growth is not strict at every tick. The per-payment conclusion of this section is supported by the target future birth actually generated at every step of the original cycle. What increases strictly is the image of fulfilled occurrences with the whole old image retained. This cannot be replaced by per-tick growth in the number of currently available actions, income, or welfare.

E.3 The complete contract and reconstruction of the paid-exit lifecycle

This section proves Theorem 5.5. The quantifiers throughout are

$$o : \mathrm{OldVisit}, \qquad a, i \in \mathbb{N}.$$

Here o is the prescribed old-visit parameter, a fixes the source, and i selects the starting point in the paid chain after joint settlement. The chain index i is distinct from the actual service depth, which the original construction computes from the occurrence.

E.3.1 Three payment edges and four occurrences

Write L_i for the original generated standing renewal. Its source and target incidences lie at indices $i, i+1$, and the source payment's actual successor is the target root occurrence. The target reads the exit bearer anew from its own payment, so the same action content acquires new authority at a new occurrence.

From L_i, obtain the target bearer's leave execution X_i; from X_i, obtain continuation outside the role C_i; and from C_i, obtain external-responsibility disposition D_i. These are dependent readouts of one original production chain. Let F_i be the further standing renewal and T_i the further paid step. The three payments are

$$p_0 = L_i.\mathrm{sourcePayment}, \quad p_1 = C_i.\mathrm{executedPayment}, \quad p_2 = C_i.\mathrm{successorPayment}.$$

The four occurrences are, in order, the source and target occurrences of L_i, the successor root occurrence of C_i, and the target root occurrence of T_i. Write $s(L), t(L), e^+(L)$ for a renewal's source payment, target payment, and target root occurrence, and Q_i for the successor standing

renewal used by C_i. The original construction supplies the following interfaces, each comparing objects in its prescribed type.

Interface	Exact equality
Standing renewal enters execution	$t(L_i) = p_1$
Execution enters the source of continuation renewal	$s(Q_i) = p_1$
Target of continuation renewal	$t(Q_i) = p_2$
Source of further renewal	$s(F_i) = p_2$
Target payment of further renewal	$t(F_i) = t(T_i)$
Target occurrence of further renewal	$e^+(F_i) = e^+(T_i)$

Together they establish the three edges

$$p_0.\text{nextPaidBy} \simeq e_1, \qquad p_1.\text{nextPaidBy} \simeq e_2, \qquad p_2.\text{nextPaidBy} \simeq e_3. \tag{E.3.1}$$

Here $\simeq$ is dependent equality, HEq: dependent indices are aligned through the original successor interface before the occurrences are compared. The three proofs use, respectively, the source payment's target occurrence, the executed payment's target occurrence, and the further paid step's target occurrence. The six interfaces align the payments appearing in these proofs. Expanding their producers yields the same original payment or root occurrence on both sides. Thus a separately existing successor cannot substitute for the successor of this execution.

E.3.2 The independent temporal contract

Let c be an action code in the original exit-relation compiler. The temporal contract $R_i(c)$, PaidExitLifecycleRootTemporalAt, contains all the following material.

1. **The occurrence chain.** Equation (E.3.1) and the six payment interfaces in the table.
2. **Response and the bearer's execution.** The target response action equals the target execution action and the original (t_1, leave) action. The self-action power uses exactly this action. The original employment relation has employed source role and released target role, and loses the work future. The action is uncaptured under the installed condition, and the separate termination controller differs from the worker.
3. **Continuation outside the role.** The code c equals the successor continuation action's code in C_i, and that action equals the preceding execution action. The independent relation has phase zero and independent source and target roles. It supplies an actual future and its occurrence witness, actionable exit, and noncapture evidence. The type reporting another employment-loss execution power at this phase-zero relation is empty.
4. **External-responsibility disposition.** The compilation at e_2 is the native write of its own p_2; actual exit is obtained from that write. Original source ownership, the independent root processing operation, exact actual settlement, and root answer-and-next factorization all exist. The local open-responsibility type is empty, while the external-responsibility type attached to this occurrence is nonempty.
5. **Every channel and the future.** For every original EssentialAccessChannel, there is a consumed paid service at indices $i + 1, i + 2, i + 3$. The whole-ledger successor of D_i equals

the successor of p_2. The further target retains an external-responsibility processing operation, and its payment generates another occurrence.

6. **Authority and service-occurrence distinctions.** Both maps from the four stages to outer authority occurrences and to actual service occurrences are injective; the action readout is constantly the original leave action.

Continuation in item 3 has its own source type. The actual leave in the phase-one employment relation already occurred in item 2. The phase-zero independent relation carries the same action content forward; it does not thereby generate another employment loss.

In item 4, the universal local-disposition function exists by elimination from an empty domain:

$$\left(\prod_{\delta:\mathrm{OpenLocal}(e_2)} \mathrm{Disposition}(\delta)\right) \neq \emptyset, \qquad \mathrm{OpenLocal}(e_2) = \emptyset.$$

A disposition witness containing an actual local responsibility row is nevertheless empty. Meanwhile, the original payment supplies $\lambda_i : \mathrm{ExternalLiability}(e_2, p_2)$. A proposed function saying that an empty local ledger entails absence of external responsibility therefore yields a contradiction when applied first to the known local emptiness and then to λ_i. The original payment and cross-root provenance carry that responsibility; a universal disposition over an empty domain cannot replace it.

The six groups of fields are obtained component by component from $L_i, X_i, C_i, D_i, F_i, T_i$. To construct $R_i(c)$, take c to be the successor action code of C_i. No future, payment, responsibility, or processing operation is supplied by the caller.

E.3.3 Both directions of the full semantic contract

Let σ_i be the actual service depth of C_i, and let J_i be the finite index type of the original service list at that depth. Define

$$\begin{aligned} S_i(c,j) \quad \Longleftrightarrow \quad & c \in \mathrm{Carrier} \ \wedge \ \mathrm{Future}_{\mathrm{independent}}(c) \neq \emptyset \\ & \wedge \neg\mathrm{Idle}(c) \ \wedge \neg\mathrm{Captures}(c) \\ & \wedge \mathrm{PaymentReceipt}(c) \neq \emptyset \\ & \wedge \mathrm{RouteReceipt}(c, \mathrm{ActualServedCrossing}_{\sigma_i}(j)) \neq \emptyset. \end{aligned} \tag{E.3.2}$$

The payment receipt uses the original exit program's purchasing-power standing. The route ends at the j-th actual service in the original exact settlement. The two receipts cannot witness different codes or different service slots.

Proposition E3.1. Let M_i be the corresponding type ProofCarryingPaidExitLifecycleMaximumAt. Then

$$M_i \neq \emptyset \quad \Longleftrightarrow \quad \exists c, \exists j : J_i, \ S_i(c,j) \wedge R_i(c). \tag{E.3.3}$$

Proof. The data of M_i are exactly one actual feasible exit renewal r and the temporal contract indexed by $r.code$. The exact characterization of $r.feasible.codeSurvives$ yields inventory membership, an independent future, nonidleness, and noncapture. The two receipts of the same feasible edge supply payment and route. Taking $j = r.slot$ proves the right-hand side.

Conversely, suppose the right-hand side is given. Reassemble its first four conditions through the original surviving-code criterion, obtaining membership of c in this program's surviving exit cone. Using the same c, j, payment, and route, generate a feasible edge q in the original actual settlement. Put $r = (c, j, q)$ and pair it with the given $R_i(c)$. This is an element of M_i. The reverse construction does not invoke the canonical maximum-lifecycle producer, so the equivalence carries an independent reconstruction proof. This proves the proposition.

Proposition E3.2. The type M_i is nonempty for every o, a, i.

Proof. At σ_i, the original noncapture geometry supplies an actual feasible renewal whose code is leave. The original continuation action has the same code. Transport the temporal contract constructed in E.3.2 along that code equality and pair it with the renewal. The result belongs to M_i.

E.3.4 The same action, distinct authority, and the full-capture counterexample

The four stages are $s : \mathrm{Fin}\, 4$. Their authority positions are the values of the original reachable-position function at $i + s$. That function is exactly a fixed source starting position plus the index. Equality of two outer authority packages therefore yields equality of positions; canceling the common summands gives equality of stages. The original injectivity of actual service occurrences gives the second stage injection. Both compare occurrence identities: the actual service occurrence κ_s is not the numerical capacity K.

All stages still read as leave. Stages $0, 1$ give a concrete pair with the same readout, distinct authority, and distinct κ. A left inverse recovering stages from the action readout would make that readout injective and force $0 = 1$, a contradiction.

The full-capture contract replaces the original noncapture compiler in (E.3.2) by the original full-capture compiler, retaining the other actual material. A witness would have one of the three codes idle, work, or leave. Idle violates nonidleness; work has no future in the independent relation; leave is captured by full capture. All cases are impossible. The full-capture geometry's own equivalent characterization therefore makes actionable exit renewal empty. The positive temporal chain, bare future, and external responsibility remain. The counterexample locates the failure precisely in whether the bearer can use the action independently.

The original direct consumer is proofCarryingPaidExitLifecycleMaximumDirectConsumer. M14's paidExitLifecycleMaximum field actually takes this consumer, and canonicalSociety_paidExitLifecycleMaximum exports it from that field. The independent equivalence nonempty_iff_maximumContract establishes the full two-way contract; nonemptiness of the total object does not replace that proof.

E.4 Structural anti-retaliation and strict counterexamples to administrative readouts

This section proves Theorem 5.6, still for arbitrary o, a, i. Let L_i be the original standing renewal in E.3 and $p = L_i$.targetPayment. All actual content of the anti-retaliation object is read from this target payment.

E.4.1 The target payment's own response

Let r_p be the payment's response root read and b_p the bearer obtained from it. Write α_p for the response action, c_p for its code, and α_{leave} for the original complete leave action. Then

$$\begin{aligned} b_p &= \text{choose}(r_p), \\ \alpha_p &= b_p.\text{exerciseAction} = p.\text{exerciseAction} = \alpha_{\text{leave}}, \\ c_p &= \text{code}(\alpha_p) = \text{leave}, \qquad \neg\text{Captures}_{\text{original}}(c_p). \end{aligned} \tag{E.4.1}$$

The response occupies the original root's bearerExerciseResponse projection. Choice here extracts a bearer from an already generated nonempty read; no external Boolean verdict saying that exit should be allowed is supplied.

The response retains the actual source trace of the bearer's action. It separately retains the source trace of the employment loss and termination action, with termination controller distinct from worker. Their respective action compilers supply the powers of these two actions. The structural response in (E.4.1) does not yet assert that the bearer's leave was executed; the subsequent execution construction in E.3 establishes that fact.

For any original service channel q, the target paid step supplies a consumed channel T_q. Its actual exit is the actual exit of p. Its processing operation reads the payment and settlement there and satisfies

$$T_q.\text{actualNext} = p.\text{nextPaidBy}. \tag{E.4.2}$$

The same payment supplies external source ownership, an independent root processing operation for this actual exit, and its whole-ledger successor, with

$$p.\text{nextPaidBy} = \text{emit}(\text{current}(p) + 1). \tag{E.4.3}$$

Equations (E.4.1)–(E.4.3) thus retain response, actual service, responsibility, and continuation together.

E.4.2 Two capture geometries and four empty fibres

The same c_p is uncaptured under the original compiler and captured under full capture. The original finite geometry gives

$$K_{\text{original}} = 1, \qquad K_{\text{full } capture} = 0. \tag{E.4.4}$$

This comparison fixes the action, payment incidence, and original relation, changing only the capture compiler. The full-capture comparison object is itself nonempty and does not contradict the positive object.

Four exact false-report types retain the original anti-retaliation object and its payment, respectively keeping the original policy/price, service, administrative, or employment fields, and then assert capture of the original response action under the original capture compiler. Their source types are:

- `ExactAfterRefusalOwnPolicyPriceRecaptureAt`

- `ExactAfterRefusalServiceRecaptureAt`
- `ExactAfterRefusalAdministrativeRecaptureAt`
- `ExactAfterRefusalEmploymentRetaliationRecaptureAt`

For any report x, its added capture assertion contradicts the final clause of (E.4.1). All four types are therefore empty. A further false-report type retains the genuine full-capture comparison but reports the same action as uncaptured. Its actual capture witness similarly contradicts the added negation. Since the full-capture comparison is inhabited, no function can reconstruct such a false noncapture report from every comparison object.

These counterproofs concern exact retypings retaining the prescribed original material. They do not treat a new institution with changed payment, service, or action provenance as another name for the original response.

E.4.3 No administrative labeling recovers occurrence authority

Let $\Omega_{o,a}$ be the dependent-sum type of original renewal occurrences, and choose any $\lambda : \Omega_{o,a} \to \mathrm{Bool}$. The original administrative readout is

$$F_\lambda(x) = \mathrm{administrativeReadout}(\lambda(x)).$$

The original rule gives the same readout for either Boolean value. For the source and target authorities x_0, x_1 of E.3, it follows that

$$F_\lambda(x_0) = F_\lambda(x_1), \qquad x_0 \neq x_1, \qquad \kappa(x_0) \neq \kappa(x_1). \tag{E.4.5}$$

Strict increase of the actual position and injectivity of service occurrences prove the inequalities, independently of the choice of label.

If $r \circ F_\lambda = \mathrm{id}$, applying r to the readout equality gives $x_0 = x_1$. If $g \circ F_\lambda = \kappa$, it gives $\kappa(x_0) = \kappa(x_1)$. Both are contradictions. A faithful reconstruction requiring both occurrence and authority recovery is consequently impossible as well. This proves a universal statement over labeling functions, rather than exhibiting only one defective label.

The direct consumer proofCarryingPaidSuccessorAfterRefusalAntiRetaliationDirectConsumer consumes the positive response, every channel, both geometries, and these counterproofs. The corresponding M14 field and canonicalSociety_paidSuccessorAfterRefusalAntiRetaliation retain that complete consumer.

E.5 Two complete interfaces for nonemployment relation exits

This section proves Proposition 5.7. Fix

$$o : \mathrm{OldVisit}, \quad a, b \in \mathbb{N}, \quad c_0 : \mathrm{CycleAt}(o, a, b), \quad h, d \in \mathbb{N}.$$

The cycle-input type contains an actual source morphism and its independent processing evidence. Put $i_d = b + d + 2$. The finite exit constitution in this section and the joint pulse in E.6 use this same c_0.

E.5.1 Bearer alignment for the tenure relation

The original tenure seed generates the subject t, relation ρ_t, controller action, and interest. Its power gives

$$\text{freedomChange} = \text{lost}(\text{tenure}), \qquad \text{phase}(\rho_t) \neq 0. \tag{E.5.1}$$

The entitlement is actionable at the source, and its code is absent at the target. The target role is computed by the relation as the original tenure-provider role. The original ExactTenurePaymentPurposeAt identifies this relation.

Bearer alignment uses two existing provenance equations:

$$\text{bearer}(t) = \text{parentBearer}, \qquad \text{bearer}(\text{worker}) = \text{parentBearer}. \tag{E.5.2}$$

The first follows because the tenure subject is exactly the original payment counterparty; the second follows from the original generated worker's lineage. Their composition gives equality of the common bearer while retaining the respective relation roles and subject constructions.

The essentialAccessTenure coordinate in the original root law's actual scenario has passed its admission audit, and exact-once coverage supplies the unique corresponding point. The finite constitution used by the positive object is precisely the source constitution of the same joint pulse's root-law transition. Eliminating its stored operational incidence yields

$$\exists r, c, j: \quad r.\text{exitCode} = c \ \wedge \ \text{Semantic}_d(c, j) \ \wedge \ \text{Temporal}_d(c). \tag{E.5.3}$$

The semantic part comprises the six same-code, same-service conditions in E.3.3. The temporal part retains external-responsibility disposition, all services and the future, and injectivity of actual service occurrences. The independent consumer eliminates this incidence once and sends its fields to the appropriate processing operations; it selects no parallel ordinary-exit program. Actual and diagnostic scenarios share the original root occurrence, whole ledger, and successor, while the diagnostic scenario gains no actual writing authority. The same joint pulse also supplies same-source law revision and compatible future at this depth.

Deleting the relation seed, interest, controller action, common bearer, original payment purpose, actual root audit, finite constitution, or operational incidence contradicts the element of the deleted type supplied by the positive object. Setting the phase to zero contradicts (E.5.1). Reporting the absent target code as actionable contradicts the original absence proof. An alternative target role contradicts the role uniquely computed by the relation. Although health and platform relations have their own genuine sources, they cannot replace this payment's tenure purpose: every lawful compiler of the original payment purpose preserves its exact purpose relation.

A separate coarse-readout counterexample compares tenure and health subjects. Their protected-interest readouts agree, but their relation phases differ. A function recovering both phases from that readout would equate the original phases. The authority-reconstruction counterexamples for price, action, and administrative readouts are inherited from the original pairs in the same lifecycle. Full capture retains the nominal transfer while deleting actual exit renewal, alongside these provenance conditions.

E.5.2 The actual sequential exit from a mandate

The original mandate standing has restored power, renewal phase, and held role. Its generated next visit is the source visit v_d of the sequential exit action, and

$$\text{sourceEvent}(v_d) \simeq \text{rootOccurrence}(P_{d+1}^{\text{constitution}}). \tag{E.5.4}$$

Here $P_{d+1}^{\text{constitution}}$ is the original payment embedded in the depth-$d+1$ constitution. The sequential action has its own target root and occurrence presentation ϕ_d, satisfying

$$\phi_d(\text{sourceEvent}(v_d).\text{occurrence}) = P^{\text{target}}.\text{rootOccurrence.occurrence}. \tag{E.5.5}$$

Equation (E.5.4) joins the source in its original dependent types; (E.5.5) lawfully transports it to the target. They do not identify bare occurrences from two roots.

The fixed action's actual receipt retains the prior standing trace in full and proves lost target power, exit phase, and exited target role. The original mandate slot is written to a terminal archive. The target also installs the original UBI payment restriction, with root answer-and-next factorization for both the responsibility and payment coordinates. The target controller, relation controller, and original obligation bearer position agree. The relation and controller action each have seed provenance in their actual traces. The critical-relation bearer is uniquely the original parent bearer and therefore equals the exit worker's bearer.

The action's initial complete responsibility update gives

$$\text{firstSuccessor} = \text{mandateExitTargetFirstSuccessor},$$

while the successor actually adopted is

$$\text{actualSuccessor} = \langle V_{\text{target}}, R_{\text{target}}, v_{\text{target}} \rangle. \tag{E.5.6}$$

The first is the row and visit authority after the first whole-ledger write; the second is the complete successor current generated by the target answer. They retain their different types.

Taking an operational incidence of the form (E.5.3) from the depth-$d+1$ finite constitution supplies continuation outside the role, payment, the original service slot and route, external responsibility, all services, and the future. The depth-$d+1$ joint pulse supplies its root-law transition, actual/diagnostic same-root relation, same-source law revision, and compatible future. The complete consumer uses the source join, target transport, and receipt component by component, connecting exit and the life after it through the same original action chain.

The three false target reports restored, renewed, and held contradict the original target values. Deleting the action, bearer lineage, target payment factorization, constitution material, or first write contradicts the corresponding generated object. Divergence of the macro successor from (E.5.6) contradicts its exact successor equation. The original sequential action's compilation audit is answered; labeling it as the old revision obstruction would identify different compilation constructors. Nor does the actual action obtain authority from a diagnostic scenario lacking writing authority.

The original consumers are sourceGeneratedProofCarryingTenureOrdinaryExitConstitutionAt and sourceGeneratedProofCarryingMandateOrdinaryExitConstitutionAt. They enter the M14 whole consumer together with their respective NoGoAt evidence. The whole consumer's direct entry quantifies o, a, b, h, d and obtains c_0 from the total object's registered cycle. It accepts no caller-supplied subject, bearer, payment, or constitution.

E.6 Cofinal joint pulses and actual open execution

This section proves Theorem 6.4 while preserving two levels. Every step of the original cycle jointly produces same-source law revision and a future at the payment target. The local branch for finite representation failure has its own actual revision and subsequent execution. Their assembly in the total object has explicit source interfaces; their runtime states are not merged into an untyped "next step."

E.6.1 Recursing from the actual successor

Fix o, a, b, c_0 as in E.5. Let $m(c)$ be the actual morphism carried by an input. Generate complete paid-renewal evidence from this same $m(c)$, then feed it to the one-step revision-recurrence producer, denoted $s(c)$. Its output input contains the new source span, new source authority, new morphism, and the morphism's complete independent processing evidence. Define

$$c(0) = c_0, \qquad c(d+1) = s(c(d)).\text{nextCycle}, \qquad s_d = s(c(d)). \tag{E.6.1}$$

The definition immediately gives

$$m(c(d+1)) = s_d.\text{nextMorphism}.$$

The next step's paid certificate is regenerated from this new morphism. It is dependently equal to the original renewal producer applied to s_d.nextMorphism.

A one-step splice uses two original replays. At boundary $b+d$, the current right replay has index $b+d+2$, and the paid target has index $b+d+3$. The left and right replays at the next boundary $b+d+1$ have exactly these two indices. The next novelty's left replay equals the current right replay, with payload equal to the actual replay in the current span; the next span consumes the novelty's right replay. The current paid target's root occurrence equals the next source submission's exact occurrence, and its actual payment occurrence equals that used by the next span. The next source's own authority and morphism consequently enter same-source law revision, with actual disposition equal to the installed submission disposition. The complete consumer retains its exact failure, extension, new row, first write, runtime consumption, and macro successor.

The cycle replay is read from the span physically carried by $c(d)$. Unpacking that span and its source morphism identifies both index and payload with the original paid replay at $b+d+2$. Thus

$$i_d = b + d + 2, \qquad i_{d+1} = i_d + 1.$$

The index equation does not license a fresh selection of a canonical input in the recursion. The next input in (E.6.1) is always the preceding step's output.

E.6.2 Lifecycle overlap, root law, and the payment-target future

The morphism's paid renewal at this step and the first stage of the E.3 lifecycle at i_d are one producer. Hence

$$\mathrm{stage}_1(d) = \mathrm{stage}_0(d+1). \tag{E.6.2}$$

The target root occurrence, target payment occurrence, target authority, and target actual service occurrence respectively equal the next step's corresponding source objects. Equality of the actual stages accompanies equality of the absolute indices; occurrence identity is not inferred merely from matching numbers.

Let g_d be the occurrence actually handled by this step's complete root law. Its lifecycle parameter is i_d, while its actual visit is at $i_d + 2$ on the original reachable chain. It is precisely the successor root occurrence of continuation outside the role in E.3, with exactly the same whole responsibility write-back. The three edges are

$$i_d \longrightarrow i_d + 1, \qquad i_d + 1 \longrightarrow i_d + 2, \qquad i_d + 2 \longrightarrow i_d + 3. \tag{E.6.3}$$

The third edge uses g_d's own payment and targets the next step's actual root-law occurrence. This root law's macro successor is exactly the next root-law current. The same root-law step retains both endpoints' actual/diagnostic exhaustion, complete projection coverage, whole-ledger write, finite constitution, and lawful restriction of every original payment. Actual and diagnostic scenarios read the same root, ledger, and successor, but only the actual scenario has writing authority.

Write $\Pi_{h,d}$ for the corresponding proposition ExactCanonicalSocietyRenewalPulseAt. Its full fields are:

- This step's actual revision recurrence and its independent consumer.
- The source and target root laws, and the exact restriction of every original payment.
- The actual next input in (E.6.1).
- The original macro-future-birth audit at i_{d+1}.
- The equality $i_d + 1 = i_{d+1}$.
- Equality of this paid target's future with the original future producer at i_d+1, and dependent equality with the producer at i_{d+1} used by the audit.

For every h, d, generate these fields from the actual recurrence, original root-law step, and its target payment. Thus $\Pi_{h,d}$ holds. Compatible future birth is a projection of this joint object: apply its restriction of every original payment to this step's actual source payment, then combine the last three fields. This yields the rich future, every service, external responsibility, and the next constitution at the same payment target.

Let π_d, π_d^+ be the occurrence identities of the current and next revision payments in their common dependent sum, and let B_d be the target future generated by this payment. The exact join is

$$\pi_d^+ \simeq \mathrm{paymentOccurrence}(B_d), \qquad \pi_d \neq \pi_d^+. \tag{E.6.4}$$

For the first statement, expand the next-payment readout and use equality of morphism paid renewal with lifecycle stage zero; the result is the same target actual exit. Index locking of

payment occurrences proves the second. The joint pulse therefore belongs to one complete step while retaining distinct current and target identities.

E.6.3 Cofinality, arbitrary-depth separation, and no unrevisable terminal constitution

Given any n, h, take $m = n + 1$. The preceding section constructs $\Pi_{h,m}$, and $n < m$. This proves directly

$$\forall n, h\ \exists m > n\ \Pi_{h,m}. \tag{E.6.5}$$

Revision and compatible future are projected at the same m from the same $\Pi_{h,m}$. Separate existential proofs selecting different later positions would not establish this statement. The observer parameter h enters the original audit restriction; it neither bounds d nor changes the recursive next input. The consumer at each requested point also generates the finite root-law proof through that point and its next edge.

Take arbitrary $d \neq e$. Equality of actual paths projects to $b + d + 2 = b + e + 2$, which is impossible. Equality of actual-service authorities becomes the same impossible position equation by the original service-occurrence injection. Equality of payment occurrences is equally impossible by index locking. Meanwhile, every replay's normal form under the original singleton-pulse observer is 1. Thus

$$\mathrm{path}_d \neq \mathrm{path}_e, \qquad F(d) = F(e), \qquad \kappa_d \neq \kappa_e, \qquad \pi_d \neq \pi_e. \tag{E.6.6}$$

This is a direct proof for arbitrary pairs of depths, not a consequence of chaining adjacent inequalities. A left inverse recovering occurrences or a factorization recovering authority would contradict this pair.

For a given root-law step S_d, FinalUnrevisableRuntimeConstitutionAt has the following exact definition: it retains the current finite constitution while claiming absence of the generated target root law and its dependent faces, the target finite constitution, and target lifecycle no-go evidence. Its type is empty because S_d.targetLaw already inhabits the supposedly deleted target-root-law type. The target constitution or target lifecycle evidence independently supplies the same contradiction. This rejects deletion of an already generated runtime successor; completeness of the root-law theorem at the metalevel retains its own proof.

If every joint pulse beyond a finite bound were absent, substituting the $m, \Pi_{h,m}$ supplied by (E.6.5) would give a contradiction. There is therefore no cofinal gap in this exact sense.

E.6.4 The original replay interface of full platform admission

The original full platform admission contains a fulfilled paid authority. Its exact root occurrence equals the payment root occurrence of the actual exit carried by the original rightReplayAt. The left replay of the cycle's first next pair is this same rightReplayAt. These two existing interfaces connect full admission to the original paid recursion.

For any finite root-law-prefix parameter f and observer parameter h, take the cofinal consumer's $m > f$, joint pulse, and point material. The material contains the finite root-law audit, distinct actual paths, equal normal form with distinct actual-service authority, distinct payment occurrences, the target-future payment equation (E.6.4), and the empty fibres for deleting target

future, channels, or external responsibility. Equation (E.6.1) additionally gives

$$\text{generatedFinalNextCycle}(m) = c(m+1).$$

These fields form PlatformGateFiniteLawDiagonalPointAt. The construction reads only from full admission and the cofinal object. It neither identifies root occurrences in different ambient contexts by name nor reconstructs payment authority from a platform readout.

E.6.5 A substantive two-slot failure and local same-source revision

For every finite representation boundary r, the original construction supplies two actual replays x_0, x_1 strictly beyond it. Their installed visible readouts agree, while their actual positions satisfy $u_0 < u_1$. The original expression programs are functions $f : \text{Visible}_r \to \mathbb{N}$ with no future parameter, and their fixed semantics is

$$[\![f]\!] = (f(F_r(x_0)), f(F_r(x_1))).$$

Every program's denotation is therefore diagonal, whereas the actual requirement (u_0, u_1) is off the diagonal. Old responsibility content is retained in the left summand, and the new pair requirement lies in the right; the left summand cannot denote right-summand content either. This proves exact inexpressibility.

The original theory has two law slots. Joint realization requires one f to satisfy $f(F_r(x_j)) = u_j$ for $j = 0, 1$. Equal visible values force $u_0 = u_1$, so the joint-realization type is empty. Deleting slot 0 leaves the constant function u_1 as a realization; deleting slot 1 leaves the constant function u_0. The program language is nonempty, and every one-law remainder has an explicit realization. The failure has therefore not been defined through an empty language or another failure predicate.

The actual sequential action first preserves the original three open responsibilities. The finite compiler's row enumeration and inverse index cover every row; source and target rows are carried individually, and both origin and destination functions return the original row. Its complete write moves the ingress state to the active state. That active visit installs intent, inquiry, and failure coordinates together. The existing target-obstruction row supplies bearer authority. Its original responsibility content differs from the new obstruction content, and the original intent material supplies the semantic change. The ordinary nonterminal successor connects this failure to responsibility continuity at the same visit.

Applying the original minimal-extension construction to this actual failure produces new responsibility content, incidence, and row, all with no old preimage. The extension preserves the old rows' provenance, content, and budget conditions. It yields a typed revision grounded field by field in this same extension, a first whole-ledger write, and a unique first-write successor. At this visit, the installed inquiry program actually compiles to the revision branch carrying this failure and this core.

The proof-carrying legislation coordinate and runtime coordinate agree in their original dependent types on root, visit, responsibility row, and occurrence. The actual runtime-selected revision branch therefore consumes this exact core and yields the local reflexive same-source revision. Its failure, minimal extension, and first write are the producers just described, and old

and new rows share the generated macro successor. This is the local branch's own revision, beyond presenting its failure to another already existing revision site.

E.6.6 Every finite continuation of the same local runtime

Visits of the revised root are generated recursively from its initial visit and original actual successor. Its first-write successor reaches visit 1; the new row's answer then generates visit 2. States of the local process are either a source state or a revised state carrying a visit index:

$$\text{source} \longmapsto \text{revised}(2), \qquad \text{revised}(k) \longmapsto \text{revised}(k+1). \tag{E.6.7}$$

Each state's actual query type is a singleton. The unique activation law therefore determines the actual inquiry. The source state consumes the revision core generated above, while a revised state consumes the original answer and independent processing operation installed at its visit.

Induction gives

$$\text{state}(k+1) = \text{active}\big(\text{revisedVisit}(k+2)\big) \quad (k \in \mathbb{N}). \tag{E.6.8}$$

The base case is the actual revision's target visit 2. In the induction step, the preceding state's actual query and original generated successor increment the visit once more. Hence

$$\text{tick}(k+1).\text{next} = \text{answerAndNext}(\text{revisedVisit}(k+2)).$$

Every positive finite depth has a nonempty query type and a next step.

To distinguish these actual states, read the pair “root projection count, visit-history depth” from the erased authoritative state. The source root has 5 projections, and the revised root 7; revised visit j has history depth j. This separates source and revised states. Equality of two revised states forces equality of their depths. The map of positive depths to states is consequently injective: for all $s, t \in \mathbb{N}$ with $t > 0$, the states at $s+1$ and $s+t+1$ differ. Although the revised visits repeat the surface current label true, their actual histories do not return to an old visit.

The existing external revision Q also has a stronger actual continuation into this local runtime. The root, target visit, target row, target authority, and exact occurrence generated by Q's first write are respectively dependently equal to the five source components of the local sequential action. This interface connects Q's original first write to the local intent action, then to the local actual revision and (E.6.8). The authoritative currents of the two revisions remain distinct: Q's current carrier is $\mathbb{N}$, whereas the local revised carrier is Bool. Equality of their complete authoritative packages would identify their current carriers and hence equate an infinite type with a two-element type, a contradiction. The argument also distinguishes Q's first-write target and macro successor from the current and successor at every finite local depth.

The cofinal main object, platform diagonal, and local pair-intent branch each have complete direct consumers. They enter M14 through the registered cofinal field, platform-diagonal export, and canonicalSociety_cofinalRepresentationBreaking_localPairIntentSelfU8OpenRuntime, respectively. The local runtime's initial revision and later answered steps, and the original cofinal

cycle's freshly generated revisions at each step, are two results with their respective provenance. Their total assembly preserves the actual interfaces and distinct types.

E.7 Rich birth, bearer compatibility, and futures across settlement

This section gives the complete construction for Theorem 6.5. Write v for an original root visit, s for a reopening-source index, and n for a subsequent index within a root. These parameters range over the existing occurrence families; the constructions below generate their objects pointwise. A responsibility budget is the natural-number parameter used by native row evolution. It is read separately from monetary amounts and numbers of futures.

E.7.1 Complete target provenance and genuine birth

In one responsibility network N, let L be a complete source responsibility set and e an open target row. Define

$$\mathrm{Origin}_L(e) = \sum_{a \in L} \mathrm{DebtLineage}_N(a, e).$$

Same-debt lineage preserves responsibility content and lineage. Freshness evidence $\mathrm{Fresh}_L(e)$, relative to L, requires the same-debt-lineage fibre from every $a \in L$ to be empty. Hence

$$\mathrm{Origin}_L(e) \neq \emptyset \quad \Longrightarrow \quad \mathrm{Fresh}_L(e) = \emptyset. \tag{E.7.1}$$

Lemma E.7.1 (Every ordinary target has an original source). For every complete ordinary row evolution $w : L \to L'$ and every $e \in L'$, the compiler-selected origin $\mathrm{orig}_w(e)$, together with its row evolution, gives an element of $\mathrm{Origin}_L(e)$, and

$$B(e) \leq B(\mathrm{orig}_w(e)).$$

Every row in the first target A_{13} of ordinary joint settlement satisfies this conclusion.

Proof. The origin function of a complete row evolution gives a pair (a, u), where u is the exact evolution from source row a to e. Carried, maintained, and transferred evolutions all preserve same-debt lineage and each has non-increasing budget. Take $(a, \mathrm{lineage}(u))$. If freshness evidence also existed, applying its old-origin exclusion to these materials would produce an inhabitant of the empty type, proving (E.7.1).

For the concrete A_{13} construction, let I be the two-way presentation of the two inherited ingress rows. For any target row e, choose $I(I^{-1}e)$ as origin and use $I(I^{-1}e) = e$ to obtain equality of same-debt content. Thus every row of this complete target has an old origin. ■

For capacity revision, the comparison domain is instead the **pre-revision target face** L_v^- embedded in the revised network, rather than the three-row face after the first revision. The three-row target L_v^+ has the presentation

$$e_v : \mathrm{Fin}(3) \longrightarrow L_v^+, \qquad e_v(0) = \delta_v, \quad e_v(1) = \omega_v, \quad e_v(2) = \rho_v.$$

Here δ_v is the inherited contribution row, ω_v the inherited obstruction row, and ρ_v the new residual generated by exact failure. The reverse index j_v inspects a responsibility and its openness witness:

the inherited contribution witness gives 0, the inherited obstruction witness gives 1, and the new-direction witness gives 2. It satisfies

$$e_v(j_v(a)) = a \qquad (a \in L_v^+). \tag{E.7.2}$$

Proposition E.7.2 (Provenance of every row in the three-row target). For every v, both δ_v and ω_v have old same-debt origins relative to L_v^-. The residual ρ_v is fresh relative to that face and is outside the image of the old-row embedding. Every actual target row obtains one of these three provenance conclusions through (E.7.2).

Proof. For the first two rows, take the embeddings of the contribution and obstruction rows after capacity debit. The lineage and responsibility-content equalities come from the original transport. Now inspect any old-face row a. If its responsibility belongs to the old component, an assumed DebtLineage(a, ρ_v) would equate old responsibility content with the new obstruction content, contradicting the no-old-preimage property of the new content on the minimal revision face. If its responsibility belongs to the new-direction component, that component's openness type on the old face is empty. These cases exhaust a, so the old same-debt-origin fibre of ρ_v is empty. Equality with an embedded old row would supply the old origin already carried by that embedding, contradicting freshness. Finally, apply j_v to any target row and substitute (E.7.2) to obtain the complete classification. ■

Rich birth $\mathcal{B}_v(\rho_v)$ retains the following dependent materials: this freshness and no-preimage evidence; the generated row of ρ_v in the first complete write; causal authority at the same initial occurrence; an obligation admitted natively from that row; the exact equality identifying the obligation with its native admission; bearer equality; and an independent inquiry-answer processing operation at the same occurrence and row. The construction reads these materials from the first write, selecting the processing projection for row 2 of the three-row inventory.

The bearer equality is

$$\mathrm{bearer}(\mathcal{B}_v(\rho_v)) = b_\star = \mathrm{bearer}(q_b), \tag{E.7.3}$$

where q_b is the parent responsibility retained by the original clearing material. The native admission's content, budget, and lineage equal those of ρ_v. No new actor registry is an input to this row.

E.7.2 One bearer's payment, service, and continuation outside the role

Local current $n+1$ of the reopening root corresponds to actual service depth

$$D(s,n) = D_{\mathrm{reopen}}(s) + (n+1).$$

Its compatible new future $\beta^{\mathrm{r}}_{v,s,n}$ retains $\mathcal{B}_v(\rho_v)$, the original reopening action across roots, this current's own complete payment $p^{\mathrm{r}}_{v,s,n}$, residual transport, later causal authority, an independent processing operation, and an installed bearer-exit face.

Proposition E.7.3 (Compatible futures at every positive index). For every v, s, n, $\beta^{\mathrm{r}}_{v,s,n}$ exists. Its source and target residual rows preserve responsibility content and lineage, and both budgets are 0. Its actual service and bearer-exit face use the same payment occurrence and actual

settlement. The counterparty of purchasing-power standing is the existing actor t_1, with

$$\mathrm{bearer}(t_1) = b_\star.$$

The target of the former employment relation has no work continuation. The independent relation supplies leave, which is nonidle, uncaptured, and distinct from work. Exit capacity is exactly 1, while the full-capture control has capacity exactly 0.

Proof. At positive indices, the native reopening compilation carries all three rows completely. Read the residual's row evolution at its corresponding index. It gives budget $0 \to 0$ and preservation of content and lineage. Causal authority, inquiry-answer compilation, and the processing projection of the same root all read this residual rather than choosing another responsibility. The native payment itself contains an actual service existence witness. The actual exit selected from it equals the exit retained by the compatible future, because at fixed depth the payment type has only its exact native exit materials.

The bearer face is read from the installed projection. Its purchasing-power counterparty is t_1, and the exchange-bearer map sends t_1 to the original parent bearer, giving the same bearer in (E.7.3). The original employment relation has no work witness in its target continuation type; the independent relation has a native leave witness. Distinct relation seeds generate these two relations, so they differ. The independent continuation is neither idle nor captured.

The actual payment receipt, native route receipt, and first actually served slot form a feasible edge for leave, giving capacity at least 1. Inspect the original future codes $\{\mathrm{idle}, \mathrm{work}, \mathrm{leave}\}$: idle cannot act; work has no such independent target continuation; leave is the sole survivor. Capacity is therefore at most 1. The full-capture control captures exactly leave, giving capacity 0. Both geometries continue to read the payment, crossing, service, and settlement already present at this depth. ■

The original reopening action has service depth $D_{\mathrm{reopen}}(s)$, whereas every positive-index future has depth $D(s,n) > D_{\mathrm{reopen}}(s)$. Actual service occurrences are injective in depth. Later futures therefore use later services; the original action receipt is not replayed as their occurrence.

E.7.3 Residual settlement, inherited-row settlement, and cofinal new futures

The future used at reopening current 3 is $\beta^{\mathrm{r}}_{v,s,2}$. Its runtime payment is literally equal to the source payment of the residual-settlement action:

$$p^{\mathrm{r}}_{v,s,2} = p^{\mathrm{residual}}_{v,s}.\mathrm{sourcePayment}. \tag{E.7.4}$$

The action's actual successor enters the three-row settlement root. After faithful termination, that root sends the two inherited rows to the survivor root, while ρ_v no longer appears in the target open fibre. The residual's settlement consists of a local terminal receipt and absence of successor same debt. Both inherited rows retain their original same-debt origins and continue bearing responsibility.

Joint settlement subsequently gives each inherited row its final terminal receipt and hands off to a post-settlement root with an empty local responsibility set. Write

$$c_n = c_0(s) + n, \qquad e_n = \mathrm{emit}(c_n), \qquad p_n = \mathrm{Payment}(e_n).$$

The write material of p_n comes from the actual service witness in e_n itself. Its target is obtained from the dependent successor generated by that write.

Proposition E.7.4 (Cofinal rich futures after settlement). For every v, s, n, a rich macro future $\beta^{\mathrm{m}}_{v,s,n}$ exists. It retains (E.7.4), the residual receipt, settlement receipts for both inherited rows, the present post-settlement payment, actual service, the original bearer's continuation outside the former role, and source-grounded external responsibility. Moreover,

$$\mathrm{nextPaidBy}(p_n) = \mathrm{emit}(c_n + 1). \tag{E.7.5}$$

There is such a future beyond every finite bound m.

Proof. Take $\beta^{\mathrm{r}}_{v,s,2}$ and use (E.7.4) to connect it to the residual-settlement action fixed by the source event. This future's residual is exactly the source residual of settlement, so the terminal receipt read here is its own. Next read the joint-settlement receipts of the two inherited rows separately. Their target local responsibility set is empty, and the native terminal handoff supplies the post-settlement root.

At e_n, native compilation is a complete write carrying that event's actual service witness. The complete successor reads the same write-back. By the compilation's definition, its target occurrence is $\mathrm{emit}(c_n + 1)$, proving (E.7.5). The bearer-exit face is read from this root's installation. Its payment and actual settlement still equal those of p_n's actual exit, and Proposition E.7.3's bearer geometry supplies the capacity conclusions. External responsibility is generated from the source-responsibility material of this actual service witness, as shown in E.10. Every field is thus obtained from the existing occurrences and receipts above, constructing $\beta^{\mathrm{m}}_{v,s,n}$.

Given m, choose $n = m + 1$; then $c_0(s) + n > m$. Rich futures are therefore cofinal on the actual macro path that has crossed settlement. The reopening microroot also generates arbitrary later indices pointwise; the present macro conclusion is borne by the e_n family after settlement. ■

E.7.4 Finite counts and failures of reconstruction

For a fixed starting index a and length h, the finite future list is

$$F(a, h) = [\,\beta_a, \beta_{a+1}, \ldots, \beta_{a+h-1}\,], \qquad \mathrm{pulse}(F(a, h)) = h.$$

The listed objects retain their own indices. Equality of two entries projects to $a + i = a + j$, hence $i = j$. The original finite range has no repetitions, so neither does $F(a, h)$, and its length is h.

Proposition E.7.5 (A count cannot carry occurrence authority). For the count observation on microscopic compatible futures, there is no occurrence-reconstruction left inverse, authority factorization, faithful authority reconstruction, or factorization of the generated successor. Counting rich futures across settlement likewise cannot recover actual service authority or the actual macro successor.

Proof. Two adjacent singleton futures both have count 1. Their actual service depths are D and $D+1$. Injectivity of the original service-occurrence map makes their occurrences and authorities distinct. A reconstruction left inverse applied to the equal observation values would equate the two rich objects; projecting actual service gives a contradiction. An authority factorization would likewise assign the same value to the two distinct authorities. Faithful reconstruction includes authority factorization and is therefore also impossible. The microscopic successors have adjacent current coordinates, so their successor authorities also differ.

Across settlement, select the source future in (E.7.4) and the first post-settlement future. Both have count 1, but their actual service occurrences differ. For their actual macro successors, put authoritative current points from the different networks in a common dependent carrier. The former enters the finite settlement root, whose current type is Bool; the latter enters the post-settlement continuation root, whose current type is $\mathbb{N}$. Equality of the complete current points would project to Bool $=\mathbb{N}$. This would send the three distinct natural numbers $0,1,2$ to a two-element type while preserving their distinction, a contradiction. Hence the cross-settlement successor also does not factor through the count. ■

The original statements are the direct consumers for target origin or birth, compatible futures, and macro futures. The source table for this expansion gives their complete identifiers, first commits, and the versions read for this manuscript.

E.8 Three termination loci in the full history

This section fixes the canonical original visit and source index 1, tracking the contribution responsibility δ after capacity debit. Write B for a row-evolution budget. Service-occurrence depth, local root current, and observer-runtime depth retain their separate types.

E.8.1 The strict-renewal fibre

At an exact current occurrence e, let δ_e be the current responsibility row fixed by its old origin. The strict-renewal fibre $\mathcal{R}_e(\delta)$ contains:

1. This occurrence's actual complete payment.
2. Same-debt provenance from the old contribution row to δ_e.
3. A same-debt target row and its row evolution u.
4. Equality between this whole-ledger disposition on δ_e and evolved(u).
5. A strict debit $B(\text{target}\, u) < B(\delta_e)$.

These conditions connect obtaining another service and paying for the strict continuation of this particular old responsibility at one occurrence.

Lemma E.8.1 (Zero-budget obstruction). If the current source row and every permitted same-debt target row have budget 0, then $\mathcal{R}_e(\delta) = \emptyset$.

Proof. Any element of this fibre would have $0 < 0$ as its final condition, contradicting the strict order on natural numbers. ■

More generally, every step of paid same-debt continuation strictly decreases the existing natural-number budget. Using this budget as rank proves the relation well founded, without taking a completed history as an input.

E.8.2 From the last strict debit to the first empty fibre

The capacity-debit payment at depth 2 consumes the preceding contribution-merge payment. Its source occurrence, action receipt, and first target write retain exact equalities. Both complete source and target inventories are covered by Fin(2):

Source row	Target row	Compiler-selected evolution
Contribution, budget 1	Contribution, budget 0	Strict maintenance $1 \to 0$
Obstruction, budget 0	Obstruction, budget 0	Non-increasing continuation $0 \to 0$

(E.8.1)

The reverse indices prove complete coverage. The action's first successor equals the generated target's first successor, and its actual following state is precisely depth 3.

At depth 3, same-source law revision uses this very capacity failure. The first-write occurrence and whole-ledger write-back equal the native payment occurrence and write-back at local current 0 of the capacity-revision root. The inherited contribution row has a pre-revision origin and is excluded from freshness by E.7.2. The new residual is a separate third row.

Proposition E.8.2 (The first empty fibre and its persistence). At this first revision write immediately following (E.8.1), the old contribution responsibility's strict-renewal fibre is empty. Its exact strict-renewal fibre is also empty at every capacity-revision index n, at the first reopening action and all positive-index reopening writes, and at the ingress into the three-row settlement root.

Proof. At every n, the complete Fin(3) write of the capacity-revision root sends the inherited contribution row to itself. Source and target budgets are both 0, and the selected disposition is exactly this carried evolution. Lemma E.8.1 applies.

The first reopening action transports that old row to the target contribution row, preserving content and lineage and retaining budget 0. Every subsequent positive-index complete write reads this row and disposes of it by the source-fixed carried evolution. The ingress transport likewise covers all three rows and sends the old contribution to a same-debt target with zero budget. Each step retains its original provenance and compiler-disposition equality, so another target with available budget cannot be substituted into this fibre. Reapply Lemma E.8.1 at each stage. ■

The "first" locus is fixed by the existing adjacent actual history. The original contribution at depth 2 has the strict debit (E.8.1); its inherited row in the actual successor first has the stated empty fibre at depth 3. The three coordinates of this one occurrence are

$$(\text{revision-root local current},\ \text{actual service depth},\ \text{observer-runtime depth}) = (0, 2, 3). \tag{E.8.2}$$

Each number comes from its own generating function.

E.8.3 Local termination with a remaining same-debt successor

Take the state type $\{\bot\} \sqcup \mathbb{N}$. State $\bot$ denotes the three-row faithful-terminal point at depth 9; state n denotes the n-th actual visit of the following two-row root. Native continuation is

$$\bot \longmapsto 0, \qquad n \longmapsto n + 1.$$

This representation is injective. The terminal root has current type Bool, while the inherited root has current type $\mathbb{N}$, so these cases cannot be equal. Equality of two inherited states projects to equality of their finite-history depths and hence of their indices.

For a fixed debt, a **final settlement receipt** combines a local terminal receipt with absence of same debt at the generated successor. At $\perp$, all three rows have local terminal receipts. Yet the following two-row root still contains the original contribution and obstruction, preserving their responsibility content and lineage. The new residual alone has no same-debt target there.

Proposition E.8.3 (Residual settlement while inherited debt remains unsettled). The residual ρ finally settles at depth 9. The original contribution debt δ has no final settlement receipt there and has a same-debt target at depth 10. Every subsequent two-row-root index has an actual service payment, while both the old contribution entity's strict-renewal fibre and its paid macro-continuation fibre are empty.

Proof. The three-row terminal compilation supplies the residual's local terminal receipt. Its responsibility belongs to the new direction, whereas the following two-row face opens only the two old components. No target open row can therefore have the same debt as the residual. Together these facts construct the residual's final settlement receipt.

The source-fixed handoff directly constructs a successor for the contribution row. A final settlement receipt at $\perp$ would apply its no-successor-same-debt component to this actual target and yield a contradiction. At every successor index, the native event still contains an actual service payment existence witness. The contribution entity retains its same-debt origin, every permitted row has budget 0, and the local faithful-terminal type is empty. Lemma E.8.1 excludes strict renewal. The first strict payment of any paid macro continuation would lie in the same empty fibre, so the macro-continuation fibre is empty as well.

A total no-free-eternity debt-closure classifier would have to supply either termination or strict continuation for this zero-budget current debt without local termination. Its local type excludes the former; zero budget excludes the latter. Such a closure classifier therefore does not exist. Continued actual service and these negative same-debt conclusions retain their respective witnesses. ■

E.8.4 Final settlement of both inherited responsibilities

Along the later actual joint-settlement path, the target two-row root's native terminal compilation gives, row by row,

$$\mathrm{disp}_{A_{14}}(e_i) = \mathrm{terminal}(t_i), \qquad i \in \mathrm{Fin}(2). \tag{E.8.3}$$

The complete row presentation exhausts both rows, and the terminal handoff's target local responsibility set is empty.

Proposition E.8.4 (The locus of final settlement). Every $i \in \mathrm{Fin}(2)$ receives a final settlement receipt at A_{14}, after which execution enters native post-settlement payment. Local termination, final settlement, and arithmetic obstruction at depth 3 are borne by different actual evidence.

Proof. Take t_i from (E.8.3) as the local receipt. Any successor-same-debt witness would first have to supply an open target row. The target type is empty, so eliminating that row gives no successor same debt. The two parts form a final settlement receipt. Native terminal

handoff continues to generate the post-settlement root, making old-debt settlement compatible with subsequent service. The same-debt successor that still existed at depth 9 is now excluded, strictly separating these loci. The evidence at depth 3, by contrast, was the arithmetic impossibility $0 < 0$ and did not provide a local terminal receipt. ■

E.9 Finite publication, complete payment, and target recomputation

This section first constructs the result for general v, s, then reads current coordinate 12 in the consumer at $v = v_{\mathrm{can}}, s = 1$.

E.9.1 A two-section field and unique fold

The field is $\mathbb{N} \to \mathrm{Bool}$. Local sections restrict it pointwise to finite scopes. The overlap condition is the unit type, and gluing forms the pointwise function. Restriction and gluing commute by pointwise equality; uniqueness follows from function extensionality.

The local reflexive query reads the current Boolean value and writes true. Starting from false, its field satisfies

$$F(0) = \mathrm{false}, \qquad F(1) = \mathrm{true}.$$

The evaluation trace of one actual query is

$$\mathrm{false} \xrightarrow{\text{read/write}} \mathrm{true}. \tag{E.9.1}$$

The occurrence unfolding is a false source node with a true child. Define the locus algebra by

$$\alpha(t, C) = \begin{cases} \mathrm{some}(0), & t = \mathrm{true}, \\ \mathrm{none}, & t = \mathrm{false},\ C = [], \\ \mathrm{map}(+1, C_0), & t = \mathrm{false},\ C \neq []. \end{cases}$$

Folding the unfolding gives some(1).

Lemma E.9.1 (Unique readout with genuinely finite sections). Every evaluator commuting with the occurrence construction returns some(1) on this unfolding. No finite section recovers every ambient field.

Proof. A compatible evaluator returns 0 at the leaf according to α, then adds one at the source, giving 1. The general uniqueness of this fold follows as well by structural induction on the occurrence tree.

Fix a finite scope through k. Take the left field to be constantly false and the right field to be true only at $k+1$. Their restrictions to this scope agree, but they differ at $k+1$. A function recovering every ambient field would recover the same field from these equal restrictions, a contradiction. Finite publication therefore uses the exact local unfolding (E.9.1). ■

E.9.2 Root installation and the minimal locus

The publication demand is a zero-data carrier dependent on one existing survivor-source occurrence. It is installed before root emission. Its finite section and locus are read from the field above; its actual service witness is read from that source occurrence's own native payment material.

The joint-settlement entry has complete row presentation $\mathrm{Fin}(2) \simeq L$. At local section 0, every row has a continuation gate and an empty termination type. At section 1, every row has an exact terminal receipt, and both its continuation gate and the local successor type are empty.

Lemma E.9.2 (Minimality over the complete responsibility domain). Section 0 has no substantive-termination witness, whereas section 1 has one. Thus 1, supplied by the unique fold, is also the minimal termination locus on the complete domain.

Proof. A substantive-termination witness at section 0 would contain an actual row and a termination for that row. The inverse of the complete row presentation gives its index $i \in \mathrm{Fin}(2)$. Transporting termination to row i contradicts the emptiness of its termination type. At section 1, row 0 and its native terminal receipt supply a witness; row 1 terminates as well. The two-point scope has no other earlier candidate. ■

E.9.3 Publication payment does not receive a target as an extra input

The exact publication payment $\Pi_{v,s}$ is indexed by this same source occurrence and retains the demand, existing joint-settlement action payment, and actual service. It requires the demand's actual service witness to equal the action-source payment witness, the action payment to equal the existing generated object, and the original material and row-level provenance to agree exactly. For both source rows,

$$\mathrm{disp}_e(e_i) = \mathrm{evolved}(u_i), \qquad i \in \mathrm{Fin}(2). \tag{E.9.2}$$

The action's first successor comes from its own receipt. The macro successor is defined as the authoritative current point generated by the action output; a separate target is not supplied to the constructor.

Proposition E.9.3 (A unique actual target and a non-fixed response). The type $\Pi_{v,s}$ is inhabited. Its actual successor is the joint-terminal point A_{14}, where the recomputed relative locus is 0. Every publication payment satisfying the exact equalities generates the same target. The response type requiring the published readout to equal the target recomputation is empty; the actual non-fixed response type is inhabited.

Proof. Use the native source occurrence's service witness as source payment, the existing actual action and its receipt as action payment, and compiler identities for (E.9.2). These materials construct $\Pi_{v,s}$. Its actual successor is the first successor already generated by that action's target, namely A_{14}.

Both target rows terminate by (E.8.3); the recomputation algebra returns 0 at a terminal point. Any other payment satisfying the same exact action-payment equality reads this same successor after substitution, so its target agrees. This is independence of the target from exact payment evidence; it does not identify occurrences from arbitrary different sources.

A fixed point would require $1 = 0$, giving an empty fibre by natural-number elimination. The actual response directly retains the present payment, readout 1, generated target, and target recomputation 0. The construction above supplies its inhabitant. ■

In the canonical consumer, the source coordinate is exactly 12. The search state there compiles the inquiry into a direct answer. The joint-settlement actual-action state at the **same root and visit** compiles the input into the existing actual action. These compilation results have different constructors. The payment chain above explicitly uses the actual action's receipt.

E.9.4 The payment seam after termination

The actual terminal receipt consumes the action payment of $\Pi_{v,s}$, then gives

$$\mathrm{next}(A_{14}) = A_{15}.$$

The first post-settlement payment comes from the native occurrence at A_{15}. After consuming this complete write-back, its generated successor satisfies

$$\mathrm{nextPaidBy}(p_{15}) \;\; \mathrm{HEq} \;\; \mathrm{occurrence}(A_{16}). \tag{E.9.3}$$

Heterogeneous equality retains legitimate transport between the dependent indices. To prove it, substitute the terminal handoff into the first post-settlement visit, then substitute the next coordinate generated by native compilation into the canonical-current construction. Both sides become the same generated occurrence. Publication, action payment, terminal receipt, and first subsequent payment are separately generated objects connected exactly by this seam.

The relative readout changes from 1 to 0 because execution has reached the terminal target of the source point. This result contains no death time from an unpublished control. Its strength is that finite publication enters complete payment, changes the target state, and natively generates subsequent payment.

E.10 Risk splitting, the complete action path, and external responsibility

E.10.1 Source-fixed loss material

The original material contains an existing local clearing failure, the exact credit-support activation seed it generates, the purchasing-power standing generated from that seed, the original parent bearer, and the bearers of the two original exchange legs a, b. Those legs come from the existing exchange seed. Their bearers project to distinct target identifiers t_0, t_1, so they differ.

Periodic depth d selects only the original root visit v_d. This visit's credit material, actual root occurrence, original row, and exact failure for same-source law revision agree. The credit root's budget $2 \to 2$ is retained separately from strict budget decrease in the subsequent split branches.

E.10.2 Three targets of the first complete write

Write q for the parent row, ω for the obstruction, q_a, q_b for the children, and ω' for target obstruction. The children share the parent's world-responsibility identifier and content, but their openness witnesses contain the original exchange leg a or b, respectively. Observation of that leg therefore distinguishes the children.

The complete compilation table is

Write-row index	Source	Target	Evolution	
0	q	q_a	maintained,	$1 < 2$
1	q	q_b	maintained,	$1 < 2$
2	ω	ω'	transferred,	$0 \leq 0$

(E.10.1)

The source responsibility set has only two rows, q, ω. The write table repeats the parent origin to generate two child targets. The source reverse indices are $0, 2$, and the target reverse indices are $0, 1, 2$. Analysis of child openness witnesses gives the original leg a or b; the admission proposition fails for every other leg. New-direction responsibility is exactly the obstruction row. This proves complete source and target coverage.

Proposition E.10.1 (Exact splitting and two processing operations). At every original root visit, (E.10.1) generates a genuine split receipt. Its children are distinct and have distinct bearers while sharing the parent origin and same-debt content. Each child has an independent processing operation indexed by that receipt. This first write has no authority to merge two different source rows.

Proof. The origin function satisfies

$$\operatorname{orig}(q_a) = q = \operatorname{orig}(q_b), \qquad \operatorname{orig}(\omega') = \omega \neq q.$$

Take the child-obligation list $[Q_a, Q_b]$. Its content list consists of the parent's content repeated twice, supplying split coverage. For any target with the same origin as q_a, use the complete three-row classification: both children are in the list; the obstruction target is excluded because its origin differs. Apply $1 < 2$ to each child for strict debit. Conversely, any child obligation in the list is recovered at finite index 0 or 1 as an actual target. Thus there is no phantom child obligation.

For each actual child member, the receipt supplies original-event ownership of the responsibility, descendant incidence, source-anchor transport, same-debt lineage, and preservation of local discharge conditions. The independent processing operation reads these materials from this receipt and membership. The exchange-leg observations are a and b, making the children distinct; their bearers are read from the original material and differ as well. On (q_a, q_b), the first compilation's classification is exactly this split receipt. Equal-row pairs are classified by identity, and an assumed common origin for obstruction and child is excluded by $\omega \neq q$.

Finally, the selected targets of the source rows are q_a and ω', whose responsibility components differ. The selected-target map is therefore injective on the complete two-row source set: rows of the same kind agree, and rows of different kinds have unequal targets. Merging two different source rows is thus impossible. The budget conclusion is structural descent along each child branch; (E.10.1) carries no additional assertion of monetary conservation. ■

The construction is generated separately from v_d for each d, and the original credit-support occurrence varies injectively with depth. The depth-zero periodic certificate agrees with the existing run's first-revision certificate, whose actual following state is the contribution-merge action. This uses the already proved depth-zero installation seam; the universal periodic construction itself retains the source and action output at each visit.

E.10.3 Seams in the actual economic chain

Along the existing canonical execution, the following objects each retain their own complete write-back and actual successor:

$$\begin{gathered}\text{Credit support and risk split} \longrightarrow \text{contribution merge} \longrightarrow \text{capacity debit } 1 \to 0 \\ \longrightarrow \text{same-source revision at capacity failure} \longrightarrow \text{residual processing and reopening} \longrightarrow \text{residual settlement} \\ \longrightarrow \text{two inherited rows} \longrightarrow \text{actual joint settlement} \longrightarrow \text{post-settlement payment.}\end{gathered} \tag{E.10.2}$$

The arrows denote equalities of actual generated successors or legitimate transports across roots. E.8 gives the concrete runtime depths from capacity debit through inheritance; E.9 gives the seams from joint-settlement entry to later payment. Actual joint settlement and search inquiry at the same root and visit retain different compilations. At that point, (E.10.2) reads the actual-action branch. Its payment comes from the survivor-source event itself. The same-depth original-root service occurrence separately retained by the material is used through the proved provenance relation, preserving the distinction between these root events.

E.10.4 External provenance in an empty local ledger

Fix any v, s, n. The canonical post-settlement current is $c_n = c_0(s) + n$, with root event e_n. The local support equality gives

$$L_{\text{local}}(e_n) = \emptyset.$$

The same event nonetheless contains native actual service existence evidence. From it select

$$x_n \in \text{PeriodicActualServiceExit}(c_n).$$

This choice is restricted to the event's own witness. The write of actual payment p_n equals that witness, and its actual exit is exactly the selected element of the write's witness.

Source-grounded external liability $\Lambda(e_n, x_n)$ contains the following provenance relations:

- The installation of x_n's payment is exactly the original periodic installation at this depth, and its payment-root occurrence is the original root event at the corresponding temporal visit.
- The original activation seed occurs in the original actual trace.
- The original source owns the exact residual obligation's responsibility incidence; the bearer is read from that obligation.
- The original installation supplies an independent root responsibility processing operation, and its settlement is exactly x_n's actual service settlement.
- Actionable continuation is exactly x_n's continuation, and the root answer and successor satisfy the original installation factorization.

- The post-settlement root event and original-root payment event retain their respective types and their legitimate provenance relation across roots.

Proposition E.10.2 (External responsibility at every post-settlement current). For every v, s, n,

$$\text{Nonempty}\,\Lambda(e_n, x_n) \quad \text{and} \quad L_{\text{local}}(e_n) = \emptyset. \tag{E.10.3}$$

An empty local ledger therefore does not exclude this source occurrence's external payment responsibility.

Proof. The event's support equality sends any local open row into the original empty responsibility type and eliminates it. Meanwhile, x_n's native payment record already contains exact installation and root-occurrence equalities. The original event trace supplies the activation seed; the original responsibility incidence supplies source ownership of the residual obligation, which retains its own bearer. Read the original installation's responsibility processing operation, actual service settlement, actionable continuation, and answer-successor factorization together to construct $\Lambda(e_n, x_n)$. All these materials come from the actual service evidence already contained in the current event. No arbitrary new obligation is added to the empty local ledger.

If the sum of the two root-event types is used as a common carrier, the post-settlement event lies in the left summand and the original-root payment event in the right, so they differ. The existing provenance relation carries responsibility between them. Applying an alleged implication from local emptiness to absence of external liability to the just-constructed $\Lambda(e_n, x_n)$ yields a contradiction. ■

E.10.5 Index locking and the actual next payment

Proposition E.10.3 (Separate injectivity of root, payment, and service). For fixed v, s, each of the following canonical occurrence families is injective in n:

$$n \mapsto e_n, \qquad n \mapsto \hat{p}_n, \qquad n \mapsto \text{serviceOccurrence}(x_n).$$

More strongly, take any witnesses x, y in the actual service types at indices n, m, respectively. If their payments are equal in the common carrier retaining each dependent temporal visit, then $n = m$.

Proof. Equality of root events projects to $c_0(s) + n = c_0(s) + m$, and cancellation gives $n = m$. Equality of canonical payment occurrences first projects to the original temporal visits, then to causal depth, yielding the same equation. For service occurrences, injectivity of the original periodic actual-service occurrence family gives the same depth equality.

For arbitrary witnesses x, y, package each payment with its temporal visit in a dependent pair. Equality of these pairs first forces equality of the visits. Their depths are still $c_0(s) + n$ and $c_0(s) + m$, so again $n = m$. This argument does not first replace arbitrary witnesses by canonical choices. ■

A complete post-settlement payment also retains the successor u_n generated by its event's compilation and $\Lambda(e_n, x_n)$. Its next occurrence is defined as the target of u_n, giving

$$\text{targetCurrent}(u_n) = c_n + 1, \qquad \text{nextPaidBy}(p_n) = \text{emit}(c_n + 1).$$

Complete local disposition holds on the empty responsibility domain; Λ positively retains external responsibility; the next payment depends on the next actual occurrence. Together these three parts establish the universal conclusion of Theorem 7.7.

E.11 Complete separation of income, material service, and job standing

Proposition 5.8 uses the actual separation construction of the fivefold constitution. Fix the old visit, source index, boundary, and original cycle entry C, and choose arbitrary finite recursion parameter f and observer parameter o. Same-point inhabitation supplies a later index $m > f$ and its constitutional point. The original full platform admission retains the actual replay joined to this cycle. Every object below is generated from these inputs.

Let U be the type of role-external exit in the positive geometry, and U_c the full-capture exit type retaining the original payment, purchasing power, employment loss, and interest axes. Let J_0 be job standing in the ended employment and J_1 standing for the same worker in the new-job relation already generated by the original program. Let M_1 be actual material service at a later same-lineage occurrence, and M_0 admission of a third material service while retaining both old paid slots. We prove

$$U \neq \emptyset, \quad J_1 \neq \emptyset, \quad M_1 \neq \emptyset, \qquad U_c = J_0 = M_0 = \emptyset.$$

Emptiness means that the respective type has no inhabitant; the equalities are used as propositional statements of emptiness.

Exit and jobs. Positive exit combines the original leave action, role-external continuation, actual payment, route, and service into a feasible edge. Full capture retains these nominal materials but captures the only actionable non-idle continuation, so U_c is empty. The ended employment has an empty target future-code set. Any J_0 would have to select an actionable work code from that set, a contradiction. The new-job relation comes from the original relation seed already produced by the leave-execution program. Its work code belongs to the target set $\{\text{idle}, \text{work}, \text{leave}\}$, its continuation witness is inhabited, and work differs from idle. The original role authority and this work continuation therefore form an actual J_1. Pairing it with positive exit or with emptiness under full capture gives the two coexistence results.

The old slots and the new service. The original payment has exactly 2 served slots: tenure and health occupy slots 0 and 1. Preserving both services requires admission of a third material service to supply

$$M_0 = \{j \in \mathrm{Fin}(2) \mid j \neq 0 \ \wedge \ j \neq 1\}.$$

Every $j < 2$ is 0 or 1, so this type is empty. Both old channels enter the revised channel family through the original embedding, retaining the actor, relation, original payment, actual settlement, route, feasible exit, and independent processing individually. Their actual successors also agree.

The new service comes from the next actual occurrence of the platform lineage. If the admission gate has depth q, the original three-cycle relation construction returns the same relation seed and bearer at $q + 3$, with a strictly different action occurrence. The seed admits a maintenance obligation: history length is 1 after admission and 2 after maintenance. The original service budget

is $2 - |\text{history}|$, so one maintenance event strictly debits $1 \to 0$ while retaining the same live obligation and bearer. A later native fulfilment event writes a fulfilled terminal archive into the same slot; its reopening-event type is empty. The whole action reads its own whole-ledger receipt, and its actual successor is precisely the root and visit of that action's target. Thus M_1 has an actual inhabitant, and its service bearer equals both the original bearer and the bearer of the admitted obligation. Emptiness of M_0 remains, as do the two old services.

All non-substitution results. Take the constructed $u : U$, $j : J_1$, and $x : M_1$. A function $U \to M_0$ or $J_1 \to M_0$, applied to u or j, would inhabit M_0. A function $M_1 \to U_c$ or $J_1 \to U_c$, applied to x or j, would inhabit U_c. All four function types are therefore empty. The same argument excludes $M_1 \to M_0$: the later service retains its occurrence and cannot collapse into the already full old slots. Since U and J_1 are both inhabited, new-job standing and emptiness of positive exit are not logical complements either.

For arbitrary f, o, the construction obtains a same-point witness and assembles positive exit, jobs, and platform materials into a complete carrier. Hence every choice of inputs has this joint object. The original declaration is `canonicalSocietyUBIUBSJobGuaranteeSeparationAt`, with direct consumer `canonicalSocietyFivefoldConstitution_directConsumer`. The theorem correspondence records full source identities and direct dependencies.

Appendix F Capability portfolios, actual repair, and continuing responsibility

F.1 Complete capability portfolios at positive capacity

Fix an arbitrary finite visit depth d. The original domain-authority family D has tenure and health constructors, whose actors, relations, and phases are generated from their respective relation seeds. Let Q_d be the original installed queue. The full source has prefix cutoff 3, actual capacity 2, and served positions $0, 1$. The original domain-to-queue-occurrence map selects the first and middle positions respectively.

Service incidence for a domain $a \in D$ comprises a served ordinal j, equality between that slot's queue unit and the unit originally generated for a, and a service receipt at that ordinal of the same actual debited run. It also retains equality of the original root payment with the actual root event at depth d. Enumerating $j < 2$ proves slot uniqueness: tenure must use 0, health 1. Interchanging them would identify the first and middle queue positions under projection, a contradiction; an ordinal above one violates $j < 2$.

Combine a domain's relational interest, generated phase, target role, service incidence, payment incidence, route, and uncaptured feasible edge into $\mathcal{A}_d(a)$. The complete portfolio is

$$\mathcal{P}_d = \prod_{a \in D} \mathcal{A}_d(a).$$

For each domain constructor, its original service ordinal, debit receipt, and leave action form the required component. The two service identities are dependent pairs $(j, \text{receipt}_j)$, distinct because their first projections are $0, 1$. Underlying repeated event values may still coincide; actual service-ordinal identity is retained.

The second source takes cutoff 1 in the same queue and executes only the first actual maintenance event. Its served list has only position 0; its original payment occurrence, root event, payment rule, and route rule are the same as the full source's. Tenure obtains an actual feasible edge at this slot. A health-service incidence would identify the sole slot's unit with the original middle-position unit, contradicting their distinct positions. The second source's complete portfolio is therefore empty.

Both geometries have only the leave action among surviving actionable codes, so maximum matching capacity is at most 1. Each has an actual feasible edge and hence positive capacity. Thus

$$K(x_d) = K(y_d) = 1, \qquad \mathcal{P}(x_d) \neq \emptyset, \quad \mathcal{P}(y_d) = \emptyset.$$

A left inverse reconstructing sources from capacity would give $x_d = g(K(x_d)) = g(K(y_d)) = y_d$, contradicting prefix cutoffs 3 and 1. If a predicate $q : \mathbb{N} \to \mathrm{Prop}$ classified portfolio inhabitation exactly on both sources, the full source would give $q(1)$, and the equivalence for the one-service source would then inhabit its portfolio, again a contradiction. This latter result excludes even scalar classification of completeness without full source reconstruction.

Four independent controls remove payment incidence, route incidence, non-capture, or actual service respectively. In the first three, a putative complete portfolio yields an object in the deleted type by projecting the tenure component. The fourth sets every service-receipt type to empty while retaining payment and routes. A complete settlement would yield an actual service at the first slot, a contradiction. Each control retains its other declared actual materials.

The portfolio also enters the actual post-repair viable branch. Its payment at depth $d + 1$ is dependently equal to the original post-repair actual-exit payment; their economic root event, fold image, and original landing agree individually. This connection makes the domain-specific portfolio available to the existing successor. Every construction is supplied for arbitrary d and retained by S_h at every foundation horizon h. The original direct consumer is `proofCarryingActionableFuturePortfolioDirectConsumer`, and the whole-program exports are `canonicalSociety_actionableFuturePortfolio` and its `sharedAxes` result.

F.2 Shortest actual recovery and arbitrary finite viable continuation

This section proves Theorem 5.9. Fix any foundation parameter h of the original canonical society. The objects below are generated from the existing sources at every original depth $d \in \mathbb{N}$; h bounds neither d nor the requested continuation length. Complete portfolios and actionable edges of the original essential domains are as defined in Appendix F.1.

F.2.1 One viability type and two failures

Let S_d^{full} and S_d^{one} be the original complete-service and stable one-service sources, and let κ_0, κ_1 be the original no-capture and full-capture compilers. The configuration type and three particular

configurations are

$$\begin{aligned} \mathcal{C} &= \sum_{q:\mathbb{N}} (\mathrm{PortfolioSource}_q \times \mathrm{CaptureCompiler}), \\ M_d &= (d, S_d^{\mathrm{one}}, \kappa_0), \qquad C_d = (d, S_d^{\mathrm{full}}, \kappa_1), \\ V_d &= (d+1, S_{d+1}^{\mathrm{full}}, \kappa_0). \end{aligned} \tag{F.2.1}$$

Thus the actual visit depth of V_d is $d+1$. Its subscript d identifies the initial recovery and does not relabel its target as the source visit.

For any (q, S, κ), an element of the original viability type has the following dependent data: a complete portfolio $P : \mathrm{CompletePortfolio}_q(S)$; proof that the leaving action ℓ is not captured by κ; an original accounted economic occurrence ω, its fold image I, and its landing L; and an original viable economic disposition u at (ω, I, L). These satisfy

$$u.\mathrm{depth} + 1 = q, \qquad P \simeq u.\mathrm{portfolio}. \tag{F.2.2}$$

Here $\simeq$ denotes heterogeneous equality, retaining the dependent source, visit, and portfolio indices. Viability enters the viable branch of the original economic disposition directly; a separately introduced Boolean label does not replace these data.

M_d is nonviable because the health component of its complete portfolio would have to inhabit the original empty health-service fibre. This source still has the original payment, route, uncaptured tenure action, and capacity 1. C_d is also nonviable: its leaving action is captured by κ_1, while viable data require that same action to be uncaptured. The complete services remain present in this case.

The positive target is generated from the canonical successor of the original repair. The complete portfolio at visit $d+1$, its proof of non-capture, the original economic occurrence and its fold image, and the viable disposition jointly supply (F.2.2). Hence

$$\mathrm{Viable}(M_d) = \emptyset, \qquad \mathrm{Viable}(C_d) = \emptyset, \qquad \mathrm{Viable}(V_d) \neq \emptyset. \tag{F.2.3}$$

The path-minimality argument below uses this one type.

F.2.2 Three actual edges and their authority

Write v_d for visit d of the original root and N_d for its actual successor. Service restoration comes from the health-service operation installed in the original service-restoration coordinate. Its actual data identify the missing source health unit, a required service cut of 2, the selected responsibility row, the original whole-ledger write-back, and its independent processing operation. At the next visit, the target health-service slot actually serves the unit required by that operation, and the complete portfolio is generated again at the target visit.

Capture repair comes from the original exit disposition. Its exact repair is capture removal, and its responsibility row remains the original UBI-selected row. The selected write commutes with the world whole ledger, and root answer and successor retain their factorization. The two operations belong to different original projections: the health-service-restoration projection and

the UBI projection. Their actual receipts satisfy

$$\begin{aligned} W_d^{\mathrm{service}} &= W_d^{\mathrm{capture}} = W_d^{\mathrm{root}}, \\ N_d^{\mathrm{service}} &= N_d^{\mathrm{capture}} = N_d = \mathrm{Next}_R(v_d), \\ N_d.\mathrm{visit} &\simeq v_{d+1}. \end{aligned} \tag{F.2.4}$$

The common row and successor are supplied by the original root compiler, while the distinct operational projections and their exact source indices are retained. The common target has the complete portfolio of S_{d+1}^{full}, an uncaptured original leaving action, capacity 1, and the original law surface.

An edge in the original actual action graph carries more than two configurations. It also carries a depth, the original adjacent viability boundary at that depth, an actual operation, and four kinds of exactness evidence: the boundary is the originally generated boundary; the operation is the corresponding original receipt; source and target are the configurations prescribed by that operation; and the operation's actual successor is the corresponding canonical successor. Equation (F.2.4) supplies the first two edges:

$$a_d : M_d \longrightarrow V_d, \qquad b_d : C_d \longrightarrow V_d. \tag{F.2.5}$$

The third edge comes from viable renewal. The complete portfolio of V_d and the service operation of the adjacent boundary at depth $d+1$ share the same root occurrence. The service and capture dispositions of that boundary retain the original selected row, the same whole ledger, and $\mathrm{Next}_R(v_{d+1})$; the new complete portfolio is at v_{d+2}. This gives

$$r_d : V_d \longrightarrow V_{d+1}. \tag{F.2.6}$$

Its source and target payment occurrences belong to their respective actual visits. If payment occurrences are placed in a common dependent-sum package K_q retaining the visit, original visit injectivity gives $K_{d+1} \neq K_{d+2}$. Equation (F.2.6) is joined through the common intermediate occurrence and exact successor; adjacent payments are not identified as one occurrence.

The three operation types form

$$(\mathcal{A}_d^{\mathrm{service}} + \mathcal{A}_d^{\mathrm{capture}}) + \mathcal{A}_d^{\mathrm{renewal}}. \tag{F.2.7}$$

The two repairs use different constructors of the inner sum, while renewal uses the other constructor of the outer sum, so the three are pairwise distinct. Exact source conditions also exclude false retyping: a service receipt cannot be an edge from C_d, and a capture receipt cannot be an edge from M_d. Neither death repair can begin at V_d at that same d; renewal cannot begin at M_d or C_d, nor can it remain at the same V_d. The first two exclusions follow from the distinction between complete and stable sources; the others follow by projecting configuration depth.

F.2.3 Minimality over all legal candidates

For a configuration x, the legal candidate type $\mathcal{Q}(x)$ carries a target y, a path $\gamma : x \rightsquigarrow y$ in the original actual action graph, and a viable object at y. Original public generation accepts only depth and finite length; the target, path, and viable data in a candidate are generated from these inputs. Minimality holds for every element of this original candidate type.

Suppose $\mathrm{Viable}(x)$ is empty and $q = (y, \gamma, u) \in \mathcal{Q}(x)$ has $|\gamma| = 0$. The zero-length theorem for the original path type gives $x = y$. Transporting u back to x along this equality yields a contradiction. Therefore

$$\mathrm{Viable}(x) = \emptyset \implies \forall q \in \mathcal{Q}(x), \quad 0 < |q.\gamma|. \tag{F.2.8}$$

Applying (F.2.3), every candidate from either failed source has length at least 1. On the other hand, a_d and b_d each yield a one-edge path whose target viability comes from (F.2.3), and a one-edge path has length exactly 1. Thus

$$\tau(M_d) = \tau(C_d) = 1, \qquad \forall q \in \mathcal{Q}(M_d),\ 1 \le |q.\gamma|, \qquad \forall q \in \mathcal{Q}(C_d),\ 1 \le |q.\gamma|. \tag{F.2.9}$$

Here τ is simply the path length of the original canonical candidate, not an optimum supplied as an input.

This readout loses repair authority. Let $\mathcal{A}_d = \mathcal{A}_d^{\mathrm{service}} + \mathcal{A}_d^{\mathrm{capture}}$, and write α_d for the original service-restoration operation and β_d for the original capture repair. The time readout $t_d : \mathcal{A}_d \to \mathbb{N}$ takes the path length of the canonical repair in the respective branch. Sum-constructor separation and (F.2.9) give

$$\alpha_d \neq \beta_d, \quad t_d(\alpha_d) = t_d(\beta_d) = 1, \qquad \nexists s : \mathbb{N} \to \mathcal{A}_d, \quad s \circ t_d = \mathrm{id}. \tag{F.2.10}$$

For the final claim, a left inverse would give $\alpha_d = s(t_d(\alpha_d)) = s(t_d(\beta_d)) = \beta_d$. Its codomain is the original complete repair-action type, retaining the service-operation or capture-disposition receipt, rather than a cause label assigned afterwards.

F.2.4 Requested lengths, actual continuation, and literal prefixes

Define recursively in the same actual action graph:

$$\rho_{d,0} = \mathrm{nil}_{V_d}, \qquad \rho_{d,n+1} = \rho_{d,n} \cdot r_{d+n}. \tag{F.2.11}$$

Equation (F.2.6) makes the source at each step coincide exactly with the existing endpoint, so $\rho_{d,n} : V_d \rightsquigarrow V_{d+n}$. By the recursive equation for path length, zero steps have length 0 and appending an edge increases length by 1. Induction gives $|\rho_{d,n}| = n$. The two repair paths are $a_d \cdot \rho_{d,n}$ and $b_d \cdot \rho_{d,n}$, each of length $n + 1$, with the originally generated $\mathrm{Viable}(V_{d+n})$ at the target. This also gives legal candidates for every requested length, still subject to the original minimum in (F.2.9).

Fix k and induct on $n \geq k$. At $n = k$, take the empty suffix. Given $\rho_{d,k} \cdot \sigma = \rho_{d,n}$, append r_{d+n} to σ. The recursive equation for path composition gives

$$\rho_{d,k} \cdot (\sigma \cdot r_{d+n}) = (\rho_{d,k} \cdot \sigma) \cdot r_{d+n} = \rho_{d,n+1}. \tag{F.2.12}$$

This is the exact suffix factorization of Theorem 5.9. The viable object at each cutoff is the original object generated at that cutoff's depth and has the type of the path's actual endpoint. The proof gives compatible finite paths for $\forall d\, \forall n\, \forall k \leq n$; it does not take a completed future as an input.

F.2.5 Domainwise structural rent capture and recovery

The original payment-purpose compiler generates the tenure relation, its phase, and its provider role from an actual relation seed. Its capture disposition consumes the open price domain of the same unsettled settlement, retaining the protected interest, residual bearer, source-owned responsibility, and activated independent processing operation. This purpose chain joins the payment root occurrence, actual service occurrence, and payment successor at visit d, yielding the original structural rent capture object.

For every existing essential domain u, the original uncaptured configuration has an actual actionable edge. If an edge existed in the fully captured domain, its action-survival field would show that ℓ could still be exercised as an independent exit. Full capture makes the entire actionable exit cone empty, so this field cannot exist. Hence

$$\forall u \in \mathcal{D}, \qquad \mathcal{E}^{0}_{d,u} \neq \emptyset, \qquad \mathcal{E}^{1}_{d,u} = \emptyset. \tag{F.2.13}$$

$\mathcal{D}$ is the originally generated essential-domain authority type. Emptiness of each fibre is stronger than emptiness of the complete portfolio alone. The original tenure and health crossings also remain distinct, so this result does not first identify two domains as the same edge.

The change preserves the original payment, purchasing-power standing, every route incidence, and actual services. Capture settlement remains exactly the original open settlement, and its bearer and independent processing operation are retained. Actual repair uses the capture-removal disposition of that same capture-responsibility object. Its selected write commutes with the original whole ledger and yields b_d through (F.2.4). This captured configuration therefore belongs to the viable region reached by the proved minimal recovery and every requested finite continuation.

Exact causes remain distinguishable. When routes are missing, the original domainwise route fibres are empty without a capture verdict. The stable one-service source retains capacity 1 but has an empty health-service fibre. A fully fulfilled occurrence of the same payment has no open price domain and therefore cannot generate this structural rent capture. These controls retain the actual premises used by the capture interpretation.

Finally, check that the original graph contains no unstated shock. Suppose there were an edge $V_d \to C_{d+1}$, and let j be its internal depth. For service restoration or capture repair, source-depth equality gives $d+1 = j$ and target-depth equality gives $d+1 = j+1$, a contradiction. For renewal, source-depth equality gives $d + 1 = j + 1$ and target-depth equality gives $d + 1 = j + 2$, again a contradiction. Thus

$$\mathrm{Hom}(V_d, C_{d+1}) = \emptyset. \tag{F.2.14}$$

Equation (F.2.13) compares uncaptured and captured configurations from the same source. It holds together with the actual recovery edge, but supplies no additional temporal edge into capture.

F.2.6 Direct consumption and the total object

The original minimality consumer reads the one-edge actual paths, non-interchangeability of sources, minimality over arbitrary candidates, and non-invertibility of the time readout. The finite-continuation consumer reads whole-ledger renewal, the three actual edges, original path lengths, every literal prefix, and preservation of the minimum. The capture-recovery consumer adds the original structural rent responsibility, domainwise emptiness, and the strict controls above. The three complete certificates enter the canonical society's minimal-recovery, finite-continuation, and capture-recovery fields, respectively.

The corresponding original declarations are:

- `maintenance_hitting_time_is_universally_minimal`
- `capture_hitting_time_is_universally_minimal`
- `viableContinuationPath_prefix_factorizes`
- `structural_capture_chokepoint_is_in_viability_basin`
- `current_quiver_has_no_viable_to_capture_shock`

The whole-program interfaces below quantify over every foundation parameter, original depth, and requested number of steps:

- `canonicalSociety_minimalEconomicViabilityHittingTime`
- `canonicalSociety_finiteViabilityContinuation`
- `canonicalSociety_captureChokepointBasinRecovery`

F.3 Full-domain observation, the authority kernel, and positive refinement (G25)

F.3.1 The original sources and all domains

Fix any depth $d \in \mathbb{N}$. Let P_d^{c} be the proposition giving the original tenure-capture source, and P_d^{r} the proposition giving the original route-interruption source. The former preserves the original relational interest, participant and role identities, and capture witness. The latter preserves the original relation identity, non-capture, actual payment and service, and the emptiness of every required route-receipt type. Both are propositions, and the original construction provides proofs p_d^{c} and p_d^{r}. The original failure-source type is

$$\Omega_d = P_d^{\mathrm{c}} \sqcup P_d^{\mathrm{r}}, \qquad c_d = \mathrm{inl}(p_d^{\mathrm{c}}), \quad r_d = \mathrm{inr}(p_d^{\mathrm{r}}). \tag{F.3.1}$$

The sum retains the source of failure; proof irrelevance governs the identity of the proof payload on each side. Thus every $x \in \Omega_d$ equals the displayed canonical element of its branch, and $c_d \neq r_d$.

Take the original type $\mathcal{D} = \mathrm{EssentialAccessRelationAuthority}$. It ranges over all essential-access relation authorities currently present in the original program; the original relation compiler generates the slot and crossing associated with each domain. Write the feasible edge types in domain

D under full capture and a missing route as $\mathrm{CapturedFeasible}_d(D)$ and $\mathrm{RouteMissingFeasible}_d(D)$, respectively. Define

$$\begin{aligned} F_d(c_d, D) &= \mathrm{CapturedFeasible}_d(D), \\ F_d(r_d, D) &= \mathrm{RouteMissingFeasible}_d(D), \\ \rho_d(x) &= (K_d(x),\ \{D \in \mathcal{D} \mid \mathrm{Nonempty}\, F_d(x, D)\}). \end{aligned} \tag{F.3.2}$$

Sum elimination extends this definition to arbitrary source proofs; no domain list is supplied by a caller.

Lemma F.3.1. For every d and every $D \in \mathcal{D}$, both $F_d(c_d, D)$ and $F_d(r_d, D)$ are empty, and

$$\rho_d(c_d) = \rho_d(r_d) = (0, \emptyset). \tag{F.3.3}$$

Proof. The capture branch uses the original full-capture result: every feasible edge must satisfy non-capture, whereas this continuation relation is captured, so no such edge exists. If the route branch contained an edge e, one of its fields would supply a route receipt for the actual crossing of this domain. The original source's route-emptiness proof at that crossing immediately rules out the receipt. Domain reachability is therefore false pointwise on both sides, so both reachability sets are empty. The original capacity computations give zero on each side, proving the claim. ■

This step uses universal emptiness over all original domains. Adding their reachability coordinates to the observation preserves the same kernel.

F.3.2 The exact kernel, whole-source symmetry, and arbitrary outputs

Write the original repair type as $\mathcal{R} = \{\mathrm{removeCapture}, \mathrm{restoreRoute}\}$, the originally installed repair compiler as R_d, and the source's original failure kind as ϕ_d. Canonical repair is defined by

$$\kappa_d(x) = R_d(\phi_d(x)), \qquad \kappa_d(c_d) = \mathrm{removeCapture}, \quad \kappa_d(r_d) = \mathrm{restoreRoute}. \tag{F.3.4}$$

Hence (c_d, r_d) is an exact authority kernel for the observer (ρ_d, κ_d): its sources differ, its visible values agree, and its authority values differ. The dependent boundary of the original generated obstruction direction is exactly this source pair; no substitute kernel is chosen.

Proposition F.3.2. None of the following reconstructions exists:

1. An authority factorization A satisfying $A(\rho_d(x)) = \kappa_d(x)$ for every x.
2. A source left inverse L satisfying $L(\rho_d(x)) = x$ for every x.
3. A faithful authority reconstruction carrying a visible right section, authority readout, and source recovery together.

Moreover, for every output universe u, every type $Y : \mathrm{Type}_u$, every $q : \Omega_d \to Y$, and every factorization $q = f \circ \rho_d$, there is no $g : Y \to \mathcal{R}$ such that $g(q(x)) = \kappa_d(x)$ for every x.

Proof. If the first item existed, (F.3.3) would give $\kappa_d(c_d) = A(\rho_d(c_d)) = A(\rho_d(r_d)) = \kappa_d(r_d)$, contradicting (F.3.4). The same equality in the second item would give $c_d = r_d$, contradicting disjointness of the sum constructors. The third item includes the data in the first and is therefore also impossible.

For an arbitrary output type, the existence of g would make $A = g \circ f$ a factorization of the first kind. This contradiction uses only function composition; it imposes no finiteness, countability, or particular numerical structure on Y. ■

The naturality result uses a symmetry of the entire original source type. Define $\sigma_d : \Omega_d \to \Omega_d$ by sending every capture source to r_d and every route source to c_d. Define $\tau : \mathcal{R} \to \mathcal{R}$ by exchanging the two repairs. Case analysis on the sum, with proof irrelevance within each branch, gives

$$\sigma_d^2 = \mathrm{id}, \qquad \rho_d \sigma_d = \rho_d, \qquad \kappa_d \sigma_d = \tau \kappa_d, \qquad \forall a \in \mathcal{R},\ \tau(a) \neq a. \tag{F.3.5}$$

Thus σ_d, identity on the visible type, and τ form an admissible automorphism of the original observer. At the reachable anchor $y = \rho_d(c_d)$, it fixes visibility and changes the authority of every source in the fibre.

Proposition F.3.3. There is no visible right section natural under all original admissible automorphisms, and no natural authority reconstruction.

Proof. Suppose s were such a right section and put $x = s(y)$. The right-section condition gives $\rho_d(x) = y$. Naturality under the admissible automorphism (F.3.5) requires $\sigma_d(s(y)) = s(\mathrm{id}(y)) = s(y)$, hence $\sigma_d(x) = x$. Applying κ_d yields $\tau(\kappa_d(x)) = \kappa_d(x)$, contradicting the absence of fixed points of τ. A natural authority reconstruction includes such a natural right section and is therefore excluded as well. This conclusion concerns the natural-section contract; an ordinary right section only chooses a preimage of each visible value, and its existence is governed by surjectivity of the visible map. ■

F.3.3 The positive role of the original failure kind

Refinement reads the existing failure kind from the original source. That type already belongs to the input language of the original pre-emitter repair compiler. Set

$$\hat{\rho}_d(x) = (\rho_d(x), \phi_d(x)). \tag{F.3.6}$$

Projection to the first coordinate recovers ρ_d. Equal refined values have equal failure kinds, so the two sources lie in the same sum branch. That branch stores proofs of the same proposition, and proof irrelevance identifies the sources. Thus $\hat{\rho}_d$ is injective on all of Ω_d.

For the refined visible repair readout, take $\widehat{A}_d(y, k) = R_d(k)$. Equation (F.3.4) gives $\widehat{A}_d(\hat{\rho}_d(x)) = \kappa_d(x)$ for every original source. This supplies both factorization of the canonical-repair readout and authority factorization. The compiler remains the originally installed R_d.

We must also show why the compiler's chosen repair is effective. The original geometry preserves the same payment, interest, responsibility phase, and associations outside the intervention. Its outcome capacities are:

Original failure source	Remove capture	Restore route
Capture through control of a tenure relation	1	0
Uncaptured route interruption	0	1

Define effectiveness as positive outcome capacity. Reading the two rows gives

$$\mathrm{Effective}_d(x,a) \quad \Longleftrightarrow \quad a = \kappa_d(x). \tag{F.3.7}$$

To construct the proof-carrying repair, choose $\kappa_d(x)$, attach the corresponding row calculation giving outcome capacity 1, and attach (F.3.7). The original candidate-repair type has two constructors, so this exhausts the effectiveness test. The object generated here is a repair proof in counterfactual geometry; the two original branches in Appendix F.4 provide actual installation, selected write-back, and successor.

The direct consumer preserves the kernel and reversible symmetry at every depth, all negative conclusions, the obstruction for arbitrary output readouts, a proof-carrying repair for every original source, and the injective refinement and positive factorizations. The total object S_h exports the same results for every foundation horizon h; h does not bound the depth d above. The original declarations are `fullDomainRepairAuthorityNoGos`, `fullDomain_factoredReadout_not_authorityComplete`, `failureKindRefinementControls`, and `proofCarryingFullDomainRepairAuthorityObstructionDirectConsumer`.

F.4 Actual failure support, finite traces, and probability folds (G26)

F.4.1 Actual support at the two original installed coordinates

Continue with $\Omega_d, c_d, r_d, \kappa_d$ from Appendix F.3. Write $\mathcal{A}^{\mathrm{c}}_d$ for the original UBI coordinate's actual renewal type indexed by capture removal, and $\mathcal{A}^{\mathrm{r}}_d$ for the independent route coordinate's actual restoration type indexed by route restoration. The actual-witness type depends on the original source branch:

$$\mathcal{A}_d(x) = \begin{cases} \mathcal{A}^{\mathrm{c}}_d, & x \in P^{\mathrm{c}}_d, \\ \mathcal{A}^{\mathrm{r}}_d, & x \in P^{\mathrm{r}}_d. \end{cases} \tag{F.4.1}$$

The first branch is generated from the installed UBI restriction at the original visit. Its repair equals both the installed compiler's output and the action in the actual payload disposition. The original operational authority generates the bounded reform, responsibility consumer, selected responsibility row, whole-ledger write-back, and same-law successor together. The second branch is generated by the independent route-restoration recognition. Its original source retains non-capture, payment, actual service, and the emptiness of the route-receipt type; the actual disposition's repair is route restoration. Its own operational authority generates the selected row, responsibility consumer, and bounded reform. At the exact next visit it then reads the target route receipt, feasible exit edge, and capacity 1.

Both original producers take only depth. Take their actual witnesses $u^{\mathrm{c}}_d \in \mathcal{A}_d(c_d)$ and $u^{\mathrm{r}}_d \in \mathcal{A}_d(r_d)$. Case analysis on any original source using (F.3.1) supplies a witness, so

$$\mathrm{Support}_d = \{x \in \Omega_d \mid \mathrm{Nonempty}\, \mathcal{A}_d(x)\} = \Omega_d. \tag{F.4.2}$$

This equality concerns the entire original source type; both endpoints of the original authority kernel therefore have actual witnesses jointly.

The root events of these two witnesses equal the event emitted by the original root at the same exact temporal visit, and their source entries return to the original root entry. The original ledger compiler supplies both whole-ledger write-backs, and both successors equal the original root's generated successor. Under the original transport for dependent payload types, the source entries, selected rows, and whole-ledger write-backs are compatible; successor states are directly equal. Each branch retains its own original consumer and actual action. The route projection differs from the UBI projection, and the canonical repairs differ. Retyping route restoration into the first branch would make its repair index equal both capture removal and route restoration, a contradiction. The converse retyping fails in the same way. This proves both actual common provenance and separation of the coordinates.

F.4.2 Finite traces and event identity

Let the carrier of all dependent actual occurrences be

$$\mathcal{A} = \sum_{d \in \mathbb{N}} \sum_{x \in \Omega_d} \mathcal{A}_d(x), \quad a_d^{\mathrm{c}} = (d, c_d, u_d^{\mathrm{c}}), \quad a_d^{\mathrm{r}} = (d, r_d, u_d^{\mathrm{r}}). \tag{F.4.3}$$

The original temporal visit v_d is generated recursively from the initial visit and the original actual successor. Its causal depth satisfies $\mathrm{depth}(v_d) = \mathrm{some}(d)$: this holds directly at zero, and the successor step increases historical depth by one. Thus v_d is injective in d. The exact root event $e_d = (v_d, \mathrm{event}(v_d))$ contains this visit, so e_d remains injective in depth even if a local receipt value repeats.

For every finite horizon h, the original trace concatenates the two canonical actual branches:

$$T_h = [a_0^{\mathrm{c}}, \dots, a_{h-1}^{\mathrm{c}}] +\!\!+ [a_0^{\mathrm{r}}, \dots, a_{h-1}^{\mathrm{r}}]. \tag{F.4.4}$$

This is a dependent trace of two actual coordinates of the same root over a finite horizon. Every item retains its branch provenance.

Proposition F.4.1. For all $h, d \in \mathbb{N}$,

$$\begin{aligned} |T_h| = 2h, \qquad & \mathrm{Nodup}(T_h), \\ a_d^{\mathrm{c}} \in T_h \iff d < h, \qquad & a_d^{\mathrm{r}} \in T_h \iff d < h. \end{aligned} \tag{F.4.5}$$

Every member retains its exact source, branch consumer, actual action, commutation of the selected row with the world, whole-ledger write-back, and original successor.

Proof. Each branch is an image of the original $\mathrm{range}(h)$, so has length h. Equal dependent occurrences have the same depth by their first projection; hence each branch is injective. If occurrences from different branches were equal, reading their original failure kinds would equate capture with route interruption, a contradiction. The lists are therefore individually duplicate-free and disjoint, so their concatenation has no duplicates and has length $2h$.

The canonical capture item belongs to the first list exactly when its depth belongs to the original range; it cannot belong to the second. The route item is analogous. Eliminating membership of a general list member shows that it is one of these two canonical items at some $d < h$. Expanding its original producer yields all the stated provenance and consumer-commutation facts.

The membership result concerns the canonical actual occurrences selected in (F.4.4); it does not identify an arbitrary additional actual witness with a witness in that list. ■

The bare-root readout sends both a_d^{c} and a_d^{r} to e_d. For $h > 0$, both depth-zero occurrences lie in T_h, so the bare-root list has a duplicate. The readout stamped with the original failure kind,

$$a \longmapsto (\phi(a), e(a)), \tag{F.4.6}$$

has no duplicates: within a branch, injectivity of e_d in depth recovers the original item; across branches, failure kind separates them. The bare-root list of either branch separately also has no duplicates. The collision is thus located precisely at the identification of different repair coordinates at the same depth.

F.4.3 Unit counting and the normalization rule

The original finite measure attaches weight 1 to each member of (F.4.4). The absence of duplicate occurrences supplies exactly the support condition required by the finite measure. For any decidable event $B \subseteq \mathcal{A}$, its mass is

$$\mu_h(B) = \sum_{\substack{a \in T_h \\ B(a)}} 1 = \big|\mathrm{filter}(B, T_h)\big|, \qquad \mu_h(\mathcal{A}) = 2h. \tag{F.4.7}$$

List induction proves the second equality: both sides are zero for an empty list; when a head is added, both increase by one or remain unchanged according as B holds or fails. Taking the everywhere-true event and using (F.4.5) gives the total mass. The original construction supplies uniform weights; no separate probability is stipulated.

For an observer $\eta : \mathcal{A} \to V$ and an event E, write $F_h(y) = \mathrm{filter}(\eta = y, T_h)$ and $F_h^E(y) = \mathrm{filter}(\eta = y \wedge E, T_h)$. The original normalized finite-observer kernel fold reads

$$P_h(E \mid y) = \begin{cases} \mathrm{none}, & |F_h(y)| = 0, \\ \mathrm{some}\left(\dfrac{|F_h^E(y)|}{|F_h(y)|}\right), & |F_h(y)| > 0. \end{cases} \tag{F.4.8}$$

The ratio lies in the nonnegative rationals. Normalization is performed only at positive fibre mass; since the event fibre is contained in the observation fibre, the resulting ratio is at most one. At horizon zero, $T_0 = []$, so the original zero-horizon coarse fold returns none for every visible query.

F.4.4 The original per-depth fold and its refinement

The actual observer and event are read from the original occurrence:

$$\eta(a) = (e(a), \rho_{d(a)}(x(a))), \qquad E(a) \iff \mathrm{repair}(a) = \mathrm{removeCapture}. \tag{F.4.9}$$

The original installed component supplies the actual repair, so $E(a)$ is equivalent to the original failure kind being capture. Fix any d. The original definition `sourceGeneratedActualRepairFailureKernelFoldAt` d uses μ_{d+1}, with query $y_d = \eta(a_d^{\mathrm{c}}) = (e_d, (0, \emptyset))$.

If the observation of a_k^{c} or a_k^{r} equals y_d, equality of exact root events gives $k = d$. Conversely, (F.3.3) makes both endpoints at this depth satisfy the query. Filtering range$(d+1)$ by $k = d$ leaves exactly its last index. Consequently,

$$F_{d+1}(y_d) = [a_d^{\mathrm{c}}, a_d^{\mathrm{r}}], \qquad F_{d+1}^{E}(y_d) = [a_d^{\mathrm{c}}]. \tag{F.4.10}$$

Equation (F.4.7) gives fibre mass 2 and event mass 1; (F.4.8) then yields $P_{d+1}(E \mid y_d) = \mathrm{some}(1/2)$.

The probability readout itself is $P_{d+1}(E \mid -) \circ \eta$, and therefore factors through the original actual observer. The two actual occurrences at the same depth have the same observation but different repairs, forming an authority kernel for this observer. A decoder recovering every actual repair from this probability readout would compose to an authority factorization forbidden by that kernel. The common value $1/2$ therefore lacks repair-authority completeness.

Refine the actual observer to $\hat{\eta}(a) = (\eta(a), \phi(a))$, retaining the same μ_{d+1} and event E. The original compiler satisfies $\mathrm{repair}(a) = R(\phi(a))$, so $\widehat{A}(y, k) = R(k)$ is an authority factorization of actual repair. The two original queries have fibres

$$\begin{aligned} \widehat{F}_{d+1}(\hat{\eta}(a_d^{\mathrm{c}})) &= [a_d^{\mathrm{c}}], \quad \widehat{F}_{d+1}^{E}(\hat{\eta}(a_d^{\mathrm{c}})) = [a_d^{\mathrm{c}}], \\ \widehat{F}_{d+1}(\hat{\eta}(a_d^{\mathrm{r}})) &= [a_d^{\mathrm{r}}], \quad \widehat{F}_{d+1}^{E}(\hat{\eta}(a_d^{\mathrm{r}})) = []. \end{aligned} \tag{F.4.11}$$

Both observation fibres have mass 1, with event masses 1 and 0. Their readouts are thus some(1) and some(0). The latter has a nonempty observation fibre and differs from the zero-horizon value none.

What is proved here is separation of the canonical actual pair by refinement, together with repair-authority factorization over the entire actual-occurrence carrier. The injectivity result for the original source type in Appendix F.3 is not extended to all dependent actual witnesses. The value $1/2$ uses precisely the horizon $d + 1$ and query y_d of the original per-depth fold.

The original direct consumers `proofCarryingActualRepairFailureTraceDirectConsumer` and `proofCarryingActualRepairFailureProbabilityDirectConsumer` retain actual support at every depth, traces and unit counting at every finite horizon, and the exact coarse and refined fibre lists, masses, and probabilities at each depth. The actual-support, kernel, and probability exports of the total object S_h consume these proofs field by field.

F.5 The conservative first successor of an exhausted complete ledger

This section separates Proposition 7.8 into a closure lemma for arbitrary world networks and a three-row instance in an existing actual runtime. Progress budgets take values in $\mathbb{N}$; they measure the structurally permitted progress of responsibility rows.

F.5.1 Two levels of origin coverage

In any type universe, fix a world network N, vocabulary V, source-native root R, visit v, its exact event η, a live responsibility row a_0, and its source-native authority α. Put

$$L = \mathrm{Rows}(R.\,\mathrm{emit}(v)), \qquad \mathrm{Pos}(L) = \sum_{a \in L} \{\, 0 < B(a) \,\}. \tag{F.5.1}$$

The dependent sum retains the particular responsibility row and its proof of positive budget.

A source-native sequential action target T over $(R, v, \eta, a_0, \alpha)$ contains a lawful world transport, target root R_T, target initial ledger L_T^0, and its first complete write. The ingress map $i : L \to L_T^0$ preserves the original responsibility, its content, and protected interest, and satisfies

$$\forall a \in L, \quad B(i(a)) \le B(a), \qquad \forall a' \in L_T^0, \quad \exists a \in L,\ i(a) = a'. \tag{F.5.2}$$

The second condition covers the entire ingress ledger, beyond the queried row a_0.

The first compilation generates an exact successor occurrence η_T^1 and its complete ledger L_T^1. Every target row $e \in L_T^1$ has a source-native origin

$$\mathrm{origin}(e) = (a', u_e), \qquad a' \in L_T^0, \quad u_e : a' \rightsquigarrow e, \qquad B(e) \le B(a'). \tag{F.5.3}$$

The last inequality is the no-refill law for lawful row evolution, including permitted carry, maintenance, and transfer. Merge and split coverage receipts remain subject to the same rowwise law.

Lemma F.5.1 (zero-budget closure). If $\forall a \in L,\ B(a) = 0$, every target T above satisfies

$$\forall a' \in L_T^0,\ B(a') = 0, \qquad \forall e \in L_T^1,\ B(e) = 0, \qquad \mathrm{Pos}(L_T^1) = \emptyset. \tag{F.5.4}$$

Proof. Take $a' \in L_T^0$. By (F.5.2), choose a with $i(a) = a'$. Then $B(a') = B(i(a)) \le B(a) = 0$. Non-negativity of natural numbers gives $B(a') = 0$. For any $e \in L_T^1$, its source-native origin in (F.5.3) gives $B(e) \le B(a') = 0$, hence $B(e) = 0$. An element (e, h) of $\mathrm{Pos}(L_T^1)$ would now turn $h : 0 < B(e)$ into $0 < 0$. All three conclusions hold for arbitrary T. □

The proof leaves the target world arbitrary and permits different targets to use different representation types. It consumes the two levels of complete origin coverage and rowwise conservativity already carried by the action type.

F.5.2 Actual expenditure, an actual answer, and the universal restriction coexist

Let v_{old} be any admissible old visit. The actual contribution-capacity action writes its contribution row from budget 1 to budget 0. The subsequent same-source law revision generates the complete live ledger

$$L_{v_{\mathrm{old}}}^{\mathrm{rev}} = \{c^{\mathrm{inh}}, o^{\mathrm{inh}}, r^{\mathrm{new}}\}, \qquad B(c^{\mathrm{inh}}) = B(o^{\mathrm{inh}}) = B(r^{\mathrm{new}}) = 0. \tag{F.5.5}$$

These three elements are the exhaustive row constructors. The first two inherit the old contribution and obstruction. The third is a residual with no old-row preimage and with the existing parent bearer. Appendix E.7 gives its origin and first write.

Proposition F.5.2 (an answer exists while the positive-restoration fibre is empty). For every v_{old} and every revised-root visit index $k \in \mathbb{N}$:

1. The generated residual has the answer face and independent processing operation generated natively at that visit.
2. Every live responsibility row of the revised root has budget zero.
3. For any source row a at that visit, its source-native authority α, exact source event η, and any source-native sequential action target T over these objects, $\mathrm{Pos}(L^1_T) = \emptyset$.

Proof. The revised root's residual inquiry directly generates the answer. For the second claim, exhaustive three-way classification reduces any live row to one constructor in (F.5.5), each with budget zero by definition. This is the complete source ledger at every revised visit. Lemma F.5.1 therefore applies separately to arbitrary k, a, α, η, T, proving the third claim. Taking the actual answer consumer from the first claim also gives

$$\mathrm{Answer}_{v_{\mathrm{old}},k} \neq \emptyset, \qquad \mathrm{Pos}(L^{\mathrm{rev}}_{v_{\mathrm{old}}}) = \emptyset. \tag{F.5.6}$$

A proposed "restoration receipt" pairing an answer consumer with a positive-budget row has its second component in the empty fibre. □

At the canonical old visit, the actual runtime certificate also retains the lawful strict $1 \to 0$ expenditure, exhaustive three-row classification, root-semantic law, residual answer, absence of an old preimage, and existing bearer. The universal part of Proposition F.5.2 is installed with these actual witnesses in the same canonical society object. It constrains a conservative first successor whose complete source is the exhausted ledger; later service and regeneration occurrences remain accountable through their own sources, responsibilities, and complete writes.

F.6 The sixteen-point root grammar and joint realization at every depth (G1/G2, G14/G18)

F.6.1 Eight bearing faces in one source

Fix any original visit v, source index s, and grammar parameter i. The original grammar is visited at the post-settlement renewal visit $v_{s,i+2}$, and its occurrence is the exact event emitted by the original root there. The grammar parameter i and actual occurrence index $i + 2$ remain distinct.

The original vocabulary has two scenarios, actual and full-capture search, and a bearing-face type with the following eight constructors. Every passed payload is read from the same post-settlement occurrence:

Bearing face	Actual payload
Whole ledger	Ledger evolution generated by the original source ledger compiler
Renewal	The occurrence's actual service exit
New-bearer exit	Bearer-exit material generated from that occurrence's actual exit
Bearer exercise response	The actual exercise response of the same bearer exit
Bearer leave execution	The actual leave execution of the same bearer exit
Bearer role-independent continuation	Role-independent continuation of the same bearer exit

Bearing face	Actual payload
Tenure access	The same actual exit and the original tenure relation authority
Health-service access	The same actual exit and the original health-service relation authority

Write the scenario set as $\mathcal{S} = \{\mathrm{a}, \mathrm{f}\}$ and the eight-face set as $\mathcal{B}$. The fixed index list contains the eight actual faces followed by the eight search faces. For each $(s', b) \in \mathcal{S} \times \mathcal{B}$, the original point constructor produces a dependent point $p_{s',b}$ carrying its scenario, bearing face, and already classified outcome. All eight actual points carry the passed payloads in the table. The search points carry those original payloads except at role-independent continuation. At that point, the passed type requires actionable exit under full capture, which the original full-capture emptiness proof rules out; the constructor therefore produces the original failure bundle.

Take the first actual ledger point as the root and the remaining fifteen points as branches of this source occurrence, obtaining the original accounted unfolding U. Its trace satisfies

$$\mathrm{trace}(U) = [p_{\mathrm{a},b_1}, \dots, p_{\mathrm{a},b_8}, p_{\mathrm{f},b_1}, \dots, p_{\mathrm{f},b_8}]. \tag{F.6.1}$$

Its length is 16. Equality of two points gives equal coordinates under both the scenario and bearing-face projections. Since the index list has no duplicates, neither does the point list. Every scenario–face pair has its canonical point in the list. If another member has the same two coordinates, extracting its index from list membership recovers that same index and point. Every scenario–face combination therefore occurs exactly once in the original trace.

The original root-projection type is also actually covered: each projection is either the container projection installing the complete society grammar or the projection of one of these eight faces. The proof splits on the original projection constructors. The essential-access branch then splits into the original tenure and health-service channel constructors, with proof irrelevance handling their proof payloads. This exhausts the original root-projection type without requesting a coverage proof from a caller.

F.6.2 Actual and failure folds of the same unfolding

The complete-audit algebra places the current point before the concatenation of all child audits:

$$\mathrm{AuditAlg}(p, [L_1, \dots, L_k]) = [p] +\!\!+ L_1 +\!\!+ \cdots +\!\!+ L_k. \tag{F.6.2}$$

Induction over leaves and branches shows that this fold is exactly the original trace, preserving the original passed payload or complete failure bundle within every point. Scenario restriction only filters this complete audit by its scenario field.

Grammar compilation folds the same U. It first combines all compiled child branches and then examines the current point: a point in another scenario contributes no outcome of its own; a passed point in the chosen scenario contributes to the passed list; and a failed point in that scenario returns

its original failure. The combination rules are

$$\begin{aligned} \mathrm{failed}(f) \star z &= \mathrm{failed}(f), \\ \mathrm{canonical}(L) \star \mathrm{failed}(f) &= \mathrm{failed}(f), \\ \mathrm{canonical}(L) \star \mathrm{canonical}(M) &= \mathrm{canonical}(L +\!\!+ M). \end{aligned} \tag{F.6.3}$$

Substituting (F.6.1), the actual scenario produces the eight passed items in table order, while the search scenario produces the original role-independent-continuation failure bundle. Both compilations are scenario restrictions of the same installed unfolding.

If a candidate evaluator C commutes with the constructor algebra (F.6.3) for every source point and branch, then $C(U)$ equals the original fold. The proof is induction over the accounted unfolding. At a leaf, the commutation condition fixes the result. At a branch, the induction hypotheses replace each child result by its original fold value, and the commutation condition fixes the parent value. This uniqueness concerns compatible evaluators on a fixed source and fixed algebra.

The role-independent-continuation item in the actual passed list is literally a member of the eight-item list. Its readout equals the successor-continuation root readout of the original paid-exit lifecycle. The original dependent construction retains that membership proof, readout equality, and original lifecycle maximum together, so the lifecycle actually consumes this item of the actual fold. The search failure does not replace this actual consumer chain.

F.6.3 The failure bundle and original world writing

The search failure retains the following actual objects: the original event's actual exit as residual bearer, original payment-source ownership, the original root responsibility consumer for that bearer, and the audit disposition dependent on those objects. The disposition further retains the original whole-ledger write-back, original role-independent continuation, original payment-root factorization, and exact target-current increment. The emptiness proof for actionable exit under full capture remains in the same bundle. At index $i+2$, the original source also generates external liability from the same grammar event and the same actual exit.

For each generated object, a claim that its type is empty is refuted by the object itself. Hence each of the four exact deletion interfaces—for the residual bearer, source ownership, original consumer, and dependent disposition—is empty. These non-deletion conclusions are witnessed by the original failure bundle.

Before emission, the original installation has fixed the actual scenario as a. The full-capture search scenario satisfies $\mathrm{f} \neq \mathrm{a}$, so its actual-write authority type is empty. Relabelling search as the installation's actual scenario likewise contradicts disjointness of the two scenario constructors. The search audit's ledger disposition is identity on the original ledger. Actual grammar Γ_i^{a} and search grammar Γ_i^{f} read the same recognition and visit, giving

$$\begin{aligned} \mathrm{event}(\Gamma_i^{\mathrm{a}}) = \mathrm{event}(\Gamma_i^{\mathrm{f}}), \qquad \mathrm{ledger}(\Gamma_i^{\mathrm{a}}) = \mathrm{ledger}(\Gamma_i^{\mathrm{f}}), \\ \mathrm{next}(\Gamma_i^{\mathrm{a}}) = \mathrm{next}(\Gamma_i^{\mathrm{f}}) = \mathrm{current}(v_{s,i+3}), \\ \mathrm{applySearchAudit}(L_i) = L_i. \end{aligned} \tag{F.6.4}$$

Correspondence between dependent ledger types follows the original source transport; the generated successor factors through the original root's same answer and next state. Search can read this actual successor exactly without thereby acquiring new write authority.

The original observer–consumer interface further handles sealed original submissions with arbitrary output universes. Its local use disposition is presentation, rooted use, or rejection. Let $\mathrm{Cert}(z)$ mean that the generated submission carries certified authority, let $s(z)$ be its scenario, and let s_{inst} be the installed actual scenario. Then

$$\mathrm{Rooted}(\mathrm{compile}(z)) \quad \Longleftrightarrow \quad \mathrm{Cert}(z) \wedge s(z) = s_{\mathrm{inst}}. \tag{F.6.5}$$

This interface handles the use of observation results within root law; root-law revision continues to use the original U8 entry. The original direct consumer supplies a rooted positive case for an actual certified submission, rejection of a certified search submission, ordinary presentation, rejection of an uncertified authority submission, and rejection of a same-visible/different-authority kernel challenge. Two generation proofs for the same exact submission produce the same compiled use disposition. This coherence and the preceding fold uniqueness each concern their own interface with fixed inputs.

F.6.4 Joint totality over every original cycle

Take any original cycle $C \in \mathcal{C}(v, s, b)$ and its original fivefold constitutional proof Q_C. Let $\Gamma(v, s, i)$ denote the original root law with grammar parameter i. For every observer resource o and depth d, write

$$i_d = b + d + 2, \qquad \Gamma_d = \Gamma(v, s, i_d). \tag{F.6.6}$$

The actual grammar occurrence of Γ_d is at $i_d + 2$; the next actual grammar occurrence is at $i_d + 3$. These offsets come from the original lifecycle and grammar-visit definitions; the grammar parameter is not substituted for the occurrence index.

Proposition F.6.1. For every C, Q_C, o, d, there is a joint realization point retaining:

1. The original root-law transition from Γ_d to Γ_{d+1}.
2. The full capture-search failure generated from the same Γ_d.
3. The original actual same-source self-U8 at this depth.
4. Equality of the next states read by both actual and search grammars with the authoritative current of Γ_{d+1}.
5. The exact original cycle-successor equation $C_{d+1} = \mathrm{nextCycle}(\mathrm{step}(C, d))$.

At the same time, the conjunction type asserting both actual-write authority for search and the original actual self-U8 at this depth is empty.

Proof. At (d, o), take the root-law transition and actual self-U8 from the original root-law openness field of Q_C. Invoke the failure producer with the original grammar parameter i_d. The resulting failure bundle's grammar is the transition's source law: unfolding both sides gives the same definition, so the equality is definitional, without a submitted grammar candidate. The original transition proof identifies the actual grammar's next state with the target law's authoritative current. For search, first use (F.6.4) to identify its next state with the actual grammar's next

state, then use that transition equality. This gives the same target. The original recursive definition supplies the exact cycle-successor equation.

An element of the stated conjunction would have a first component asserting that search actual-write authority is nonempty. Choosing that authority object immediately contradicts the write-authority emptiness in the same failure bundle. The contradiction uses only search write authority, while the original constitution positively retains actual self-U8. ■

Thus, for every existing Q_C, these joint realization points are nonempty at every (o, d). The original totality consumer also carries the observer–consumer exhaustion proof in (F.6.5).

Finally, fix any finite request parameters f, o. The original fivefold constitution gives a depth m and a requested constitutional point there. Apply Proposition F.6.1 at this same m, and pair the original requested point with the original grammar realization point. Writing their types as Req and Joint, respectively, yields

$$\forall f, o\ \exists m,\ \mathrm{Nonempty}(\mathsf{Req}_{C,f,o,m} \times \mathsf{Joint}_{Q_C,o,m}). \tag{F.6.7}$$

Every original cycle has its original fivefold constitutional construction, so this result is not limited to a selected registration in the total object. The total-object export consumes its registered original cycle and original successor-recognition construction, preserving all depth points and requested points over the same constitutional proof.

The sixteen-point realization, arbitrary-depth joint point, and joint requested point are connected by these direct consumers:

- `proofCarryingUniversalSocietyRootGrammarDirectConsumer`
- `canonicalSocietyUniversalGrammarTotality_directConsumer`
- `everyRequestedCanonicalSocietyUniversalConstitutionAt`

These distinct proofs connect general root-law classification, actual grammar construction, and joint-point totality, each retaining its own inputs and quantifiers.

F.7 Actual succession after the original mandate exit

This section supplies the succession part of Proposition 5.7. Fix all the parameters of Appendix E.5:

$$o : \mathrm{OldVisit}, \quad a, b \in \mathbb{N}, \quad c_0 : \mathrm{CycleAt}(o, a, b), \quad h, d \in \mathbb{N}. \tag{F.7.1}$$

Here h is observer fuel and d is original cycle depth. Succession generation accepts only c_0, d; its complete consumer additionally quantifies over every h.

F.7.1 The second action consumes the exact actual successor of exit

Write R^- for the original exit target root and $x_0, x_1, x_2, \ldots$ for its initial visit and finite continuation. The macro actual successor of the original exit action A_d^- is exactly

$$\mathrm{Next}(A_d^-) = \langle \mathcal{V}^-, R^-, x_1 \rangle. \tag{F.7.2}$$

The visit-depth function takes value j at x_j. Induction and projection give injectivity of x_j; in particular, $x_1 \neq x_0$.

At x_1, install the original exited-mandate component and UBI payment component again. The mandate still reads the original exit history: its power is lost, its phase is exit, its role is exited, and its original responsibility slot remains a terminal archive. This visit has its own complete payment P^-, root answer–successor factorization, and complete whole-ledger write-back. Its root-generated next visit is x_2. The old responsibility has no reopening-event type.

The second sequential action A_d^+ takes x_1 as its source visit. Its original responsibility entry and actual write authority are taken directly from the target entry and target authority of A_d^-. Its exact source occurrence is heterogeneously equal to the post-exit occurrence just constructed. The second action therefore consumes the object reached by (F.7.2), rather than merely reusing a target root of the same shape.

F.7.2 Generating the successor from the exit history

Write H^- for the original exit history. For an occurrence node e and the controller-seed maxima $m_1, \dots, m_t$ already computed on its branches, the fold algebra is

$$F(e; m_1, \dots, m_t) = \max\{e.\text{controllerSeed}, m_1, \dots, m_t\}. \tag{F.7.3}$$

Its value on the originally supplied exit history is 13. The new event takes the old current period plus 1, the old current office seed, and this maximum plus 1 as its period, office seed, and controller seed, respectively, and takes continuation as its disposition:

$$e^+ = (H^-.\text{period} + 1,\ H^-.\text{officeSeed},\ 13 + 1,\ \text{continue}), \qquad H^+ = \text{next}(H^-, e^+). \tag{F.7.4}$$

Uniqueness of the original occurrence fold ensures that every candidate evaluator g satisfying the commuting equation (F.7.3) at each occurrence constructor also takes value 13 on H^-. The new controller seed 14 is thus computed from the source and differs from both the original standing seed 11 and the exit seed 13.

The trace theorem for history continuation places the current event of H^- in the occurrence trace of H^+. The original election-lineage compiler relates the controller generated by the new event to the old office. The office seed is unchanged, the relation has exact trace provenance, and the current controller is 14. Power is restored, the relation phase is renewal, and the target role is held.

The new responsibility enters a live state through the original process's admission edge from an empty local state. Write D^- for the old exit obligation and D^+ for the new obligation. Then

$$\text{slot}(D^-) = \text{slot}(D^+), \qquad D^- \neq D^+, \qquad \text{bearer}(D^-) = 13, \quad \text{bearer}(D^+) = 14. \tag{F.7.5}$$

Slot equality compares the ordinals in their respective one-slot representations. Equality of the obligations would give $13 = 14$ on bearer projection, a contradiction. The original process's terminal-reopening event is an empty type, so the new admission does not reopen the old archive.

If the old exit event were used as the new current action, current-admission equality would again force controller seeds $13 = 14$, which is impossible.

F.7.3 The actual target, two lawful transports, and the complete first write

Generate the target root R^+ from H^+. It installs exactly one mandate component, separate from the original recognition, UBI, proof-carrying, inquiry, essential-service, route, common-clearing, and clearing-backstop coordinates. The mandate component itself has a one-point coordinate type. Eliminating the one-point constructors of any two coordinates makes them equal, so a second distinct installed mandate coordinate cannot exist.

The target initial visit y_0 installs the new mandate and its UBI payment restriction, generating target complete payment P^+. Let P^{con}_{d+2} be the constitution-embedded payment at depth $d+2$ of the original cycle, let ϕ_d be the lawful presentation from its occurrence to the post-exit occurrence, and let ψ_d be the target presentation of the second actual action. The two original commuting squares are

$$\phi_d(e(P^{\mathrm{con}}_{d+2}).\mathrm{occurrence}) = e(P^-).\mathrm{occurrence}, \qquad \psi_d(e(A^+_d).\mathrm{occurrence}) = e(P^+).\mathrm{occurrence}. \tag{F.7.6}$$

The first equation follows by substituting the equality of the original embedded payment with the corresponding periodic payment and unfolding the presentation. The second follows from the action target's occurrence-presentation definition. $e(A^+_d)$ joins the exact post-exit source occurrence at their common dependent index. Equation (F.7.6) retains the two roots and their respective visits; it asserts commutation under the forward maps of the presentations.

The second action's target uses the original identity responsibility translation. It preserves the initial open ledger, responsibility content, and budget, adding no initial responsibility row without an old source. The new mandate's admission comes from the local responsibility process in (F.7.4); this and preservation of the macro initial ledger's sources have separate roles. At y_0, the target initial generator completes the whole-ledger update and obtains first-write successor S^+ from its actual generated row. The macro actual successor is the next visit produced by the target answer:

$$\mathrm{FirstWriteSuccessor}(A^+_d) = S^+, \qquad \mathrm{Next}(A^+_d) = \langle \mathcal{V}^+, R^+, y_1 \rangle. \tag{F.7.7}$$

The first type depends on the target initial occurrence and its complete write-back; the second contains the target vocabulary, root, and next visit. Target initial payment P^+, the complete first write, and the subsequently adopted y_1 are thereby connected, each in its original type.

F.7.4 Services, external responsibility, and subsequent openness

The finite constitution at depth $d+2$ of the original cycle supplies an exit renewal with the same action c and the same service slot j. Eliminating this operational incidence once simultaneously gives membership of c in the original carrier, continuation outside the role, non-idleness, non-capture, the same source payment, and a route to the actual service at j. Its temporal part retains the original lifecycle's external-responsibility disposition, services and later continuation, and injectivity of actual service occurrences.

The original joint pulse at that same depth supplies root-law continuation, the same-root/ledger/successor relation between actual and diagnostic scenarios, actual same-source law revision, and compatible future birth. These fields join the succession interface through the constitution-payment presentation above. The same-root statement for actual and diagnostic scenarios still belongs to the original cycle's root law; the second action receives the actual authority already generated at x_1.

Assemble the complete exit consumer, post-exit terminal retention, the second action's receipt, the two presentation commutations, separation of controllers and obligations, target-root factorizations, (F.7.7), and these operational data. This gives the complete succession consumer for every c_0, h, d. Deleting any of its original objects contradicts the element supplied by that object itself. Incorrect reports of lost power, exit phase, or exited role contradict the successor's restored, renewal, and held constructors, respectively. Its compilation is an actual sequential action and cannot also be the original law-revision obstruction; the diagnostic scenario has no actual write authority.

Finally, original succession-action occurrences retain their depth index in the common type

$$\mathcal{O}(c_0) = \sum_{d:\mathbb{N}} \mathrm{ActualSuccessionAt}(c_0, d), \qquad \eta_d = (d, A_d^+). \tag{F.7.8}$$

Equality $\eta_d = \eta_{d'}$ gives $d = d'$ by projection, so η is injective. A single occurrence cannot equal η_d at every depth, since depths $0, 1$ give a contradiction. Complete generation at each requested depth retains this source distinction.

The original production chain is sourceMandateSuccessorHistory, generatedMandate-SuccessionActionAt, and sourceGeneratedMandateSuccessionConstitutionAt. Its complete direct consumer is sourceGeneratedProofCarryingMandateSuccessionConstitutionAt. For every canonical foundation parameter and o, a, b, h, d, the whole-program interface canonicalSociety_rootControlledMandateSuccession takes c_0 from the original cycle registered by the total object and supplies both the complete consumer and its exact negative controls.

F.8 Actual merging of the two risk child rows

This section supplies the execution following risk allocation in §7.7. The general merge construction is parameterized by any lawful rooted failure. The three-tick runtime statement uses the existing canonical failure and its determinate runtime.

F.8.1 Complete ingress and the exact three-row write

Fix the old world vocabulary, old authoritative root and visit, original $U7$ calculus, old theory, actual obstruction and expressibility failure, and a rooted witness h of that failure. Write N_h for its already generated risk-allocation world. Its target ledger has three rows: contribution children a, b and obstruction-continuity row o. The rows a, b have the same responsibility content and protected interest, but different leg identifiers and existing bearers.

The merge world's support is N_h.Support $\times$ Bool. At phase false, every old support and row enters unchanged. Responsibilities, their content, protected interests, incidence, and lineage are

inherited through the original projection. The actual initial support pairs the risk-allocation target support with false. The resulting initial-row presentation has mutually inverse maps, so every initial target row has a preimage in the original three-row ledger.

The actual merge target lies on the true face of that risk support. It has exactly two rows, common contribution m and obstruction continuation o', supplied by disjoint openness constructors, with budgets $1, 0$. The common contribution retains the responsibility content and protected interest already shared by its two parents; the obstruction row retains the old obstruction content. The complete row table of the actual first write is

source	target	budget	lawful evolution
a	m	$1 \to 1$	transfer
b	m	$1 \to 1$	transfer
o	o'	$0 \to 0$	transfer

(F.8.1)

These are lawful transfers across supports, preserving the corresponding lineage and responsibility content. The third row also has an actual transfer proof.

Lemma F.8.1 (two-sided coverage of the first write). Table (F.8.1) covers every source row and every target row. Its destination map d and selected origin map j satisfy

$$d(a) = d(b) = m, \quad d(o) = o', \qquad j(m) = a, \quad j(o') = o. \tag{F.8.2}$$

Proof. Classification by initial openness constructors makes every source row one of a, b, o, with table indices $0, 1, 2$. Classification by target openness constructors makes every target row either m or o', with selected table indices $0, 2$. Substituting these index functions into the three-row table supplies source-side destination coverage and target-side origin coverage and computes (F.8.2) directly. The distinct responsibility constructors give $m \neq o'$. □

F.8.2 The complete parent list, bearers, and compiler selection

Each world responsibility row enters the lifecycle through a native admission payload. The payload records the source event consisting of the current support and existing risk-material witness, together with original responsibility content, budget, protected interest, lineage, and bearer. On the old face, bearer readout is inherited pointwise from risk allocation. On the actual target face,

$$\mathrm{bearer}(m) = \mathrm{childBearer}(a), \qquad \mathrm{bearer}(o') = \mathrm{parentBearer}\,. \tag{F.8.3}$$

The map from payloads to admitted obligations retains both exact readouts.

Lemma F.8.2 (injectivity of the admission presentation). At each fixed support and material witness, distinct live rows yield distinct admitted obligations.

Proof. Equality of two admitted obligations projects to equality of their responsibility content. At ordinary supports, their openness proofs are then unique. On the two-leg risk-allocation face, the bearer projection still separates a, b even when their content agrees: equality across the legs would equate the two existing bearers already proved distinct. At the actual merge target, its

two constructors belong to contribution and obstruction content, so cross-constructor equality is impossible; within one constructor, proof irrelevance yields row equality. □

The source event of the merge receipt is the exact event of this first compilation. Its parent list is $[A(a), A(b)]$, and its target is $A(m)$. The content list required by coverage is $[\mathrm{content}(m), \mathrm{content}(m)]$. Each parent carries ownership by this source event, equality of incidence and content with the target, anchor transport, lineage continuation, and preservation of local discharge conditions. Every member of the parent list can be recovered as its zeroth or first element, so these properties hold parent by parent.

Theorem F.8.3 (the exact merge, complete parents, and exclusion of a simultaneous split). The compiler selects the merge receipt for a, b at this first write. Its parent list covers exactly the old rows directed to the common contribution; every parent has an old-row preimage, and o is excluded. For any target rows e, f, equality of their selected origins implies $e = f$. Consequently, split coverage for two distinct targets with the same origin cannot coexist with this write.

Proof. Take a source row directed to m and use Lemma F.8.1 to consider three cases. The rows a, b are the two parents. The destination of o is $o' \neq m$, excluding the third case. Conversely, each parent has a or b as its preimage, so there is no phantom parent. If $A(o)$ appeared in the parent list, Lemma F.8.2 would imply either $o = a$ or $o = b$, contradicting the distinct responsibility constructors.

The compiler chooses the identity classification for equal source rows and the stated merge receipt for a, b or b, a with their common destination. Equality of the destinations of o and a contribution row is ruled out by $m \neq o'$. This exhausts every pair of source rows and proves the exact compiler-selection equation, beyond merely inhabiting a receipt type.

Finally, classify the two target rows. Their selected origins are a, o, and $a \neq o$, so the origin map is injective. A split witness carrying both "equal origins" and "distinct targets" would therefore yield $e = f$ and $e \neq f$. □

F.8.3 From the original revision to actual merging and capacity expenditure

Take the canonical rooted failure. The source root of the merge action is its revised root; the source visit is current 2, the source row is the existing obstruction continuation, and the source event is the native emission at that visit. The action's occurrence presentation supplies lawful two-way transport between this exact source occurrence and the target initial occurrence. Each end retains its own type and compiler identity.

Proposition F.8.4 (three-tick execution). In the installed determinate inquiry runtime:

- The actual execution at depth 0 is the original same-source law revision, whose actual successor enters its generated revised root.
- The actual execution at depth 1 is the merge action above, with the complete receipt of Lemmas F.8.1–F.8.2 and Theorem F.8.3.
- The source root at depth 2 is precisely the target root generated by the merge action, and that tick actually executes expenditure of the common contribution's capacity.

Proof. The initial query type is Unit. Its unique activation law selects the sole query, and expansion of the initial compilation gives the original revision. The actual successor of that tick

is the merge-action presentation, whose root therefore has native dependent equality with the root generated by revision. The compilation branch of the merge presentation is an actual action, so the second tick executes precisely the merge program above. Table (F.8.1) generates its first whole-ledger write. The visit in its answer-and-next object is exactly

$$v_T^0.\,\mathrm{next}(\mathrm{firstSuccessor}(T).\mathrm{nextEq}). \tag{F.8.4}$$

The runtime successor rule uses this actual action successor to enter the capacity-action presentation and supplies

$$\mathrm{HEq}\big(R_{\mathrm{runtime},2},\ T.\,\mathrm{targetRoot}\big)\,. \tag{F.8.5}$$

The third tick's compilation branch is then the capacity-expenditure action. The revision branch at the first tick contains no such merge action; the actual-action branch at the second tick contains no $U8$ core. Both claims follow directly by elimination of the distinct execution constructors. □

The parameter h of the general merge construction and depth 1 of the fixed runtime belong to different quantifier levels. The former gives a complete merge over any lawful rooted failure; the latter proves execution of the canonical instance. Their common certificate is consumed by the canonical society's actual-merge interface.

F.9 Actual relation reselection and reflexive memory

This section concerns the essential-service selection program's own runtime. For the old visits, source indices, boundaries, and initial paid cycles allowed by the existing interface, canonical assembly retains the same all-depth selection interface. Below, n is its relation-reselection depth.

F.9.1 The target root becomes the source root again

Write T_n for the source current at stage n, A_n for its atom type, and s_n for the target seed generated by the current lineage fold. The next type is $A_{n+1} = A_n \sqcup S$, where S is the existing exact-relation-seed type. The target initial state, first write, and retained next action state are

$$\begin{aligned} I_n &= T_n.\,\mathrm{map}(\mathrm{inl}),\\ U_n &= \mathrm{occur}(\mathrm{inr}(s_n), [I_n]),\\ T_{n+1} &= \mathrm{retain}(U_n). \end{aligned} \tag{F.9.1}$$

Ingress contains only the entire history embedded on the left; the first complete write generates the new relation root on the right. The stage structure retains the source root, exact visit, responsibility row, and source-native authority. Its target root is a function of these source data. The recursive next stage takes exactly the previous stage's target root and its retained action visit:

$$R_{n+1}^{\mathrm{src}} = R_n^{\mathrm{tgt}}. \tag{F.9.2}$$

The target selection visit reads U_n, and the following action visit reads T_{n+1}. When vocabulary types change, the visit-commutation equations use HEq.

The action receipt also retains the actual target seed, target role generated by the existing relation compiler, nonzero relation phase, and every inherited observation. For the original UBI restriction, let π_0 be the existing initial projection and define $\pi_{n+1} = \mathrm{inherited}(\pi_n)$ recursively. Each action receipt identifies the target's inherited observation with the original observation at that exact source event by dependent equality. Induction on n then preserves the projected payload's independent processing operation as the originally installed UBI restriction.

Theorem F.9.1 (actual reselection at every depth). Let $\mathcal{R}$ be the installed unique selection runtime and A_n^{act} the action generated natively at stage n. Then

$$\begin{aligned} \mathrm{resolve}(\mathcal{R}, 9+2n) &= \mathrm{Action}(A_n^{\mathrm{act}}), \\ \mathrm{resolve}(\mathcal{R}, 10+2n) &= \mathrm{Answer}(U_n), \\ \mathrm{nextState}(\mathcal{R}, 10+2n) &= \mathrm{state}(\mathcal{R}, 9+2(n+1)) \end{aligned} \tag{F.9.3}$$

for every $n \in \mathbb{N}$. The first equation carries an actual runtime resumption witness for the action, and the second carries its target selection answer. Equations (F.9.1)–(F.9.2), the complete write receipt, and the inherited UBI processing operation hold together.

Proof. The finite prefix executes the old revision, three existing service actions, and their selection readouts. Depth 7 executes the sole relation bootstrap action within one type; depth 8 reads the selection it generated, and its actual successor enters cross-type action 0. The process-state grammar gives this bootstrap branch only a $\mathrm{Fin}\,1$ index, then uses natural-number indices for cross-type actions and selections.

To establish that the later connections belong to the same runtime, first prove injectivity of erasure of process presentations. The old root's support has two elements and the revised support has three, separating the old state from every other state. The lineage readouts of the remaining states include history length. If ℓ_0 is the initial history length of the finite prefix, then

$$|T_n.\mathrm{trace}| = \ell_0 + 8 + 2n, \qquad |U_n.\mathrm{trace}| = \ell_0 + 9 + 2n. \tag{F.9.4}$$

Structural recursion on the tree shows that map(inl) preserves length. Appending a relation root and retaining a selection each add one item, so (F.9.4) follows by induction on n. The finite prefix has smaller distinct ranks; cross-type actions and selections have ranks $8+2n, 9+2n$. Equality after erasure therefore yields equality of ranks and recovers the same process state.

Now induct on actual runtime depth. The case $n = 0$ is the direct computation of the nine-tick prefix. If the node at tick $9+2n$ is the presentation of action n, injectivity of process erasure recovers that actual action state. Expanding the unique activation law and successor rules yields its selection answer at the next tick and action $n+1$ at the following tick. Runtime depths thus satisfy $d_0 = 9$ and $d_{n+1} = d_n + 2$, giving $d_n = 9 + 2n$. Each action target is generated dependently from its source event, so its complete receipt and inherited projections are preserved throughout the state induction. □

F.9.2 Computing every phase of the actual three-cycle

Fix any base depth b. The actual construction composes the action–selection–next-action transitions at $b, b+1, b+2$ into three macro edges. The renewal of the three existing service-seed forms closes after three steps, so the fourth actor's seed and lineage identity return to the first. Equation (F.9.4) shows that the four occurrences still move strictly forward and that the occurrence after three steps differs from the starting occurrence. The three consecutive actors are pairwise distinct: seed renewal separates adjacent actors; if the third equalled the first, the fourth would equal the third, contradicting separation across the third edge.

Only after these actual edges have been generated do we index them by $I = \{0, 1, 2\}$, with cyclic successor σ. The source role s_i of every edge is the existing provider role. Its target role t_i is the tenure, health, or platform provider role generated by that actual relation. The three target roles are pairwise distinct, and each differs from the source role. Define

$$r_j(x) = \begin{cases} 1, & x = t_{\sigma^{-1}j}, \\ 0, & x \neq t_{\sigma^{-1}j}, \end{cases} \qquad w_i(x) = t_i. \tag{F.9.5}$$

These read/write operations use the source and target roles of the actual edges, without accepting a separate phase table.

Lemma F.9.2 (full phase commutation). For every $i, j \in I$,

$$\gamma(i,j) = r_j(w_i(s_i)) - r_j(s_j) = \begin{cases} 1, & j = \sigma i, \\ 0, & j \neq \sigma i. \end{cases} \tag{F.9.6}$$

Thus γ is the entire phase matrix of the actual relation three-cycle.

Proof. Every s_j differs from all target roles, so $r_j(s_j) = 0$. Injectivity of the target-role map gives $t_i = t_{\sigma^{-1}j} \iff i = \sigma^{-1}j \iff j = \sigma i$. Substitution into (F.9.5) computes all nine phases, including those off the selected edges. □

Summing along the three actual edges gives

$$\sum_{i \in I} \gamma(i, \sigma i) = 1 + 1 + 1 = 3. \tag{F.9.7}$$

Point-potential differences sum to zero around a closed loop. This phase therefore cannot be an edge difference of point potentials and yields a nontrivial H^1 obstruction. Adding any coboundary adds a telescoping sum, so the residual magnitude remains 3. It is a presentation invariant of this same actual three-cycle.

F.9.3 Memory on every actual edge and presentation uniqueness in arbitrary universes

Canonical reflexive memory has the single atom $\star$ = "observation changed" . For a read/write query $q = (r, w)$, define

$$M_q(x, \star) \quad \iff \quad r(w(x)) \neq r(x). \tag{F.9.8}$$

The query q_i of actual edge i uses the downstream read $r_{\sigma i}$ and a write fixed to the upstream actual output t_i. Its downstream source state is $x_i = s_{\sigma i}$.

Theorem F.9.3 (edgewise reflexive memory and presentation uniqueness). For every base depth b and every actual selected edge i, $M_{q_i}(x_i, \star)$ holds. Take arbitrary type universes u, v, representation types $O_1 : \mathrm{Type}_u$, $O_2 : \mathrm{Type}_v$, and two sound and complete presentations (C_1, D_1), (C_2, D_2) for q_i. Then

$$D_1(C_1(x_i), \star) \iff M_{q_i}(x_i, \star) \iff D_2(C_2(x_i), \star). \tag{F.9.9}$$

Proof. By Lemma F.9.2, a selected edge reads 0 at its downstream source state and 1 after the upstream actual write. Every edge therefore satisfies (F.9.8).

In general, a presentation consists of a classifier $C : \mathrm{State} \to O$, a decoding predicate $D : O \to \{\star\} \to \mathrm{Prop}$, and

$$\begin{aligned} D(C(x), a) &\Longrightarrow M_q(x, a), \\ M_q(x, a) &\Longrightarrow D(C(x), a) \end{aligned} \tag{F.9.10}$$

These are soundness and completeness. They directly give $D(C(x), a) \iff M_q(x, a)$. Apply this equivalence to each presentation and invert the second one to obtain (F.9.9). The argument allows O_1, O_2 to be infinite, unrelated by any type isomorphism, and situated in different type universes. □

A canonical presentation itself exists: take $O = \{\star\} \to \mathrm{Prop}$, $C(x) = M_q(x, -)$, and $D(f, a) = f(a)$. Soundness and completeness are identity implications. An independently supplied seven-case predicate family requires its own proof of a connection to this actual read/write event. The all-safe predicate family has empty seven-case memory, while every actual edge here still changes observation from 0 to 1, furnishing an exact counterexample.

The canonical society interface installs this cycle-memory core in the same selection runtime. Separate universally quantified functions supply presentation uniqueness for arbitrary type universes and preserve that quantification. Whole-ledger assembly retains the actual three-cycle, complete phase matrix, and memory on every edge. What the institution remembers can therefore be traced to the action that wrote a role and the subsequent read that changed.

F.10 Independent necessity of routes and service-bearer alignment

Routes. Fix an arbitrary depth d. The original periodic route-restoration construction supplies the current root event, complete payment, standing, tenure relation, non-capture, actual service, selected responsibility row, whole-ledger update, independent consumer, and actual successor. At that same visit it also generates seven active installed operations: proof-carrying disposition, basic income, recognition witness, mandate, essential-service restoration, common clearing, and clearing backstop. Each operation's answer and successor factor through the original root.

Source route incidence is an empty type in this construction. Any feasible source edge would project a route receipt, so there is no feasible edge and capacity is zero. The remaining carrier R_d, comprising all the materials above, nevertheless has an actual inhabitant r_d. A function $R_d \to T_d$, where T_d is the deleted source route authority, would yield a contradiction when applied to r_d. Thus even retention of all seven installed operations cannot reconstruct the deleted route.

The repair's actual payment takes the visit from d to $d+1$; only the latter supplies a route receipt and capacity one. Equality of those visits, by visit injectivity, would give $d = d+1$, so they differ. The target receipt consequently has its own type; the emptiness result concerns the source occurrence. The route projection is distinguished from each retained projection by the generated projection constructors.

Services and bearers. Choose any old visit v, source index s, and post-settlement index n. One outer root occurrence already installs three different semantic faces: bearer exit, tenure service, and health service. Reading each face yields bearer-exit object b, first-service object t, and middle-service object h. Each object uses its original installed read, and its consumer retains the object returned by that read.

Exact alignment retains both root reads and their equations. It requires equality between the service slot and the bearer exit's first slot, dependent equality of service and exit receipts, return of the original exit worker by the service relation witness, and individual agreement of payment, actual settlement, purchasing-power counterparty, and original birth bearer. In the tenure channel, both slots are 0; the canonical equations of the original feasible edges identify the service receipts literally. Its relation actor is the original t1 worker, whose bearer equals the original parent-responsibility and birth-admission bearer. The remaining payment and settlement equations follow by returning both objects to the same original input. These materials construct positive alignment.

The health channel has its own genuine middle slot 1, actual service, feasible edge, payment, settlement, consumer, and actual successor. Its relation actor is t2. Exact alignment would require t2 = t1, contrary to the distinction of the original actor constructors. The alignment type is therefore empty. The old protected-interest readings of the two channels agree; their alignment status still requires the source actor.

The outer post-settlement occurrence owns its native write, empty local open-responsibility inventory, external liability, and actual successor. The embedded common-root payment has its own root event. Proved source and successor relations connect the two; in their tagged sum they occupy opposite branches and retain distinct occurrence identities. The original birth bearer remains as settled provenance, without introducing a new live residual row here.

Finally, consider functions reconstructing alignment from the health-service receipt, feasible edge, channel carrier, channel consumer, external liability, or correct-successor equation. Each input has an actual inhabitant while the target alignment type is empty, so application is contradictory. The same argument covers the entire remaining carrier. Original direct consumers are `proofCarryingRouteGateIndependentNecessityDirectConsumer` and the service-bearer alignment consumer; the correspondence records their full identifiers and whole-program fields.

Appendix G Fixed formal proofs and source index

The formal proof selection is H0mework, commit `2d6fbc96e33b7c4c3cfd91104fda74e3caeda045`. All paths below are relative to that repository snapshot; the commit and path together fix each reference. The receipts below record the complete local acceptance of this fixed selection.

G.1 From the main text to complete declarations

The repository files `docs/generated-society-map.json` and `checks/generated-society-selection.json` retain the 29 claims G0–G27 and G2a, 539 source entries, and the complete import/resource selection. Each producing declaration and direct consuming declaration records its original epoch, path and SHA, public module and file, complete declaration name, original complete signature, and accepted complete-type index and digest. G10–G12 remain cited interfaces; C.13/PT15 is retained within G12 as a separate protocol interface.

This paper's `records/public-source-binding.json` connects the 37 rows and 299 original-declaration references in the main-text correspondence to the actual public owners and types. Each row can be traced through its producing, direct consuming, and resource roles. Complete types appear in H0's `evidence/generated-society/acceptance/final-manuscript/types/result.json.gz`, stored as Lean Expr DAGs preserving the original universes, dependent parameters, quantifiers, and premises.

G.2 Seven build entries

Every module has prefix `HOmework.Papers.` and file `Lean/HOmework/Papers/<short-module-name>.lean`.

Short module name	Fixed source and scope in this paper
`GeneratedSocietyMInfinity`	e60; actors, complete foundation, root law, and the full first-refusal chain
`GeneratedSocietyRegeneration`	AB; responsibility birth and all-depth same-occurrence sixfold regeneration
`GeneratedSocietyInterfaces`	df collection context; existing G10–G12 interfaces
`GeneratedSocietyProgress`	1d7; strict paid progress on the original cycles
`GeneratedSocietyEconomy`	df; G14–G27, the current G13 instance, and grammar/relation/memory chains
`GeneratedSocietyProductivitySupplement`	df; actual resource-generated copying and direct consumption by the original root
`GeneratedSocietyProtocolInterface`	e0b; C.13's original window and the same emitter's whole ledger/literal next

The df intake cap does not replace the original proof epochs. The protocol roots read at df have the same bytes as the original certified e0b roots, while their complete imports follow the explicitly selected `e0b980ffb69e638c9f1e000932f6bb14f778dcdc` context. The map separately records actual auxiliary providers, recovery transformations, and original failed sources: Quota.Plan uses the memory-gated equation at 59df, and Trace.Challenge the complete six-branch carrier at 6c. These are actual source revisions. Reversible field adaptations and unchanged sources have their own records.

G.3 Verification and independent evidence

The complete seven-package build, trust checks, and complete-type checks of 1012 declarations passed with stable before/after inputs. The axioms are `propext`, `Classical.choice`, and `Quot.sound`; `unsafe=0` and `partial=0`. The repository files `evidence/generated-society/acceptance/final-manuscript/summary.json` and `entry.json` fix the accepted scope. The build, trust, and types receipts in that directory, together with `execution-map.json.gz`, preserve the actual execution and original input bytes. The independent acceptance entry is `evidence/economic-constitution/acceptance/initial-intake/independent-acceptance.json`.

The two failed builds, original strict/T1 certifications, and original protocol-window receipts retain their respective statuses. Original certification does not certify the current auxiliary environment; current success does not rewrite an original failure. Reproduction and public/exact source recovery follow H0's `docs/generated-society-reproduction.md`. The full provenance and original-record statuses appear in this paper's source-availability.md.